

Basic Model

Contents

BasicModel	6
Basic Model of Cognition	6
Documentation	6
Overview	6
Files	7
Quick Start	7
XML Configuration	7
Output	8
Installation and Usage	8
Prerequisites	8
Setup	8
Makefile Targets	8
Training	8
Models	9
Testing and Documentation	9
Utilities	9
Makefile Variables	9
train.py Options	9
General	10
Remote Execution (SSH)	10
Environment Variables	10
Remote Training (ArborMini)	10
Workflow	10
SSH Key Setup	10
Quick Start	11
Architecture	11
Overview	11
Spaces	11
Reconstruction Symbols	12
Single Optimizer with Overlapping Weight Spaces	13
Training Loop	13
Modes of operation	13
Pipeline as a unit, two-tier reset	14
Two-File Architecture	14
Language System	15
Short-Term Memory on ConceptualSpace	15
Per-word operational flow	15
Sigma and Pi Layers	16
Dimensionality Constraints	17
Invertible Linear Layer (LDU)	17

Ergodic Exploration	17
Sentence-level AR (InterSentenceLayer)	17
Sentence representation	17
ARMA(p, q) predictor	17
Wiring into the training loop	18
Inference (chat-loop seeding)	18
Configuration	18
Basic Model	19
Inputs and Outputs	19
Perceptual Space	19
Conceptual Space	20
Symbolic Space	20
Reasoning	20
Symmetric Perception: the Single Layer Linear Case	20
Symmetric Perception: the Single Layer Nonlinear Case	21
Distance Metrics	22
Exploration/Exploitation as a function of temperature	22
Transfer Functions	22
Spaces	22
Overview	22
Base Class: Space	23
Reset cascade — hard vs. soft	23
Sigma / Pi ownership (2026-05-13 rebalance)	24
Why the subsymbolic loop fires unconditionally	24
Where lift vs lower lives, if not at the substrate	25
Grammar XML migration	25
Normalization and Ranges	25
Codebook Similarity Metric	26
Euclidean (Perceptual / Symbolic)	26
Dot product (Conceptual)	26
Configuring the metric	26
Codebook Uniqueness Contract	27
Lexicon (Projective Unit Ball)	27
Distance and lookup	27
SBOW training: pole (attractor) and antipole (repulsion target)	28
InputSpace	28
Document streaming and <code>valid_mask</code>	29
AR cursor unfold retirement (2026-05-13)	29
Within-sentence AR retirement (2026-05-14)	29
PerceptualSpace	30
ConceptualSpace	30
ShortTermMemory	31
Lift/Lower factorization	32
SymbolicSpace	32
Monotonicity of the lift / lower chain	34
SyntacticSpace — retired	34
OutputSpace	34
Language	35
Overview	35
Tiers (P / C / S / L)	35
Grammar	36

Accessors	36
Grammar configuration	36
<code>data/grammar.cfg</code> dispatch	37
Syntactic sugar consumption	37
Shamatha speech mode	37
Operator semantics	37
Notation	37
Ternary commitment (<code>true</code> / <code>false</code> / <code>non</code>)	37
Negation (<code>not</code>) — Sym	37
Conjunction / disjunction — S-tier (post-codebook scalar)	38
Intersection / union — L-tier (lattice on bivectors)	38
Part / equals — Def (tensor scoring)	38
Absorb — base-class marker	38
Query — Mereological	38
Swap (Sinkhorn soft permutation) — Sym	38
Lift / lower — rule-id annotators over the shared subsymbolic loop	38
Chunking — Percepts	39
Mereological suite — see Mereology.md	39
GrammarLayer Implementations	39
Base contract	39
Layer summary	40
NotLayer example	40
IntersectionLayer / UnionLayer example	40
PiLayer / SigmaLayer — parametrized folds	41
Class registry	41
Inverse contract by operator	41
Rule prediction	42
Composition	42
Chart inside pass	42
Tensor word buffer	43
Chart-derived SVO	43
POS side-channel	43
Storage tiers	43
Lexical bootstrap	43
Mechanism 1 — RHS POS compatibility mask	43
Mechanism 3 — Rule-prediction conditioning	43
Why this trains POS to every word	43
<code><writeSyntax>true</writeSyntax></code> — syntax-tree dump	44
Per-space dispatch (SyntacticLayer)	44
WordSpace.forwardSymbols / reverseSymbols	44
Downward head emission (S -> C)	44
Projection-coefficient scoring	45
WordSpace.reconstruct(state, codebook_space)	45
Word encoding	45
Decomposition	45
How syntax is trained	45
Two-Layer logic	45
SymbolicSpace integration	46
Forward path	46
Reverse path	46
Key properties	46
AssociationLayer (EQUALS implementation)	46

Parthood as Clipped Cosine Projection	46
The Full Suite	47
Region Partition	47
Mereological Fusion	48
ImpenetrableLayer: Five-Relations Regularizer	48
Penalty	48
Trust	48
Diagnostics	48
Configuration	48
Why This Design	49
Logic	49
Overview	49
1. Subsymbolic Layer	49
Operators	49
2. Symbolization	50
3. Symbolic Layer (Scalars in [-1,1])	50
Parthood as Projection	50
4. Key Insight	51
5. Rationality	51
Truth Statements	51
TruthLayer (WordSpace.truth_layer)	51
Truth Field	52
Consonance and Dissonance	52
Propositional Structure	52
Truth Accumulation Pipeline	52
6. Consistency and Verification	52
Internal Consistency (within a TruthSet)	52
Verification (incoming statement against stored truth)	52
Logical Entailment (symbolic closure)	53
7. Radial Operators for Hypersphere Ternary Logic	53
Luminosity	54
Fusion	55
Supporting measures	55
Semantic Summary	55
8. Cross-references	55
Reasoning System	56
Partitioned Symbolic Space	56
Reasoning Methods	56
isConsistent() -> dict	56
ground(activation, threshold=0.6) -> dict	56
isTrue(activation) -> float	56
extrapolate(grammar, seed_indices, max_new, attenuation) -> dict	56
TruthLoss	56
Bidirectional Reasoning Loop	57
Grammar Learning	57
Architecture Modes: useGrammar	57
Flat (useGrammar="none")	57
Grammar-directed (useGrammar="all")	58
Configuration	58
Contemplative Awareness Methods	58
Testing	59

Training	59
Overview	59
Phase 1: Embedding Pretraining (<code>make train / embed.py train</code>)	59
Pipeline	59
SBOW (Sentence Bag of Words)	59
CBOW (Continuous Bag of Words)	60
Two-Pass Architecture	60
Configuration (Makefile variables)	60
Memory Considerations	60
Phase 2: Network Training (<code>BasicModel.py</code>)	61
Within-sentence training objective (IR-only)	61
<code><trainEmbedding></code> : Embedding Update Mode	61
Gradient Flow Through Codebook	61
JOINT: Single Combined Loss	62
Why MSE over Embeddings (not Cross-Entropy)	62
Training Loop	62
SBOW vs CBOW	63
Three-File Architecture	63
Embedding Space Geometry	63
Profiling	64
Ergodic Exploration via Adaptive Bias-Variance Control	64
1. The Ergodic Weights	64
2. Why Standard Adam Fails on α	65
3. The Gradient Energy Sensor	65
3.1 Modified Second-Moment Estimator	65
3.2 Bias Correction	65
3.3 What $\hat{v}_t$ Measures	65
3.4 The Alpha Update Rule	66
3.5 Layer-Local Certainty (Per-Neuron Sigma)	66
4. Comparison: Adam vs. Gradient Energy Sensor	66
5. Derivation: Why Drop m_t	66
6. Dropout via Bias	67
7. The <code>global_temp</code> Sensitivity Knob	67
8. Initialization and Bootstrap	67
9. Layer Architecture	68
ErgodicLayer Base Class	68
InvertibleLinearLayer — factor-level noise injection	68
10. Training Loop Integration	68
11. Summary of the Full Algorithm	68
Abstract	69
Background	69
Part I: What Machine Minds Are — Weight Ergodicity	69
Weights as Ergodic Samples	69
Temperature and Annealing	70
Advantages of Ergodic Weights	70
Part II: What Machine Minds Feel — Network Invertibility	70
Bidirectional Perception	70
Symbols, Worldlines, and Karmic Inertia	70
Implementation	70
Part III: What Machine Minds Know — Output Certainty	71
Certainty vs. Probability	71
Certainty-Weighted Loss	71
Knowing What You Do Not Know	71

Methods	71
Results	72
Discussion	72
Conclusion	73
References	73
XML Configuration Parameters	73
<architecture>	73
<architecture><data>	74
<architecture><training>	74
<architecture><server>	75
BasicModel / MentalModel Quick Reference	76
Space sections	76
<InputSpace>	76
<PerceptualSpace>	77
<ConceptualSpace>	77
<SymbolicSpace>	77
<OutputSpace>	78
<WordSpace>	78
InvertibleLinearLayer (programmatic)	79
Example: XOR Configuration	79
Example: Embedding / Chatbot Configuration	80

BasicModel

Basic Model of Cognition

The basic model of cognition relies on conceptual hyperplanes and perceptual prototypes to synthesize and analyze the input space. It uses a high-dimensional embedding to characterize mental space and integrates symbolic computation. Its three major operations are intersection (which forms percepts from concepts), union (a bidirectional mapping between concepts and percepts), and equality (where symbols are elements that map across perceptual and conceptual domains).

Documentation

Document	Description
Architecture	Pipeline design, layer types, invertible LDU factorisation
Spaces	All six spaces: Input $\rightarrow$ Perceptual $\rightarrow$ Conceptual $\rightarrow$ Symbolic $\rightarrow$ Syntactic $\rightarrow$ Output
Ergodic	Gradient energy sensor, adaptive exploration, factor-level noise injection
Training	Two-phase training, SBOW embeddings, masked prediction modes
Params	Complete XML configuration reference
BasicModel	Cognitive science foundations
Language	Grammar, word encoding, symbolic and syntactic spaces
Logic	Subsymbolic and symbolic logic operations
Reasoning	Truth methods, bidirectional reasoning, contemplative awareness stubs
MachineMinds	What machine minds are, feel, and know
Installation	Setup, Makefile targets, environment variables

Overview

BasicModel is a parameterized neural architecture that answers the question “what is the truest thing we can say of this experience?”.

Model configurations are specified in XML. See doc/Architecture.md for the full mathematical treatment.

Files

File	Description
bin/BasicModel.py	Main entry point: model factory, training loop, comparison plots, HTML report
bin/Model.py	Layer library: SigmaLayer, PiLayer, ErgodicLayer, LinearLayer, TruthLayer
bin/Space.py	Space classes: InputSpace, PerceptualSpace, ConceptualSpace, SymbolicSpace, OutputSpace, Gram
bin/embed.py	Word vector training: CBOW/SBOW with negative sampling, WordVectors (gensim-compatible .k
bin/SigmaPi.py	Standalone demo of the SigmaPi network solving XOR
data/	XML model configurations
doc/Architecture.md	Algorithm details: Sigma/Pi layers, ergodic exploration, gradient energy sensor
doc/Params.md	Full XML parameter reference
doc/Training.md	Embedding pretraining, CBOW/SBOW, masked prediction, <trainEmbedding> modes
doc/Installation.md	Setup, Makefile targets, train.py options, remote training
test/	Unit tests

Quick Start

```
# Set up virtual environment
make install

# Run a single model
make simple      # data/simple.xml
make ergodic     # data/ergodic.xml

# Compare two models side-by-side
make compare     # defaults: data/simple.xml vs data/ergodic-only.xml
make compare XML1=data/simple.xml XML2=data/ergodic.xml

# Run tests
make test

# Generate PDF documentation
make doc_pdf
```

XML Configuration

Models are configured via XML files in data/. Training and data parameters live in nested sub-elements:

```
<model>
  <architecture>
    <modelType>simple</modelType>    <!-- simple | lm | embedding -->
    <ergodic>false</ergodic>
    <certainty>false</certainty>
    <reconstruct>NONE</reconstruct>  <!-- NONE | symbols | output | both -->
    <maskedPrediction>NONE</maskedPrediction>

    <data>
      <dataset>xor</dataset>         <!-- xor | mnist | text | ... -->
    </data>

    <training>
```

```

    <numTrials>1</numTrials>
    <numEpochs>20</numEpochs>
    <batchSize>10</batchSize>
    <learningRate>0.001</learningRate>
    <weightsPath>output/BasicModel.ckpt</weightsPath>
    <autoload>true</autoload>
    <autosave>false</autosave>
  </training>
</architecture>

<InputSpace> ... </InputSpace>
<PerceptualSpace> ... </PerceptualSpace>
<ConceptualSpace> ... </ConceptualSpace>
<SymbolicSpace> ... </SymbolicSpace>
<OutputSpace> ... </OutputSpace>
</model>

```

See doc/Params.md for the full parameter reference.

Output

Each run produces an HTML report (timestamped in `output/`) containing:

- **Error per Epoch** – training and test loss curves
- **Accuracy per Digit** – per-class accuracy breakdown

In compare mode, additional overlay plots show combined loss and accuracy across models with color-coded legends.

Installation and Usage

Prerequisites

- **Python 3.12** with a virtual environment at `.venv/`
- **PyTorch** with MPS (Apple Silicon) or CUDA
- **pandoc** (optional, for PDF generation via `make doc_pdf`)

Setup

```
make install    # creates .venv with dependencies
```

All Makefile targets invoke Python through `.venv/bin/python`, so no manual activation is required.

Makefile Targets

Training

Target	Description
<code>make train</code>	Full training (Phase 1 embeddings + Phase 2 model), logged to <code>output/logs/</code>
<code>make train_micro</code>	Small run: 500 docs, 1 random shard, logged
<code>make train_remote</code>	Full training on <code>arbormini.local</code> via SSH
<code>make train_micro_remote</code>	Micro training on <code>arbormini.local</code> via SSH

Models

Target	Description
<code>make run</code>	Run <code>BasicModel.py</code> with the config in XML1
<code>make xor</code>	<code>data/XOR_exact.xml</code>
<code>make simple</code>	<code>data/simple.xml</code>
<code>make ergodic</code>	<code>data/ergodic.xml</code>
<code>make tomatoes</code>	<code>data/tomatoes.xml</code>
<code>make mnist</code>	<code>data/mnist.xml</code>
<code>make compare</code>	Side-by-side compare using XML1 and XML2
<code>make SigmaPi / SymPercept / SPNN</code>	Run respective scripts

Testing and Documentation

Target	Description
<code>make test</code>	Unit tests (forces <code>BASICMODEL_DEVICE=cpu</code>)
<code>make bench</code>	Training benchmarks (baseline)
<code>make doc_pdf</code>	Generate <code>BasicModel.pdf</code> via pandoc

PDF chapters in order: `README.md`, `doc/Architecture.md`, `doc/BasicModel.md`, `doc/Spaces.md`, `doc/Language.md`, `doc/Logic.md`, `doc/Ergodic.md`, `doc/Training.md`, `doc/MachineMinds.md`, `doc/Params.md`, `doc/Installation.md`.

Utilities

Target	Description
<code>make clean</code>	Remove generated files (<code>BasicModel.pdf</code>)
<code>make all</code>	Default; alias for <code>make xor</code>

Makefile Variables

Override on the command line, e.g. `make run XML1=data/ergodic.xml`.

Variable	Default	Description
<code>MODEL</code>	<code>data/BasicModel.xml</code>	XML config for training
<code>XML1</code>	<code>data/simple.xml</code>	Primary config for <code>make run</code> / <code>compare</code>
<code>XML2</code>	<code>data/ergodic-only.xml</code>	Secondary config for <code>make compare</code>
<code>TRAIN_HOST</code>	<i>(empty)</i>	Remote training host
<code>TRAIN_USER</code>	<code>arogers</code>	SSH user
<code>TRAIN_KEY</code>	<code>~/.ssh/id_ed25519_arbormini</code>	SSH private key
<code>TRAIN_DIR</code>	<code>~/WikiOracle/basicmodel</code>	Working directory on remote

train.py Options

`train.py` orchestrates Phase 1 (embeddings) followed by Phase 2 (model training).

General

Flag	Default	Description
<code>--model, -m</code>	<code>data/BasicModel.xml</code>	XML config
<code>--max-docs</code>	<i>(from XML)</i>	Override <code>maxDocs</code>
<code>--num-shards</code>	<i>(from XML)</i>	Override <code>numShards</code>
<code>--random-shards</code>	off	Pick random shard indices
<code>--force-embeddings</code>	off	Retrain embeddings even if file exists
<code>--log [FILE]</code>	off	Capture stdout/stderr to <code>.log</code> in <code>output/logs/</code> . Omit filename for times
<code>--profile</code>	off	Run Phase 2 under cProfile; writes <code>.prof</code> to <code>output/profiles/</code>

Remote Execution (SSH)

Flag	Default	Description
<code>--host</code>	<i>(none)</i>	Remote host (e.g. <code>arbormini.local</code>); enables remote run
<code>--user</code>	<code>arogers</code>	SSH user
<code>--key-file</code>	<code>~/.ssh/id_ed25519_arbormini</code>	SSH private key
<code>--remote-dir</code>	<code>~/WikiOracle/basicmodel</code>	Working directory on remote

Environment Variables

Variable	Description
<code>BASIC_MAX_DOCS</code>	Override <code>maxDocs</code>
<code>BASIC_NUM_SHARDS</code>	Override <code>numShards</code>
<code>BASICMODEL_DEVICE</code>	Force device, e.g. <code>cpu</code> . Used by <code>make test</code>
<code>PYTORCH_MPS_HIGH_WATERMARK_RATIO</code>	MPS memory limit; set to 0.0 by <code>train.py</code>
<code>PYTHONUNBUFFERED</code>	Set to 1 by <code>train.py</code> for real-time log streaming

Remote Training (Arbormini)

`train_remote` and `train_micro_remote` set `TRAIN_HOST=arbormini.local` and delegate to `train.py --host`.

Workflow

1. **Rsync** the project tree to `arbormini.local`, excluding `.venv/`, `__pycache__/`, `.DS_Store`, `output/`.
2. **SSH** and run `train.py` remotely with the same flags (minus `--host`):

```
ssh -i ~/.ssh/id_ed25519_arbormini -t arogers@arbormini.local \  
"cd ~/WikiOracle/basicmodel && PYTHONUNBUFFERED=1 PYTHONPATH=bin .venv/bin/python bin/train.py ."
```

3. **Output stays remote** under `~/WikiOracle/basicmodel/output/`.

SSH Key Setup

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519_arbormini  
ssh-copy-id -i ~/.ssh/id_ed25519_arbormini arogers@arbormini.local
```

Quick Start

```
# Sanity check: 500 docs, 1 shard, on ArborMini
make train_micro_remote
```

```
# Full run on ArborMini
make train_remote
```

Architecture

Overview

BasicModel is a bidirectional neural architecture organized as a pipeline of five **spaces** plus a grammar host (WordSpace), each implementing a distinct representational transformation:

Forward: InputSpace -> PerceptualSpace -> ConceptualSpace -> SymbolicSpace -> OutputSpace
Reverse: OutputSpace -> SymbolicSpace -> ConceptualSpace -> PerceptualSpace -> InputSpace

Feedback loops (cross-stage / cross-call):

```
SymbolicSpace ---> ConceptualSpace (S->C symbolic loopback; per stage at order >= 1)
ConceptualSpace ---> PerceptualSpace (C->P subsymbolic loopback; cross-forward)
```

The forward pass transforms raw input into predictions; the reverse pass reconstructs the original input from the symbolic representation. Both directions are trained simultaneously with a single optimizer minimizing a combined loss:

```
totalLoss = (1 - reconRatio) * outputLoss + reconRatio * reconstructionLoss
```

The legacy SubsymbolicSpace class (a parallel-mode bitonic Space alongside SymbolicSpace) and SyntacticSpace (a separate derivation-tree Space) have both been retired. The subsymbolic role is filled by PerceptualSpace itself plus the new $C \rightarrow P$ feedback loop; syntax / grammar dispatch lives on WordSpace and is attached to SymbolicSpace (the canonical grammar host). The MereologicalTree sidecar that backed part / equals / query is also retired — those operations are now pure-geometric clipped-cosine projections on the SymbolicSpace bivector codebook.

Spaces

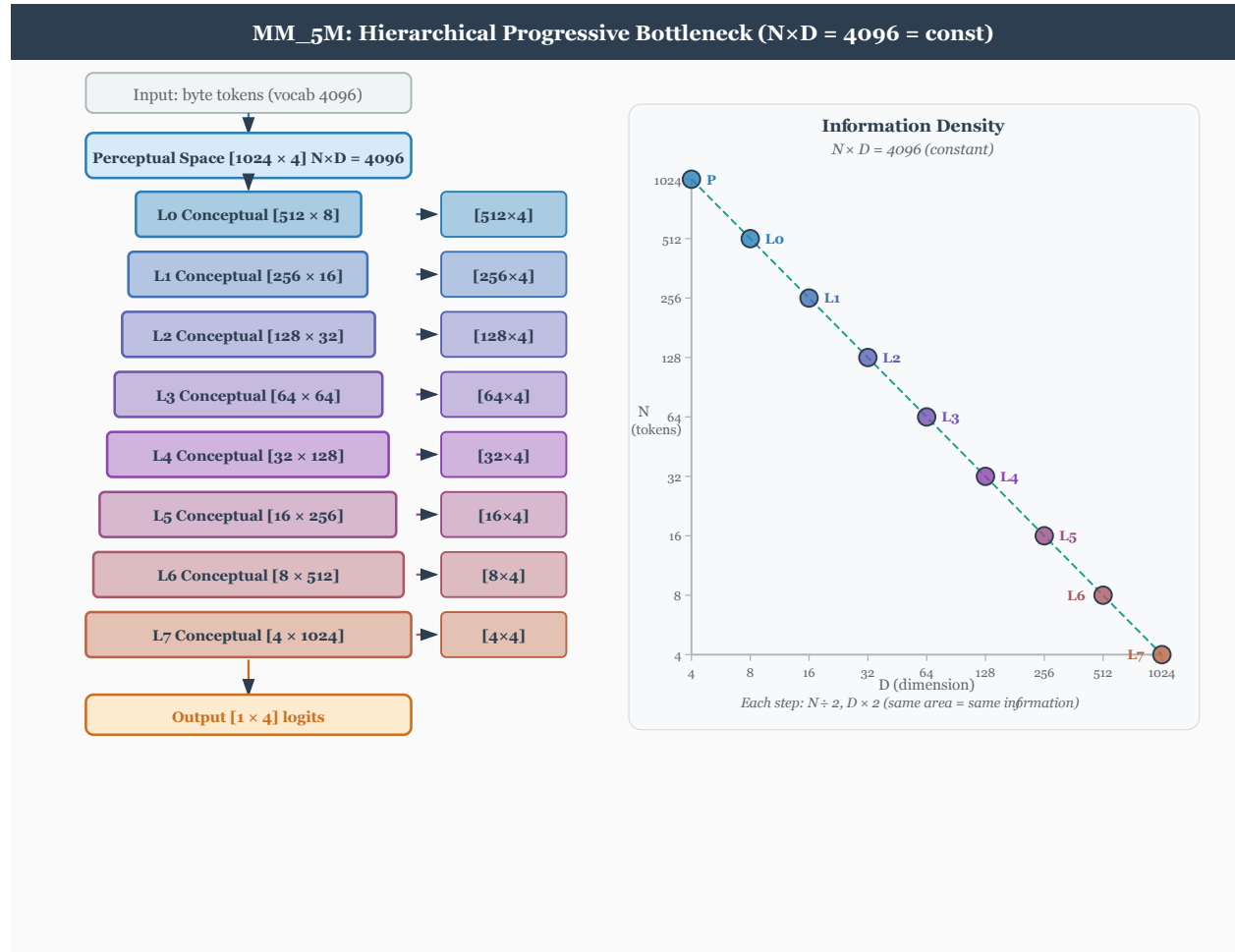
Space	Role	Layer Type
InputSpace	Lifts raw data into working dimensionality	LiftingLayer
PerceptualSpace	Per-percept aggregation; subsymbolic substrate	Owns pi_input (i
ConceptualSpace	Additive concept abstraction ($P \rightarrow C$); host of ShortTermMemory	Owns sigma_perce
SymbolicSpace	Discrete activation bottleneck ($C \rightarrow S$) + Codebook + grammar's calculator	No default sigma o
OutputSpace	Final prediction	LinearLayer

The cross-space fold contract — the canonical composition is:

```
C = sigma_percept( pi_input(IS) + pi_concept(C_prev) )
```

pi_input is the input boundary fold; **pi_concept** carries the subsymbolic $C \rightarrow P$ feedback. Both fire on every PerceptualSpace forward and are summed (no /2 averaging — the previous (**primary** + **c_event**) / 2 in **_sourced_input** is retired). **sigma_percept** then folds the combined P-state into a C-state. See Spaces.md Section “Sigma / Pi ownership” for the cognitive rationale (lift vs lower as rule-id annotations over a shared composition primitive) and the grammar-XML migration table. See Logic.md Section 8 for the algebraic constraints on sigma/pi.

Dimensions (`nDim`) are read from `TheObjectEncoding`. Codebook sizes (`nVectors`) are likewise on `TheObjectEncoding`; the factory validates `nVectors` $\geq$ `nActive`.



Layer selection by `<reconstruct>` and `invertible`:

1. **reconstruct=NONE**: Non-invertible layers (`PiLayer`, `SigmaLayer`) forward-only. No reverse pipeline.
2. **reconstruct=<any> + invertible**: Single invertible layer (`PiLayer(invertible=True)`, etc.) serves both directions, sharing weights.
3. **reconstruct=<any> + not invertible**: Two layers with separate weights — `forward()` on one, `reverse()` on the other. Avoids the expressivity limitation where a non-invertible layer can't represent the inverse of another. Reverse uses matrix `pinv` (may be numerically unstable from SVD convergence). `<invertible>true</invertible>` avoids this via shared-weight inversion.

Reconstruction Symbols

The symbolic bottleneck can lose information needed for reconstruction. XOR maps 2 inputs to 1 output, but `XOR(0,0)=0` and `XOR(1,1)=0` are distinct inputs producing the same output. A single output symbol cannot reconstruct which input was presented.

`nSymbols` is split:

- `nOutputSymbols` = `OutputSpace.nActive` — fed to `OutputSpace` for prediction
- `nReconSymbols` = `nSymbols` - `nOutputSymbols` — carried in `end_state` for reconstruction

A skip connection through the symbolic bottleneck. Reconstruction symbols receive gradient only from reconstruction loss.

Single Optimizer with Overlapping Weight Spaces

The forward and reverse passes share a **single Adam optimizer** that minimizes the combined loss. Forward and reverse weight spaces **partially overlap** — neither disjoint (allowing independent optimizers) nor identical (creating destructive interference). Some layers share weights between directions (shared embeddings, the symbolic bottleneck); others are direction-specific (`pi1/pi2`, `sigma1/sigma2`, `linear1/linear2`).

- **Shared weights** receive gradient from both losses, learning representations useful in both directions.
- **Direction-specific weights** specialize without interference.
- **No ping-pong**: separate optimizers on overlapping parameters would pull weights in alternating, conflicting directions each step.

When `invertible=true`, overlap is total: one invertible layer serves both directions and receives the full combined gradient.

Reference: A.M. Rogers, T.T. Shannon, and G.G. Lendaris, “A comparison of DHP based antecedent parameter tuning strategies for fuzzy control,” *Proceedings Joint 9th IFSA World Congress and 20th NAFIPS International Conference*, 2001, doi: 10.1109/NAFIPS.2001.944317.

Training Loop

Single Adam optimizer with persistent state (momentum/variance accumulate across epochs):

1. Forward pass: input $\rightarrow$ prediction + `end_state`
2. Compute `outputLoss` from prediction vs. target
3. Reverse pass: `end_state` $\rightarrow$ reconstructed input
4. Compute `reconstructionLoss` from reconstruction vs. original input
5. Backpropagate combined `totalLoss`
6. If ergodic: run `paramUpdate()` (gradient energy sensor updates alpha)
7. Optimizer step (embedding params excluded when `trainEmbedding` is NONE, CBOW, or SBOW)
8. If `trainEmbedding` is CBOW, SBOW, or BOTH: run embedding update step

Alpha annealing (ergodic): starts at 1.0 (full exploration), decays to 0.0 within the first 5% of epochs via `alpha = max(0, 1 - epoch / warmup)` where `warmup = numEpochs // 20`. Code convention (`alpha=1` means explore) is the inverse of the ergodic math convention; layers translate internally.

See Params.md for all XML parameters. See Training.md for embedding modes.

Modes of operation

Two orthogonal mode dimensions plus the two feedback loops.

Serial mode (`BASICMODEL_DEVICE` / runtime flag `serial_mode`): Runtime fast path for streaming / autoregressive contexts. `PerceptualSpace` / `ConceptualSpace` may use slide-and-recompute with the previous step’s per-cell warm cache (`subspace.serial_cache`). Cache is keyed on owner-Space id, cleared on hard `Reset`. (Distinct from the **serial / shift-reduce parser** — a separate deferred refactor; see `doc/plans/`.)

The two feedback loops: The architecture now exposes two concurrent recurrent paths between the spaces. Their iteration cadences differ — they do NOT both follow `conceptualOrder`.

Loop	Direction	Iterations per forward call
Symbolic loopback	$S \rightarrow C$	<code>T = conceptualOrder</code> (each stage $t \geq 1$ reads stage $t-1$ ’s S event)
Subsymbolic loopback (new)	$C \rightarrow P$	Exactly 1 per forward call (stem P reads <code>conceptualSpaces[-1].event</code> before)

Conceptual input sourcing: each stage’s `ConceptualSpace.forward` combines two sources via `_sourced_input` — its own perceptual primary (stem-cached) AND the previous stage’s `SymbolicSpace.event`,

lifted from the bivector handoff `[B, V_S, 2]` back to concept content `[B, V, concept_dim]` via `Codebook.project_reverse`. The combination is additive after the bivector lift. Stage 0 cold-starts (no S sibling event yet); the previous-stage symbolic loopback fires from stage 1 onwards.

The new $C \rightarrow P$ loopback at PerceptualSpace works the same way (mirror of the C-side mechanism): P's `_sourced_input` reads its primary input plus the terminal stage's `ConceptualSpace.event` (lifted via the C-tier codebook reverse), averaged when shape-compatible. Bivector- gated: if the C-tier codebook is not present or the event's last dim isn't 2, the loopback no-ops and P uses its primary input alone.

Pipeline as a unit, two-tier reset

`runBatch` is a pure compute brick: forward $\rightarrow$ loss $\rightarrow$ backward $\rightarrow$ optimizer.step. It does **not** decide when to reset per-row state, does **not** consume `_end_of_stream` for control flow, and (after Section 6 vectorization) does **not** issue any GPU $\rightarrow$ host sync inside the brick.

Reset lives in `runEpoch`. The same loop drives both byte cursor (AR text byte) and trial cursor (non-AR); `next_tick` is universal dispatch:

```
while not ds.all_done():
    inp, out, hard_eos = ds.next_tick()           # 3-tuple, host-side
    runBatch(inp, out)                           # compute brick
    flush_word_buffers()                         # materialize subspace.word
    dispatch_per_row_reset(hard_eos)             # hard resets
    dispatch_soft_reset()                       # grammar <start> reductions
    post_tick_compact()                         # truth_layer.compact
```

For AR text byte, `inp` is a byte slab and `hard_eos[b]` flips True when row b's cursor exhausts a doc. For non-AR / numeric data, each tick yields one batch of trials with `hard_eos = [True] * B`.

Hard reset. `TheData` walks each document one slab of ≤ 1024 bytes at a time. `hard_eos` flips True on cursor exhaustion. Full row-state cascade fires for that row only; other rows continue mid-document with state preserved.

Soft reset. `Chart.compose` signals when a row's parse reduces to `<start>`. `wordSpace._sentence_completed` is drained per-tick: re-arms `_stm_fired[b]`, clears `_last_svo[b*K..]` and parse-stack rows for b, but **preserves discourse history** (discourse accumulates across sentences within a document and clears only on hard reset).

No truncation. Documents longer than `slab_bytes` span multiple ticks; concatenating per-tick slabs for any row reproduces the original document byte-exact. `valid_mask: [B, K]` handles partial-fill tails via NULL-padding.

Compute-brick contract. No `.item()`, no `.tolist()`, no Python conditional on a tensor value, no GPU $\rightarrow$ host copy inside `runBatch`. Two residual `.tolist()` calls in `Chart._chart_inside` (`best_pair`, `best_rule_local`) plus a few data-dependent control points produce graph breaks; deferred to GB10-side capture wiring.

Two-File Architecture

File	Contents
XML config (e.g. <code>BasicModel.xml</code>)	Architecture, hyperparameters
Weights checkpoint (e.g. <code>BasicModel.ckpt</code>)	Full integrated bundle: model parameters, register-buffer state, embedding

The 2026-05-12 *integrated-weights* refactor retired the separate `.kv` embedding artifact: embeddings, vocabulary mappings, and the BPE codebook now ride inside the single `.ckpt` bundle alongside the model's other parameters. The bundle layout is:

- `state_dict`: every `nn.Parameter` and `register_buffer` in the module tree (model weights, `wv._vectors`, `TruthLayer.truths`, etc.) — serialised by the normal PyTorch path.
- `vocab_extras`: the WordVectors Python-side mappings that don't live in `state_dict` (`index_to_key`, `counts`, `total_count`).
- `bpe_extras`: the `ChunkLayer`'s pure-Python state (merges list, vocab dict, `id_to_bytes`, growth cursors). Required because `ChunkLayer` stores its merge table as Python dicts/lists, not tensors.

`bin/embed.py` still produces standalone `.kv` artifacts for CBOW/SBOW *pre-training* studies, but those artifacts are no longer part of the runtime artifact set. Cold-start training initialises the vocabulary and BPE codebook from scratch and learns them end-to-end alongside the model weights.

Language System

The chart-driven grammar runs at the S tier. `SymbolicSpace` maps continuous concept activations to a discrete bivector codebook via the $C \rightarrow S$ `SigmaLayer` + Codebook snap, and `WordSpace` (the grammar host) attaches a `SyntacticLayer` to S that dispatches the chart's per-cell rules over the S bivector activation.

`SymbolicSpace` plays the role of the architecture's **calculator**: the chart at S reads operand cells, invokes the grammar layer's compose / forward / reverse, and writes the result back. The dispatch is reactive to the chart's rule queue rather than running unconditionally.

All compositional operations live on the S tier over the bivector-shaped `SymbolicSubSpace`. The S-tier productions include:

- **Codebook-bivector unary / binary operators** (geometric, propositional): `not(S)`, `non(S)`, `intersection(S, S)`, `union(S, S)`, `conjunction`, `disjunction`, `swap`, `copy`, `true`, `false`.
- **Mereological operators**: `part(S, S)`, `equals(S, S)`, `query(S, S)`. All three are now pure-geometric — the `MereologicalTree` sidecar that formerly stored explicit parent / equality links has been retired in favour of clipped-cosine parthood on the bivector codebook. See `Mereology.md`.
- **Bivector lift / lower** (`lift(VP, NP)` / `lower(VP, NP)`): these factor the substrate's sigma / pi via an elementwise gate at C-tier whose values come from VP's symbolic codebook row. The same shared $L \cdot U$ basis on the substrate sigma (or pi) is reused across every VP; the per-call gate `VP_c * NP_c` (elementwise multiplicative) is what makes different VPs produce different transformations. See `Spaces.md` and `Layers.md`.

Parthood (`part`) is the **fundamental** mereological operation, realized as clipped cosine projection on the bivector symbol subspace. The full suite (`whole`, `equal`, `overlap`, `underlap`, `boundary`) composes through `part` on `Basis`. `equals(S, S)` is propositional identity on S; delegates to `Basis.equal`.

Short-Term Memory on ConceptualSpace

`ConceptualSpace.stm` (an instance of `ShortTermMemory`) is a per-batch stack of unquantized C-tier “ideas” — the continuous compositions that accumulate as the per-word stem processes words. Capacity auto-sizes to `<WordSpace><wMax>` (the chart's sentence-length bound), so each word fills a slot before the chart fires at C-tier in the body; an explicit `<ConceptualSpace><stmCapacity>N</stmCapacity></ConceptualSpace>` still overrides for subsymbolic configs that want a larger buffer. The STM is cleared on hard `Reset` (sentence boundary) and survives soft reset. The body's chart fires over `stm.snapshot()` at every stage. See `Spaces.md`.

Per-word operational flow

Each word traverses a per-word round trip through the full pipeline, so that a single word ends up on the STM as a single post-quantized idea. The shape:

```
byte stream -> P (BPE lex + per-percept features)
              -> C (project lexed word onto concept space)
```

```

-> S (codebook snap = word identity + POS)
-> C (S->C reverse, the unquantized "idea")
-> ConceptualSpace.stm.push(idea)

```

This per-word cadence runs **inside the stem** before the body's per-stage chart firing. The body then iterates over its `body_stages` ModuleList, calling `_chart_compose_at_C` over `stm.snapshot()` at every stage so the chart's per-rule selections shape SymbolicSpace dispatch. The reverse mirror (`_chart_generate_from_stm`) fires at the symmetric C-tier point inside the reverse pipeline.

POS rides the codebook for free. The SymbolicSpace bivector codebook carries two POS-bearing fields per atom: `category_ids`: [V] (hard POS tag — one of the grammar's nonterminals, e.g. NP / VP / N / V / ADJ) and `category_logits`: [V, C] (learnable soft POS distribution per atom, EMA-updated by the chart's `_apply_codebook_pos_seed` mechanism in Language.py:2638). So per-word snap returns (`word_id`, `POS`) simultaneously — no separate POS tagger needed. The parser uses POS for typing reduce candidates ($NP + VP \rightarrow S$, etc.); POS is *learned through parsing* alongside the codebook.

See Logic.md, Mereology.md, and Language.md.

Shamatha Speech target. Planned narrow grammar for one-pointed object speech: complete DNF over active percepts, permitting each **conjunction** / **disjunction** only when operands' `where()` supports are connected and `when()` supports are continuous. See Language.md.

Sigma and Pi Layers

For weight matrix $W \in \mathbb{R}^{m \times n}$ and input $x \in \mathbb{R}^n$:

Sigma layer:

$$y_j = Wx + b = b_j + \sum_{i=1}^n W_{ji}x_i$$

Pi layer (log-space linear):

$$s_i = \log \frac{1 + x_i}{1 - x_i} = 2 \operatorname{atanh}(x_i)$$

$$z_j = \sum_i W_{ji} s_i + b_j$$

$$y_j = \frac{e^{z_j} - 1}{e^{z_j} + 1} = \tanh(z_j/2)$$

Forward maps $[-1, 1] \rightarrow (0, \infty)$ via `_to_mult`, log, linear, exp, `_from_mult`. Domain and range both $[-1, 1]$. Reverse inverts each step: `_to_mult(y)`, log, $W^{-1}(z - b)$, exp, `_from_mult`.

Motivation. The classical product form $y_j = b_j \prod_i (1 + W_{ji}x_i)$ becomes, after taking logs, a sum. The code moves into a log-multiplicative domain via `atanh`, performs a linear op there, returns via `tanh`. The `atanh` transform stretches values near ± 1 toward infinity, making the layer sensitive to strong activations.

Monotonicity of the bivector chain. When `<bivectorOutput>true` is configured on P/C/S, every activation lives on the non-negative paired-index cone $[0, 1]^{\{2K\}}$. Pi/Sigma select `NonNegativeInvertibleLinearLayer` (or `NonNegativeLinearLayer`) under `monotonic=True`, giving $W \geq 0$. Positive matrices are the canonical monotone operators on the positive cone, so every lift / lower preserves parthood pole-by-pole. PerceptualSpace joins via the Q2 promotion $(aP, aN) = (\max(0, x), \max(0, -x))$. See Spaces.md.

Dimensionality Constraints

- Input layer output dim = perceptual layer output dim (conceptual operates on both).
 - Symbolic layer input dim = perceptual layer input dim (both operate on conceptual output).
 - Output layer input dim = sum of symbolic layers' output dims.
-

Invertible Linear Layer (LDU)

Factors $W = L \cdot D_{\text{embed}} \cdot U$:

- L : unit lower-triangular ($nIn \times nIn$, diagonal = 1).
- D : diagonal vector of length $\text{rank} = \min(nIn, nOut)$, embedded into $[nIn, nOut]$ by zero-padding.
- U : unit upper-triangular ($nOut \times nOut$).

Exact inverse via triangular solves: $W^{-1} = U^{-1} \cdot D^{-1} \cdot L^{-1}$. Each factor inverted by `torch.linalg.solve_triangular`. No SVD; inverse exact when all D entries are nonzero. Parameter count: $nIn^2 + \text{rank} + nOut^2$. Initialized at $L = I, d = 1, U = I$ (identity).

`naive=False` (default) applies L/D/U sequentially without materialising W_{eff} as a full matrix. `naive=True` materialises W_{eff} and its inverse.

Ergodic noise injection at the factor level, plus the `stable=True` clamp and the noise lifecycle, are documented in `Ergodic.md`.

Ergodic Exploration

See `Ergodic.md`.

Sentence-level AR (`InterSentenceLayer`)

Within-sentence training is IR-only (BERT-style masked-LM at the P-tier; see `doc/Spaces.md` Section “Within-sentence AR retirement”). The **autoregressive** signal in this architecture lives one scale up: between sentences, on a per-sentence representation `s_t`. That’s the job of `InterSentenceLayer` (alias `wordSpace.discourse`).

Sentence representation

`s_t` is the **root S-tier slot** of the body’s final stage: the single vector the start-symbol reduction wrote into. The chart’s parse trace already commits to this slot at sentence end; the layer pools $[B, N, D] \rightarrow [B, D]$ by taking row 0 (root). Width is `sentence_dim = n_dim` (one vector per row), **not** the full `n_symbols * n_dim` flatten that the pre-2026-05-14 contrastive layer used — that broader rep would have blown the predictor’s Linear past the allocator budget on MM_5M_bivector-scale configs.

ARMA(p, q) predictor

`InterSentenceLayer` runs an autoregressive moving-average predictor:

```
s_hat_t = predictor(s_{t-1..t-p}, e_{t-1..t-q})
e_t      = s_t - s_hat_t
loss     = MSE(s_hat_t, s_t)           # accumulated per batch
```

- `p` = AR lag count (default 5) — last `p` sentence reps.
- `q` = MA lag count (default 2) — last `q` prediction errors.

- `predictor = nn.Sequential(Linear(p*D + q*D, H), Tanh, Linear(H, D))`, with $H = \min(1024, 2 \times \text{sentence_dim})$.

The MA term lets the predictor correct for systematic bias in the AR extrapolation: if the AR model consistently under-predicts the sentence rep, the residual `e_t` carries that signal forward.

Buffers (per row, non-persistent):

- `_s_history`: [B, p, sentence_dim] ring of last p sentence reps (most recent at index -1).
- `_e_history`: [B, q, sentence_dim] ring of last q residuals.
- `_s_count` / `_e_count`: [B] long, fill levels (cap at p / q).

`ensure_batch(B)` resizes these on cascade from `WordSpace.ensure_batch`; `Reset()` clears them on hard / discourse boundary. Default behaviour is to **not** auto-reset across document boundaries — the AR lags carry information through discourse continuity unless the caller explicitly calls `Reset`.

Wiring into the training loop

1. After the body finishes (sentence end), `_forward_per_stage` stashes the S-tier event on `_current_discourse_s`.
2. In `runBatch`, when training, `discourse.observe(s_tensor)`:
 - Pools `s_t = sigma_S(s_tensor[:, 0, :])`.
 - Computes `s_hat_t = predictor(_s_history, _e_history)`.
 - Returns `MSE(s_hat_t, s_t)` (None on the first call per row when the ring is empty).
 - Computes `e_t = s_t - s_hat_t`, pushes both into the rings.
3. `runBatch` adds the loss to `TheError` under category "discourse" with weight `armaScale` (training XSD knob, default 0.1).

Inference (chat-loop seeding)

`BasicModel.generate_sentence(seed_text)`:

1. Calls `discourse.predict_next()` for the ARMA-predicted sentence-rep prior `s_hat_{t+1}`.
2. Lifts it through `discourse.cast` to `concept_dim` and stages on `ConceptualSpace._c_prior`.
3. Runs the IR forward — the body's first `sigma_percept` output gets `_c_prior` summed in as a sentence-level conditioning bias before the codebook lookup. Cleared after the forward consumes it.
4. The post-body perceptual event is decoded by nearest-neighbour against the perceptual codebook, producing the (slot, original, predicted) triples for the seed text's masked positions.
5. Commits the produced sentence's S-tier root to the ARMA ring via `discourse.observe(s_tensor)`.

The IR head plays no role at inference — the prediction lives at the masked P-tier positions, decoded against the (frozen) perceptual codebook.

Configuration

XSD knob	Section	Default	Notes
<code><armaP></code>	<code><WordSpace></code>	5	AR lag count
<code><armaQ></code>	<code><WordSpace></code>	2	MA lag count
<code><armaHiddenDim></code>	<code><WordSpace></code>	$2 \times \text{sentence_dim}$ (cap 1024)	predictor hidden width
<code><armaScale></code>	<code><architecture><training></code>	0.1	ARMA loss weight added to <code>TheError</code>
<code><sentencePrediction></code>	<code><architecture><training></code>	false	Gates <code>InterSentenceLayer</code> constr

The retired pre-2026-05-14 knobs (`<sentenceContextWindow>`, `<sentenceCentroidHistory>`, `<sentenceLambda>`, `<sentencePredictionScale>`, `<sentencePredictiveScale>`, `<sentenceContrastiveScale>`) shaped the legacy contrastive cosine machinery (recent-centroid attraction + older-centroid repulsion). They are not parsed; configs that still set them are tolerated silently.

Basic Model

The Basic Model of Cognition is a neural network of four spaces: real space, perceptual space, conceptual space, and symbolic space.

Model Entities:

Entity	Meaning
$x, \hat{x}, X$	Actual and predicted input (experiences), input space
$y, \hat{y}, Y$	Actual and predicted output (actions), output space
p, P	Percepts, perceptual space
c, C	Concepts, conceptual space
s, S	Symbols, symbolic space
W_e	Experiencing, experiential weights
W_a	Acting, action weights
W_k	Knowing, knowledge weights
W_t	Thinking, thought weights

Background

Based on the Basic Model of Cognition at cognitivesciencesociety.org/framework-cognitive-science; see also The Whole Part.

Inputs and Outputs

Inputs and outputs derive from a real space that is ineffable. Adaptable parameters initialize at zero (*tabula rosa*); learning is driven by random perturbations of the map relative to its territory at *temperature*.

Inputs scale to $[-1, 1]$ via global data min/max. The signed range represents presence and absence symmetrically. Paraphrasing Augustine, *negative is the privation of the positive*.

Action is a function of perceptual space and our expectation, filtered by conceptual projections. Performance errors are defined relative to action, desired effect, and exhaustion. Expectation errors are defined relative to perception and perception-as-reconstructed.

To accommodate a changing input context window, input is augmented with relative spatiotemporal encoding by extending its dimensionality. The attention mechanism treats what and where pathways separately.

Perceptual Space

Perceptual spaces are composed of perceptual prototype vectors, or *percepts*.¹ Percepts that do not correspond to objects are *hallucinations*.

Percepts form a Voronoi tessellation using prototype and/or exemplar vectors, related via Self Organizing Map connections, operating in parallel as prototype-theory categories.

Perceptual accuracy is the distance between inputs and prototype vectors — similar to cosine similarity, becoming a simple distance measure for poorly known percepts.

Attraction and aversion warp perceptual space: attraction is a pole at the percept's location, aversion is a zero. Perfectly recognized percepts lie on the unit hypersphere within conceptual space. Two percepts can be joined into one concept without loss.

¹Perceptual space is known as “embedding space” in transformer literature.

Conceptual Space

Concepts create boundaries in conceptual space, grounded in the support vectors of perception. Concepts support logical negation; they may refer to negative entities. Conceptual spaces are capable of representing negative entities. Concepts that do not correspond to percepts are *false*.

Concepts collect percepts into fuzzy sets — linear separating hyperplanes partitioning conceptual space, or sets defined as linear combinations of percept activations.

Concepts are relative: meaningful only insofar as they partition a meaningful space. As concepts approach certainty, decision boundary slope becomes infinite. A single unknowing state is common to all knowledge vectors (activation slope zero).

Conceptual spaces are similarity spaces partitioned by decision boundaries imposing positivity and negativity on a hypersphere of knowing.

Within conceptual space, attraction/aversion to concepts expresses as hyperplane bias. Certainty is the slope of the activation function (tuned after other parameters stop changing).

A concept can be split into two percepts without loss.

Symbolic Space

Symbolic Spaces are composed of percepts that reference concepts — a subtype of perceptual spaces mapping onto conceptual spaces.

Symbols have no spatial context: zero-dimensional points within perceptual space. In humans, perceptual space projects onto a 3D cerebellar subspace before symbolic encoding.

Symbols are independent, permanent, and integral (as references). The top-down serial activation differs from bottom-up perceptual activation — it casts shadows.

The formation of symbols increases the dimensionality of conceptual space (outer product); the formation of concepts from percepts decreases the dimensionality of perceptual space (inner product).

Symbols are erroneous if their associated concept is not true. Participation in multiple sets creates structural relations that replace positional information.²

Reasoning

Reasoning in symbolic spaces involves computing with parts and wholes — Venn-diagram computations enabling dynamic Bayesian-type statistics. Learned probabilities can be stored as unidirectional connection weights between concepts, possibly with part-whole structure imposed by corresponding symbols.

Symmetric Perception: the Single Layer Linear Case

Perception is bidirectional: we see the world and the world sees us. The egocentric/cultural view treats perception as passive; in neural networks this manifests as forgetting the influence of outputs on inputs.

Symmetric Perception finds $f(x)$ minimizing MSE with respect to:

$$\begin{pmatrix} in \\ out \end{pmatrix} \cdot \begin{pmatrix} f(x) & 0 \\ 0 & f^{-1}(x) \end{pmatrix} = \begin{pmatrix} out \\ in \end{pmatrix}$$

In the linear case, $f(x)$ becomes a single invertible matrix A . When the mapping is nonlinear and an inverse may not exist:

²For example, $\{0,1\}$ and $\{0,1\}$, if orthogonal, pose no constraint on the location of the 1 in their 2x2 grid. However, $\{0,1,1\}$ and $\{0,1\}$ partition a 3x2 space and require two 1's on the same row — a fact present in neither set alone.

$$\begin{pmatrix} in \\ out \end{pmatrix} \cdot \begin{pmatrix} f(x) & 0 \\ 0 & g(x) \end{pmatrix} = \begin{pmatrix} out \\ in \end{pmatrix}$$

This resembles $Ax = b$, except x is known and we want weights A mapping one space to another. The standard LLS solution $x = (A^\top A)^{-1} A^\top b$ becomes finding A such that $Ax = b$ and $A^{-1}b = x$.

Retrocausality suggests an additional objective: A should be optimal for both forward and inverse transforms. Approximately: A as a product of two matrices, each performing a perfect surjection: $A \cdot A^{-1} = I$.

$$\left(\frac{b}{2} * x^{-1}\right) \cdot \left(\frac{x}{2} * b^{-1}\right) = I$$

Best A requires iterative solution (NN learning, ping-pong style) or Gram-Schmidt orthogonalization with respect to A and A^{-1} .

Symmetric Perception: the Single Layer Nonlinear Case

If $f(x)$ and $g(x)$ share weight matrix W :

Forward computation and gradient:

$$\begin{aligned} \hat{y} &= f(g^{-1}(x)W) \\ e_y &= \frac{1}{2}(\hat{y} - y)^2 \\ \Delta W_y &= \eta(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x) \end{aligned}$$

Reverse computation and gradient:

$$\begin{aligned} \hat{x} &= g(W^\top f^{-1}(y)) \\ e_x &= \frac{1}{2}(\hat{x} - x)^2 \\ \Delta W_x &= \eta(\hat{x} - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y) \end{aligned}$$

Since W is shared, summing the partial derivatives gives:

$$\frac{\partial e_y}{\partial W} + \frac{\partial e_x}{\partial W} = \eta(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))g^{-1}(x) + \eta(\hat{x} - x)\delta_g(W \cdot f^{-1}(y))f^{-1}(y)$$

Setting gradient to zero, both errors going to zero when $x = i$ and $y = o$. But $xy - iy = ox - yx$ may hold — simultaneous gradient descent ping-pongs. A better cost function multiplies the two error partials — intersection of the spaces satisfying both:

$$\frac{\partial e_y}{\partial W} \frac{\partial e_x}{\partial W} = \eta(\hat{x} - x)(\hat{y} - y)\delta_f(W^\top \cdot g^{-1}(x))\delta_g(W \cdot f^{-1}(y))f^{-1}(y)g^{-1}(x)$$

Distance Metrics

$$d(x_1, x_2) = \frac{\|x_1\|^n \cdot \|x_2\|^n}{\|x_1 - x_2\|^{2n}}$$

Z-plane form:

$$d(x_1, x_2) = \|x_1 - x_2\| \cdot \frac{(x - z_1)}{(x - z_2)}$$

where z_1 is a zero and z_2 is a pole.

Exploration/Exploitation as a function of temperature

$$y = x \cdot \frac{(W + rt)}{\|W\|}$$

where r is a random vector and t is temperature.

Transfer Functions

Using rotation matrices with tunable Θ before and after a diagonal projection restricts the solution space to an eigendecomposition of the orthonormal basis (an SVD). For 2D:

$$y = \begin{bmatrix} \cos \theta_1 & -\sin \theta_1 \\ \sin \theta_1 & \cos \theta_1 \end{bmatrix} \begin{bmatrix} \Sigma_1 & 0 \\ 0 & \Sigma_2 \end{bmatrix} \begin{bmatrix} \cos \theta_2 & -\sin \theta_2 \\ \sin \theta_2 & \cos \theta_2 \end{bmatrix} x$$

With $f(x) = \sin(x)$ and $g(x) = \cos(x)$:

$$\begin{aligned} \hat{y} &= \sin(\cos^{-1}(x)W) \\ \hat{x} &= \cos(W^\top \sin^{-1}(y)) \\ \Delta W_y &= \eta(\hat{y} - y) \cos(W^\top \cdot \cos^{-1}(x)) \cos^{-1}(x) \\ \Delta W_x &= \eta(\hat{x} - x) \sin(W \cdot \sin^{-1}(y)) \sin^{-1}(y) \end{aligned}$$

The reverse perception reconstruction and the forward perception gradient share common terms. Taking the product:

$$\Delta W_y \Delta W_x = \eta(\hat{x} - x)(\hat{y} - y) \delta_f(W^\top \cdot g^{-1}(x)) \delta_g(W \cdot f^{-1}(y)) f^{-1}(y) g^{-1}(x)$$

Spaces

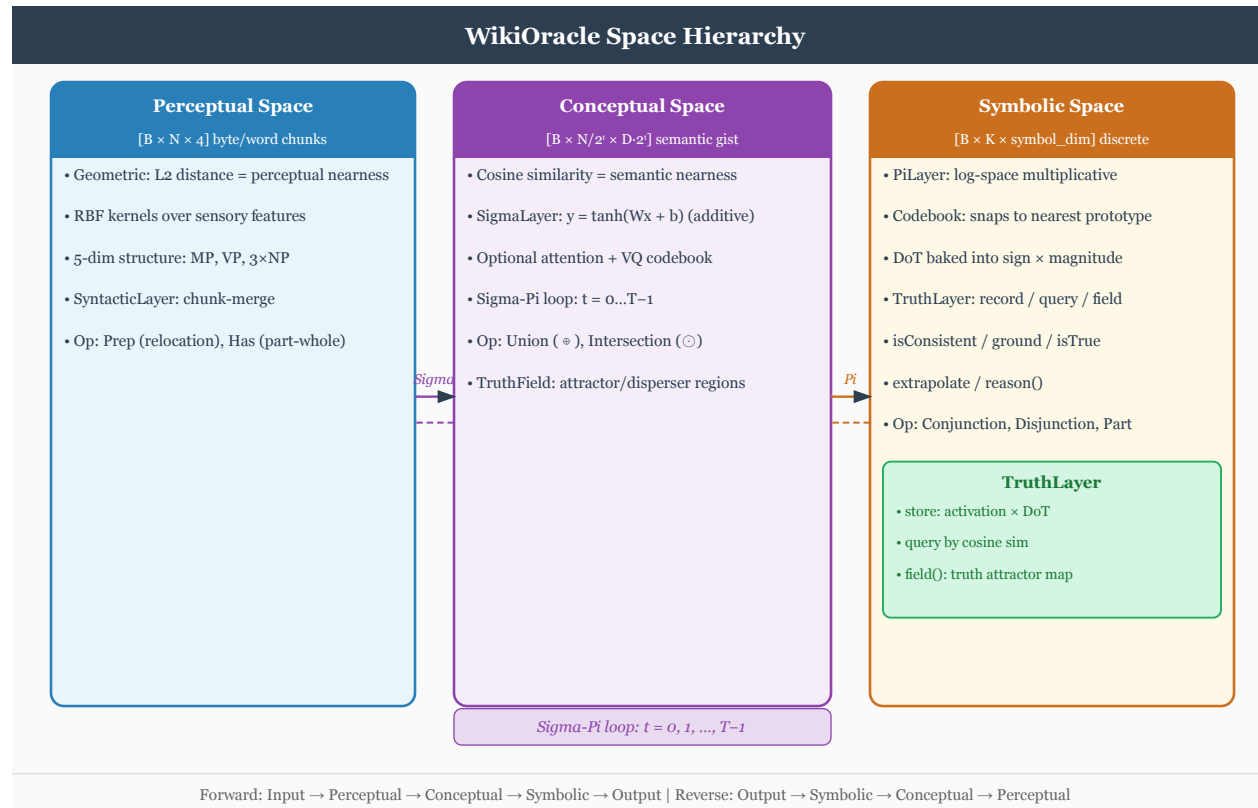
Overview

BasicModel is a pipeline of five **spaces** plus a grammar host (**WordSpace**), each performing a distinct representational transformation. Data flows forward from raw input to task output; the reverse pass reconstructs the original input from the symbolic representation. Two feedback loops connect the pipeline back upward: $S \rightarrow C$ (per-stage symbolic loopback at order ≥ 1) and $C \rightarrow P$ (cross-forward subsymbolic loopback). The legacy **SubsymbolicSpace** and **SyntacticSpace** classes have been retired — the subsymbolic role is filled by **PerceptualSpace** plus the new $C \rightarrow P$ feedback, and the grammar runs from **WordSpace**'s **SyntacticLayer** attached to **SymbolicSpace** (the canonical grammar host / “calculator”).

Forward: InputSpace -> PerceptualSpace -> ConceptualSpace -> SymbolicSpace -> OutputSpace
Reverse: OutputSpace -> SymbolicSpace -> ConceptualSpace -> PerceptualSpace -> InputSpace

Feedback:

SymbolicSpace ---> ConceptualSpace (S->C, per stage when bivectorOutput active)
ConceptualSpace ---> PerceptualSpace (C->P, cross-forward; bivector-gated)



Base Class: Space

All spaces inherit from `Space`, which manages:

- Shape management.** `inputShape` / `outputShape` as $[n\text{objects}, n\text{Dim}]$. Subclasses read dimensions from `TheObjectEncoding`.
- Codebook / VQ quantization.** When `nVectors` > `nActive`, a codebook holds candidate vectors; top-k selection gives the bottleneck.
- Reshape flag.** When in/out object counts differ, the $[B, n\text{Obj}, n\text{Dim}]$ tensor is flattened before the next space and restored on the way back.
- Attention.** Optional `hasAttention=true` reweights objects.
- set_sigma propagation.** Ergodic-mode noise level cascades from the top-level model down through every child layer.

Reset cascade — hard vs. soft

Every space exposes `Reset(batch=None, hard=True)`. The signature is required (legacy zero-arg fallback removed).

Call form	Scope	Use
<code>space.Reset()</code>	All rows	Whole-state wipe (epoch boundary)
<code>space.Reset(batch=b, hard=True)</code>	Row <code>b</code> only	Document boundary
<code>wordSpace.soft_reset(batch=b)</code>	Row <code>b</code> sentence-scoped state	Grammar <code><start></code> reduction

Hard-reset clears: parse stack, `_last_svo`, `_stm_fired`, codebook commit accumulator, discourse history, `serial_cache`, `_ar_embedded`, `_end_of_stream` for the affected rows.

Soft-reset clears per-sentence working buffers (parse stack rows, `_last_svo[b]`, category and reconstruction stacks) and re-arms `_stm_fired[b]`. Does **not** touch discourse history (`InterSentenceLayer` ring buffer) or codebook EMA — those are document-scoped.

Reset is dispatched from `runEpoch`, never from inside `runBatch` (the pure compute brick — see Architecture.md).

Sigma / Pi ownership (2026-05-13 rebalance)

The two composition operators sigma and pi live on the spaces in a **fixed isomorphic pattern**. Each space owns only the operator that fits its tier; the cross-tier feedback loops thread through those operators unconditionally; the **chart’s role is to invoke the loop, not to dispatch a separate substrate per rule**.

Space	Owns	Used in
PerceptualSpace	<code>pi_input (input_dim -> percept_dim)</code> and <code>pi_concept (concept_dim -> percept_dim)</code>	both fire
ConceptualSpace	<code>sigma_percept (percept_dim -> concept_dim)</code> only	fires un
SymbolicSpace	(none)	gramm

Composition (the end-to-end fold from input to concept):

```
C = sigma_percept( pi_input(IS) + pi_concept(C_prev) )
```

where `C_prev` is the prior C-tier event fed back via the subsymbolic loop. At the very first forward of a sentence, `C_prev` is zero/empty, so `pi_concept(0) = 0` and the formula degenerates to `C = sigma_percept(pi_input(IS))`.

Why the subsymbolic loop fires unconditionally

The earlier design dispatched `pi_concept` only when the chart’s syntactic dispatch fired a “lowering” grammar rule. We collapsed that into **unconditional firing** for two reasons:

1. **Cognitive parsimony.** “the running boy” (lowering, attribution) and “the boy runs” (lifting, predication) involve the **same neural composition act** — fusing a noun representation with a verb representation into a single bound state. The linguistic distinction between lift and lower is a *labelling* over a shared composed state, not a different composition primitive. Making `pi_concept` fire unconditionally puts the composition machinery at the substrate layer, where it can be invoked once and re-read by any number of downstream grammar-driven framings.
2. **Idempotence of the symbolic loop.** `cs.forward(ss.forward(c)) == c` — SS is a dimensional pass-through (no default sigma/pi at S), and the grammar’s S-tier ops are idempotent in their algebra, so routing a C-activation through SS and back through CS leaves it unchanged. That’s why CS owns only **one sigma** (for PS) — there is no fold needed on the SS side; SS already returns what C handed it. An unconditional `pi_concept` therefore can’t double-apply across the symbolic loop’s round-trip.

Where lift vs lower lives, if not at the substrate

The cognitively-real distinction between **attribution** (“the running boy”) and **predication** (“the boy runs”) is preserved at three downstream sites:

1. **Parse tree / rule_id metadata** (*primary*) — the chart records “*this composition fired under rule lift*” vs “*under rule lower*”; the truth layer and output decoder read the **rule_id** to interpret the composed state. This is the cheapest, structural distinction and matches the linguistic view that lift/lower is a **derivational labelling** over a shared operation.
2. **Per-slot cataskoti tag** on **STM._truth_tags** — secondary metadata stamp; useful when downstream readers need $O(1)$ access to role without traversing the parse tree.
3. **Category stack frames** in **WordSpace** — each composition pushes a stack frame tagged with category; NP-frames are attribution outcomes, S-frames predication.

LiftLayer and **LowerLayer** are therefore **pure rule-id annotators**. They do *not* own internal substrate sigma/pi layers (the legacy “borrowed substrate” pattern is retired); they record the rule firing and let the unconditional subsymbolic loop compute the composed state.

Grammar XML migration

The old rule names matched the **old** ownership ($P = \text{sigma}(P)$, $C = \text{pi}(C)$); under the new rebalance they’re inverted at the operator level. Rules re-label as:

Old	New	Meaning
$P = \text{sigma}(P)$	$P = \text{pi}(IS)$	pi_input always fires
(none)	$P = \text{lower}(C)$	pi_concept ; grammar can also fire lower as an S-tier rule that records the lowering
$C = \text{pi}(C)$	$C = \text{sigma}(PS)$	sigma_percept always fires
$S = \text{lift}(NP, VP)$	unchanged	now a rule-id annotator over the same loop
$S = \text{lower}(NP, VP)$	unchanged	rule-id annotator

Legacy XMLs keep working via an alias layer in the rule parser that maps old names to new layer bindings (parser-side, no runtime cost).

Normalization and Ranges

Space	Data Contract
InputSpace	Data scaled -1..1 for scalars or vector norms
PerceptualSpace	Modal/demuxed (what/where/when encoding). Signed unit-magnitude scalars or vectors. No negation
ConceptualSpace	Combined/muxed (event encoding). Signed unit-magnitude (tanh-bounded). Event norm on subspace .
SymbolicSpace	Symbols are percepts. Concept-to-symbol mapping. One symbol encoded at a time
SyntacticSpace	Words are concepts encoding grammar rules. Word tuples + production rules
OutputSpace	Rescaled from activation range to original data range

Data scaling. Data computes global **input_min/input_max** and **output_min/output_max** at load time. InputSpace uses **Data.normalize(x, "input")** to scale to $[-1, 1]$; OutputSpace uses **Data.denormalize(x, "output")** to restore the original output range.

Symbols as percepts. Symbols live in $[0, 1]$; since conceptual activations range $[-1, 1]$, the mapping is $\text{symbol} = (\text{activation} + 1) / 2$. **SubSpace.get_symbols()** / **set_symbols()** perform the conversion.

Demuxed mode. When **InputSpace.demuxed=true**, what/where/when components are stored independently in the SubSpace rather than concatenated. ModalSpace routes each component through independent PerceptualSpaces; downstream spaces see an identical muxed tensor via **materialize()**.

Codebook Similarity Metric

Codebook wraps `VectorQuantize`. Similarity metric per space:

Space	Codebook geometry	Stored
PerceptualSpace	$[-1, +1]^d$ hypercube	Feature <i>patterns</i>
SymbolicSpace	$[-1, +1]^d$ hypercube (bivector slots for tetralemma)	Symbol <i>patterns</i>
ConceptualSpace	Unit L2-norm directions (named directions)	Concept <i>directions</i> ; input magnitude in $[-1, +1]$ e

Euclidean (Perceptual / Symbolic)

These codebooks store *what something looks like* — $0.5 \cdot v$ carries half as much “of feature v ” as $1.0 \cdot v$, so the right notion is coordinate-wise distance. Retrieval expands $\|x - c_i\|^2$:

$$\|x - c_i\|^2 = \|x\|^2 + \|c_i\|^2 - 2(x \cdot c_i)$$

$\|x\|^2$ is constant across i and drops from argmin:

$$\arg \min_i \|x - c_i\|^2 = \arg \max_i (x \cdot c_i - \frac{1}{2} \|c_i\|^2)$$

`VectorQuantize` keeps $\|c_i\|^2$ in `_b_norms_sq`:

```
indices = (flat @ codebook.T - 0.5 * b_norms_sq).argmax(dim=-1)
```

One matmul + one broadcast subtract + one argmax. Skips the `sqrt`, the per-row $\|x\|^2$ add, and the cdist autograd plumbing.

Dot product (Conceptual)

`ConceptualSpace` concepts are *named directions* in belief space. $x \cdot c_i$ gives the *signed strength of belief that x affirms concept i* :

- +1 fully affirms; 0 orthogonal; -1 fully denies

Two consequences:

1. **Codebook must be unit L2-norm.** EMA renormalizes after each update.
2. **Input must NOT be normalized.** The magnitude *is* the certainty signal. Cosine similarity would divide it out.

For *ranking*, $x \cdot c_i$ and $\cos(x, c_i) = (x \cdot c_i) / \|x\|$ are monotone-equivalent (positive constant cancels). Omitting input normalization preserves certainty and costs less:

```
# codebook is unit L2-norm (maintained by EMA)
indices = (flat @ codebook.T).argmax(dim=-1)
```

Configuring the metric

`use_dot_product` is a class attribute on `Codebook` (default `False`). Set it on a `Space` subclass to opt in — `ConceptualSpace` does this. The underlying `VectorQuantize.use_cosine_sim` flag is historical; after the April 2026 perf pass, input-side normalization is gone, so the effective meaning is “codebook unit-norm; rank by dot product”.

Codebook Uniqueness Contract

Every codebook entry must be **unique under both WhereEncoding and WhatEncoding**:

- **.where** — **globally unique positional key**. Enforced structurally via the class-level **codebook offset registry** on WhereEncoding (Spaces.py:223–276). Each codebook calls `allocate_codebook_slice(n_vectors)` and gets a contiguous integer offset; all codebooks share $\text{div_term} = 2\pi/\text{total_allocated}$. Each codebook’s entries live in disjoint **.where** slices.
- **.what** — **distinct prototype content**. Identical **.what** collapses to the same parthood identity ($\text{equal}(A, A) = 1$) — a redundant pair the network can’t distinguish.

Current enforcement:

Source	Mechanism
SymbolicSpace codebook	ImpenetrableLayer overlap penalty + variance floor; five-relations classifier pushes pairs toward
ConceptualSpace codebook	ImpenetrableLayer available; not yet wired by default
PerceptualSpace Lexicon	Cosine-margin pole/antipode SBOW training
InputSpace vocabulary	Shares PerceptualSpace’s Lexicon

.where uniqueness is **structural** (enforced at construction); **.what** uniqueness is **learned** (encouraged by **ImpenetrableLayer** + antipodal quotient). Together they guarantee the parthood lattice is well-formed.

Lexicon (Projective Unit Ball)

The **Lexicon** (bin/Layers.py) backs PerceptualSpace word embeddings and SymbolicSpace symbol prototypes. Each row is a vector w_i in the **projective unit ball** — the closed ball $B^D = \{x : \|x\|_2 \leq 1\}$ with the **negation identification** $w \sim -w$ realizing real projective space $\mathbb{RP}^D$.

Terminology pin — three notions sometimes conflated:

- **Pode** of (a, b) : midpoint $(a + b)/2$; SBOW positive-pair attractor.
- **Wrapped pode**: midpoint via the $\pm$ -quotient, $(a - b)/2$; the midpoint through *negation* of b .
- **Antipode** of a single point p : furthest point. On the flat torus unique ($\text{wrap}(p + 1)$); on $\mathbb{RP}^D$ **not unique** — the maximum-distance set is the orthogonal hyperplane.

So $-w$ is the **negation** of w , *not* the antipode.

Distance and lookup

For $a, b \in B^D$ the projective squared distance is

$$d_{\mathbb{RP}}^2(a, b) = \min(\|a - b\|_2^2, \|a + b\|_2^2) = \|a\|_2^2 + \|b\|_2^2 - 2|\langle a, b \rangle|.$$

With $\text{pode}(a, b) = (a + b)/2$ and $\text{wpode}(a, b) = (a - b)/2$, $d_{\mathbb{RP}}(a, b) = 2 \cdot \min(\|a - \text{pode}\|, \|a - \text{wpode}\|)$. The lookup picks whichever rep of b (b or $-b$) is closer to a .

Sorting by smallest $d_{\mathbb{RP}}^2 =$ sorting by largest $\text{score}(x, w_i) = |\langle x, w_i \rangle| - \frac{1}{2}\|w_i\|_2^2$.

Implementation: `cache W_norm2 = W.square().sum(-1)` once per optimizer step; top-k is `(x @ W.T).abs() - 0.5 * W_norm2` followed by `torch.topk` — dense matmul + abs + broadcast subtract. No $V \cdot D$ outer-product.

The Lexicon API:

```

lexicon = Lexicon(V, D)
lexicon.project_unit_ball_()          # after optimizer.step()
W_index, W_norm2 = lexicon.lookup_index()

# Projective ( $\mathbb{RP}^D$ ) --- antipode-aware, default.
idx, dist_sq, scores = Lexicon.topk_rp(x, W_index, W_norm2, k=32)

# Plain L2 --- for sites where  $w$  and  $-w$  are distinct.
idx, dist_sq, scores = Lexicon.topk_l2(x, W_index, W_norm2, k=32)

# Pairwise primitives:
Lexicon.rp_distance_sq(a, b)
Lexicon.rp_similarity(a, b)
Lexicon.rp_pode(a, b)
Lexicon.rp_wrapped_pode(a, b)
Lexicon.rp_closer_rep(a, b)          #  $\text{sign}\langle a, b \rangle * b$ 

```

For $V \gtrsim 10^5$, use `topk_rp_chunked` to bound peak score-tensor size.

SBOW training: pode (attractor) and antipode (repulsion target)

- **Pode (attractor).** Positive-pair updates pull a and b toward $\text{pode}(a, b)$; the gradient picks the closer of b and $-b$ for shorter-arc attraction.
- **Antipode (balancing repulsion target).** Negative-pair updates push the row toward the furthest point. On $\mathbb{RP}^D$ this is a $(D-1)$ -sphere, so SBOW samples a representative orthogonal direction.

Negative-sampling gradient has two regimes by $\text{sign}\langle a, b \rangle$: positive case is standard contrastive repulsion along $(a - b)$; negative case pushes a away from $-b$ along $(a + b)$.

After every optimizer step the trainer calls `lexicon.normalize()`, clipping $\|w_i\| \leq 1$. `W_norm2` should be refreshed when weights change.

Torus primitives (`Lexicon.wrap`, `Lexicon.delta`, etc.) and the `torus=True` constructor flag remain as **legacy** static methods (the earlier Lexicon used the flat torus $T^D = [-1, 1)^D$ with wrapped MSE). New code must use the `rp_*` primitives.

InputSpace

Role. Receives the raw source buffer and lifts it into the model’s internal working dimensionality.

Text mode forward. Delegates tokenization to `Lex`, producing a span table of `(start, end, type)`. Each span $\rightarrow$ a vector with two components:

- `nWhat` dims — token content, encoded via `Basis` / `Codebook` (the word embedding lookup).
- `nWhere` dims — positional information from the character offset.

Result: `[nActive, nWhat + nWhere]` tensor.

Text mode reverse. Inverts the span encoding: each vector $\rightarrow$ nearest codebook entry, then spans $\rightarrow$ characters via the stored offset table.

Numeric mode. Tensor data passes through unchanged; `LiftingLayer` projects native input dim (e.g. 784 for MNIST) to `nDim`. Non-embedding inputs are scaled to `[-1, 1]` via the global data min/max.

Key parameters.

Parameter	Description
<code>nActive</code>	Sequence length
<code>nDim</code>	Output dim per vector
<code>nWhere</code>	Positional dims
<code>nWhen</code>	Temporal dims
<code>lexer</code>	Tokenization mode: "word" or "sentence"
<code>codebook</code>	Whether input values are discrete
<code>demuxed</code>	Store what/where/when independently

Invertibility. Always non-invertible; reverse is a separate reconstruction using the span table.

Document streaming and `valid_mask`

Documents longer than `nOutput` bytes are not truncated. `TheData` maintains a per-row cursor (`doc_idx[b]`, `offset[b]`) and `next_tick()` returns (`input`, `output`, `hard_eos`) where `input` is a `[B, nOutput]` slab containing the next $\leq nOutput$ bytes from each row's current document. `hard_eos[b]` is a host-side bool set when row `b`'s cursor exhausts the current document. A short fill at document end NULL-pads the slab tail; `valid_mask: [B, K]` bool flips False for padded positions, and state-mutation propagation skips them.

Cursor universal — trial mode for non-AR data. `next_tick()` is the single dispatch for both AR text byte (rolling cursor) and non-AR data (numeric). In trial mode (`slab_bytes` not set), each tick yields one batch of trials with `hard_eos = [True] * B`. The `runEpoch` outer loop drives `ds.next_tick()` directly for both modes; the `DataLoader` exists only so existing tests can grab `loader.dataset`.

`_end_of_stream` is a host-side `list[bool]` diagnostic only; the canonical hard-reset signal is the cursor's `hard_eos`.

AR cursor unfold retirement (2026-05-13)

The legacy AR-training path padded + unfolded the embedded sentence into `[B, K, N, D]` cursor windows so the body could see a `[B*K, N, D]` parallel view of every prefix. At `bs=128, K=128, N=1024, D=10`, the unfolded tensor alone was ~320 MB.

The unfold was retired for AR training on 2026-05-13, replaced with a serial K-cursor loop (`_forward_per_stage_no_unfold`) that walked the same prefixes with a `[B, N, D]` tensor and a per-cursor causal mask.

Within-sentence AR retirement (2026-05-14)

The serial K-cursor loop itself was retired one day later: the benchmark showed `_forward_per_stage_no_unfold` running at ~18 sent/sec (the K body+head calls dominate) vs the single-shot IR fast-path's ~61 sent/sec, and the real AR objective in this architecture is **next-sentence** prediction (the discourse layer) — not next-token within a sentence.

Within-sentence training is now IR-only. `InputSpace.forward` emits `[B, N, D]` (left-aligned, right-padded to N) and `_forward_per_stage` runs a single masked-LM pass:

1. **Stem:** `InputSpace.forward + PerceptualSpace.forward` $\rightarrow [B, N, D]$.
2. **Mask:** `create_ir_mask` replaces a `mask_rate` fraction of WHAT positions with `NULL_PERCEPT`; pre-mask event stored on `_ir_pre_mask_input` as the loss target.
3. **Body:** T stages on B rows (no per-cursor walk, no causal mask).
4. **Head:** `outputSpace` $\rightarrow [B, N, predDim]$. The head is a side channel — IR loss is computed at the P-tier, not at the head.

`runBatch` reads `_ir_mask_positions` and `_ir_pre_mask_input` and computes `MSE(perceptualSpace.subspace at masked positions, _ir_pre_mask_input at masked positions)`. The `<reconstruct>concepts|symbols|both</..`

knob adds optional C-tier / S-tier reconstruction terms (target derived by lifting `_ir_pre_mask_input` through `sigma_percept`; see Plan Section “Reconstruction-loss target shape” Option B).

`<maskedPrediction>` is retired; `<reconstruct>output</...>` is retired (it was the only path that fired the reverse pipeline); `<reverseScale>` is renamed to `<reconstructionScale>` (the legacy name remains parseable with a one-shot deprecation warning).

Sentence-level AR moves to `InterSentenceLayer` — see `doc/Architecture.md` Section “Sentence-level AR (`InterSentenceLayer`)” for the ARMA(p, q) design.

PerceptualSpace

Role. Transforms raw input vectors into perceptual representations via multiplicative interactions. Models prototype-based feature detection.

Forward operation (log-space linear).

```
s_i = log((1 + x_i) / (1 - x_i))      # atanh domain transform
z_j = W @ s + b                      # linear in log-multiplicative space
y_j = (exp(z_j) - 1) / (exp(z_j) + 1) # tanh back to [-1, 1]
```

Addition in log-space corresponds to multiplication of the original features.

Reverse (invertible=True). A single `PiLayer(invertible=True)` is shared for both directions: `_to_mult(y), log, W^{-1}(z - b), exp, _from_mult`. Matrix inverse via `InvertibleLinearLayer` (LDU).

Reverse (invertible=False, reversible=True). Two `PiLayer` instances — `pi1` for forward, `pi2` for reverse — with independent weights.

Key parameters.

Parameter	Description
<code>nActive</code>	Number of active perceptual vectors
<code>nVectors</code>	Codebook size; enables VQ when $> nActive$
<code>invertible</code>	True: shared invertible layer; False: separate <code>pi1/pi2</code>
<code>codebook</code>	False -> <code>.what</code> is a passthrough <code>Tensor</code> ; True -> <code>Codebook</code>
<code>hasAttention</code>	Enable attention reweighting
<code>bivectorOutput</code>	Applies Q2 promotion $(a_P, a_N) = (\max(0, x), \max(0, -x))$ to the per-slot percept event; writes a $[B, N, 2]$

Range. Vectors live in $[-1, 1]^d$ (tanh-bounded). No negation operator — percepts represent feature magnitudes with sign indicating direction.

Under `bivectorOutput=true`, per-slot scalar activation is replaced by a non-negative paired-index pair $[a_P, a_N] \in [0, 1]^2$. The signed-sum `x = sum_d event[... , d]` is split via `aP = max(0, x)`; `aN = max(0, -x)`. Reverse recovers `aP - aN` and broadcasts uniformly across the input dim (per-feature detail within a slot is intentionally lossy).

ConceptualSpace

Role. Transforms perceptual vectors into abstract concepts via additive linear operations. Models conceptual hyperplanes that partition perceptual space.

Geometry contrast with the Lexicon. `ConceptualSpace`’s codebook stores **named directions** (unit-norm $c_i \in S^{D-1}$); input magnitude in $[-1, +1]$ encodes belief certainty with sign. No antipodal identification — sign matters.

Owned layer (2026-05-13 rebalance).

Layer	Direction	Math
<code>self.sigma_percept</code> (<code>SigmaLayer</code>)	$P \rightarrow C$	additive linear $\tanh(W @ \operatorname{atanh}(x) + b)$, non-square in the general case

One `SigmaLayer` handles both directions via self-inverse in the square / invertible regime (the bivector configuration forces square dim — the codebook does the dim adaptation inside `project`). In the non-square / non-invertible regime, two `SigmaLayers` (`sigma_percept_1`, `sigma_percept_2`) are constructed when reversible without invertibility, with independent weights.

Binary forward (legacy `binary=True` flag). Preserved on the `SigmaLayer` surface for grammar layers that need it; defaults to off. The `SigmaLayer`’s additive fold is the OR-side of the tetralemma (disjunction of features into a concept), the inverse of `PiLayer`’s AND-side multiplicative fold (intersection of features).

Forward ($P \rightarrow C$).

```
s = atanh(x)                # entry transform; clamps |x| < 1
y = tanh(W @ s + b)         # back to (-1, 1)
```

`monotonic` constrains $W \geq 0$ in the invertible variant; `nonlinear=True` keeps output in $[-1, 1]$.

Reverse (invertible=True, square dim).

```
s = atanh(y)
x = tanh(W^{-1} @ (s - b))
```

Activation carrier. `ActiveEncoding.nDim = 2`: activation is a 2-dim bivector $[aP, aN]$ per position, encoding tetralemma (*catuskoti*) corners:

State	$[aP, aN]$
TRUE (<i>asti</i>)	$[1, 0]$
FALSE (<i>nasti</i>)	$[0, 1]$
BOTH (<i>ubhaya</i>)	$[1, 1]$
NEITHER (<i>anubhaya</i>)	$[0, 0]$

BOTH encodes first-class inconsistency; NEITHER encodes unknown. Operations obey De Morgan under pole-swap negation $\neg[aP, aN] = [aN, aP]$:

- Conjunction: $[\min(aP, bP), \max(aN, bN)]$
- Disjunction: $[\max(aP, bP), \min(aN, bN)]$

See `BuddhistParallels.md`.

MASK on SubSpace._active. Two orthogonal per-position tensors: `activation` (4-valued bivector) and `_active`: $[B, N, M]$ (modality presence flags). `_apply_mask` is shape-disambiguated:

Mask shape	Effect
Aligns with <code>out.shape[-1]</code> (feature axis)	Element-wise multiply on output
Aligns with <code>out.shape[-2]</code> (position axis)	Zero masked rows of <code>_active</code> ; <code>materialize()</code> gates downstream

ShortTermMemory

`ConceptualSpace` also owns `self.stm` — a `ShortTermMemory` instance, a per-batch stack of unquantized C-tier “ideas” (continuous compositions of concepts produced by reduce operations). This is distinct from

the sentence-scoped `_stm_fired` flag on `WordSpace` (which is a discourse- priming single-shot signal, not a working-memory buffer).

Property	Default	Configurable via
Capacity	Auto-sized to <code><WordSpace><wMax></code> (sentence length bound), fallback 8	<code><ConceptualSpace><stmCapacity>N</code>
Storage	<code>[batch, capacity, concept_dim]</code> buffer + <code>[batch]</code> depth pointers	<code>persistent=False</code> (working state, n
Cleared on	Hard Reset (sentence boundary)	Soft reset leaves it intact

API: `push(b, idea)`, `pop(b)`, `peek(b, n=0)`, `snapshot(detach=False)`, `size(b)`, `is_full(b)`, `is_empty(b)`, `clear(b=None)`, `ensure_batch(batch)`.

The per-word stem inside `BasicModel._forward_stem_per_word` pushes one post-quantized idea per word; the body's `_chart_compose_at_C` consumes the buffer via `snapshot()` at every stage. The reverse mirror, `_chart_generate_from_stm`, fires at the symmetric C-tier point inside the reverse pipeline. The 7 ± 2 cap is enforced by `wMax`-driven capacity plus per-row depth pointers.

Lift/Lower factorization

`ConceptualSpace`'s `self.pi` plays double duty: it's the bare C-forward operator (no grammar dispatch), AND the substrate that `LowerLayer` routes through. Similarly, the activated `PerceptualSpace.sigma` is the substrate that `LiftLayer` routes through (it's also nominally the sub-percept aggregator, currently a single-role surface).

Both grammar layers factor as:

```
LiftLayer.forward(VP_bivec, NP_bivec):
    VP_c, NP_c = S.codebook.reverse(VP_bivec, NP_bivec, project=True)
    gated_NP   = VP_c * NP_c                # elementwise mask at C-tier
    out_c      = P.sigma.forward(gated_NP)   # substrate sigma (lift)
    return     S.codebook.forward(out_c, project=True)
```

```
LowerLayer.forward(VP_bivec, NP_bivec):
    VP_c, NP_c = S.codebook.reverse(VP_bivec, NP_bivec, project=True)
    gated_NP   = VP_c * NP_c                # same gate
    out_c      = C.pi.forward(gated_NP)     # substrate pi (lower)
    return     S.codebook.forward(out_c, project=True)
```

The shared $L \cdot U$ per-layer LDU basis is reused across every VP. The per-call gate `VP_c * NP_c` (elementwise multiplicative on the C-tier prototype $\times$ dim grid) is what makes different VPs produce different transformations — “VP is the mask.” No `raw_gate` learnable parameter; the gating signal comes from VP's codebook content. Different adjective / verb codebook activations give different outputs from the same shared matrix.

Sigma vs Pi asymmetry maps directly to lift vs lower:

- Lift uses sigma (additive log-domain expansion) — naturally “lifting features onto concepts.”
- Lower uses pi (multiplicative log-domain contraction) — naturally “lowering concepts into specific percept-realizations.”

See `Layers.md` for the `GrammarLayer` specifics.

SymbolicSpace

Role. Converts continuous concept activations into a discrete set of active symbols. The information bottleneck.

Owned layer.

Layer	Direction	Math	Notes
<code>self.sigma</code> (<code>SigmaLayer</code>)	$C \leftrightarrow S$	atanh-domain additive, monotonic	<code>forwardSigma/reverseSigma</code> aliases

Symmetric to `ConceptualSpace.pi`.

Symbols are percepts. Each symbol represents presence (1) or absence (0) of a named entity. Mapping:
`symbol = (activation + 1) / 2`.

See `Language.md` for the language system design.

Forward (`codebook=True`).

1. Extract concept activation `[B, nConcepts]`.
2. Map through `SigmaLayer(nConcepts, nSymbols, invertible=True, monotonic=True)`.
3. Store as symbolic presence `[0, 1]` via `set_symbols()`.
4. Reshape to `[B, nSymbols, 1]`; codebook produces a one-hot activation weighted by similarity.

Reverse. Reads symbolic presence via `get_symbols()`, then the `PiLayer`'s exact inverse maps `[B, nSymbols]` back to `[B, nConcepts]`.

Key parameters.

Parameter	Description
<code>nActive</code>	Total symbols (output + reconstruction)
<code>nVectors</code>	Codebook size (= <code>nSymbols</code> when <code>codebook=true</code>)
<code>codebook</code>	Enable codebook quantization
<code>bivectorOutput</code>	Returns per-prototype catuskoti bivector $[B, V_S, 2]$ via <code>Codebook.forward(..., project=True)</code> ; reverses

Codebook shape. One row per symbol at the natural `nDim` width: `subspace.what.getW().shape == (nVectors, nDim)`. Each row is a free coefficient vector over conceptual axes. Codebook is *not* unit-norm; symbol prototypes are free patterns, retrieved via Euclidean L2.

The per-prototype catuskoti bivector `[B, V_S, 2]` lives on `subspace.activation`, NOT in the codebook. Populated by `Codebook.forward(input, project=True)` — the **intrinsic snap**:

```
pos[b, n] = sum_v relu(dot(input[b, v], W[n]))
neg[b, n] = sum_v relu(dot(-input[b, v], W[n]))
```

The matching decode `Codebook.reverse(bivec, project=True)` is the cached SVD pseudo-inverse: $C \rightarrow S \rightarrow C$ round-trip projects the input onto $\text{span}(W)$ and is a fixed point thereafter (verified by `test/test_idempotent_loop.py`).

The $C \leftrightarrow S$ boundary as **categorization**: calling `SymbolicSpace.forward` IS the act of *naming* — projecting concept activation onto the named codebook lattice. Clean prototype match $\rightarrow$ TRUE-corner activation; noisy match $\rightarrow$ degraded TRUE pole; off-lattice $\rightarrow$ NEITHER. The dialectic loop is snap (synthesis) $\rightarrow$ grammar (logic) $\rightarrow$ decode (analysis) $\rightarrow$ next pass.

See `BuddhistParallels.md`, `Logic.md`, and `Mereology.md`.

Hierarchical mode. When `<useButterflies>true</useButterflies>` or `<useGrammar>all</useGrammar>`, `BasicModel` stores per-stage `ConceptualSpace/SymbolicSpace` instances. Symbol dimension is geometrically partitioned per order. See `Reasoning.md`.

Monotonicity of the lift / lower chain

Under `<bivectorOutput>true</bivectorOutput>` on P , C , and S , the $P \rightarrow C \rightarrow S$ chain is an **order-preserving map on a positive cone**.

Three pieces:

1. **Activations live on the positive cone** $[0, 1]^{\{2K\}}$ (paired-index bivector). `PerceptualSpace`'s Q2 promotion is the bitonic-to-bivector entry point. The componentwise partial order $\leq$ is the parthood order: $s_1 \leq s_2$ componentwise $\Leftrightarrow s_1$ is part of s_2 .
2. **The Π / Σ maps are restricted to $W \geq 0$** entry-wise (`monotonic=True` selects `NonNegativeInvertibleLinearLayer` or `NonNegativeLinearLayer`):

$$a \leq b \text{ componentwise} \implies Wa \leq Wb \text{ componentwise}$$

3. **Therefore Π / Σ preserve parthood pole-by-pole.**

The bivector layout keeps the contradiction corner $[1, 1]$ distinct from the ignorance corner $[0, 0]$ under positive `matmul` — a single bitonic axis would let $aP - aN$ cancel under summation.

The `ImpenetrableLayer` regularizer maintains an antichain of same-rank prototypes, complementing the structural `.where`-uniqueness from the codebook offset registry. See `Logic.md` Section Parthood as Projection and `BuddhistParallels.md`.

SyntacticSpace — retired

The standalone `SyntacticSpace` class has been retired. Grammar / chart / derivation-tree machinery now lives on `WordSpace`, which attaches a `SyntacticLayer` to `SymbolicSpace` (the canonical grammar host). The CNF binary-derivation behavior previously documented here is preserved by `WordSpace.compose` + the chart at `S`; words are still concepts encoding grammatical rules, and the derivation is still stored as word tuples — just on `WordSpace` rather than on a separate `Space`.

See `Language.md` for the grammar and the chart's per-tier rule dispatch.

OutputSpace

Role. Maps symbolic (or syntactic) vectors to task targets via linear projection.

Forward. $y = W_{\text{out}} * x + b_{\text{out}}$. Always `reshape=True` — the $[B, n\text{Symbols}, \text{symbolDim}]$ tensor is flattened before projection.

Reverse. Pseudoinverse of W_{out} . Text mode snaps each output vector to the nearest codebook entry (nearest-neighbour lookup).

`getEmbeddedIO()` override. Returns raw target dimensions rather than encoded dimensions, so loss is computed in the output vocabulary space.

Text mode generation. Autoregressive: each step's predicted token vector is snapped to its nearest codebook entry and fed back as input until `maxResponseLength` or EOS.

Key parameters. `nActive`, `nDim`, `nVectors`.

Layer. `LinearLayer` with (`bias`, `temp`) for ergodic mode.

Range. Forward rescales $[-1, 1]$ to the original data range via `Data.denormalize()`. Reverse applies `Data.normalize(x, "output")`.

Invertibility. Pseudoinverse; not exactly invertible in general.

Language

Overview

The language system is a soft-superposition CKY chart parser that runs over symbol activations. `SymbolicSpace` plays the role of the architecture’s **calculator**: the chart at S reads operand cells, invokes the grammar layer’s compose / forward / reverse, and writes the result back. Three pieces cooperate:

- **Grammar** (in `bin/Language.py`) — singleton rule table loaded from XML or `data/grammar.cfg`.
- **Chart** (`Language.py`) — owns the chart parameters, runs the inside / outside passes, and dispatches each per-cell rule application through `wordSpace.host_layer(tier, rule_name)`.
- **GrammarLayer** subclasses (in `bin/Layers.py`) — one class per grammar operator (`not`, `non`, `intersection`, `union`, `conjunction`, `disjunction`, `lift`, `lower`, `equals`, `part`, `true`, `false`, `swap`, `query`).

InputSpace -> PerceptualSpace -> ConceptualSpace -> SymbolicSpace -> OutputSpace
chart runs here

Deferred refactor — serial / shift-reduce parser. A separate follow-up pass will replace the batched CKY with a serial / shift-reduce parser driven word-by-word, consuming `ConceptualSpace.stm` as the working-memory stack. The cadence is **per-word**: each word makes a full round trip P (BPE lex) -> C (project) -> S (codebook snap = word identity + POS) -> C (S->C reverse, the unquantized idea) -> `stm.push(idea)`, and the parser then fires reductions on the STM via the chart’s grammar rules. POS rides the codebook hit for free via the bivector codebook’s `category_ids` (hard POS tag per atom) and `category_logits` (learnable per-atom soft POS distribution); the chart’s `_apply_codebook_pos_seed` already EMA-updates these from the inside pass, so per-word snap returns (`word_id`, POS) simultaneously — no separate POS tagger needed.

The chart’s launch site is at C (over `ConceptualSpace.stm` snapshot) rather than at S (with S retained as a “calculator” reachable via a per-reduce dispatch buffer for relational ops like `query` / `equals` / `part`). The 2026-05-12 refactor retired the legacy `ChartCompose` / `ChartGenerate` stem modules; `_chart_compose_at_C` fires inside every body stage and `_chart_generate_from_stm` mirrors it on the reverse path. See `doc/plans/2026-05-12-serial-parser-handoff.md` for the original handoff spec.

Tiers (P / C / S / L)

Tier	Owner	Activation domain
P	<code>PerceptualSpace</code>	bivector [B, V, 2] per percept
C	<code>ConceptualSpace</code>	bivector [B, V, 2] pre-codebook
S	<code>SymbolicSpace</code>	scalar [B, V] post-codebook (monotonic)
L	(logical, none)	bivector [..., 2] – pure lattice primitive

S-tier ops compose **monotonic** functions over the post-codebook scalar activation (`effective_activation()`: bivector poles reduced via `max` and gated by modal presence). L-tier ops (`intersection`, `union`) are pure lattice min/max on the bivector activation; the chart binds them at whichever space’s tier the operands live in.

Per-space dispatchers are `SyntacticLayer` instances built by `build_space_syntactic_layer`; each holds a tier-specific `host_layers` dict keyed by `rule_name`. Rule-firing probability gating (`GrammarLayer.gated_run`) lives on `Chart` itself, installed as `GrammarLayer._chart_authority` at `WordSpace.__init__` time.

Grammar

TheGrammar is a singleton Grammar instance. Production configs load `data/grammar.cfg` through `<WordSpace><language><grammarCfg>...`, which is the authoritative rule table. An XML `<grammar>` block is the legacy alternative.

A `RuleDef` namedtuple carries (tier, canonical, arity, method_name, lhs, rhs_symbols). `method_name` is the op name ('lift', 'intersection', 'not', ...), or 'merge' for bare-symbol-sequence rules, or 'emit_head' for the downward `S -> C` projection.

Accessors

- `Grammar.symbolic()` — indices of all S-tier rules
- `Grammar.symbolic_transition()` — None (no cross-tier transitions)
- `Grammar.binary_rules()` — indices of all arity-2 rules
- `Grammar.rule_by_id(i)` — canonical production string
- `Grammar.method_name(i)` — dispatch method name (e.g. 'swap')
- `Grammar.arity(i)` — 1 or 2
- `Grammar.tier(i)` — 'P' / 'C' / 'S' / 'L'

Grammar configuration

`Grammar.configure()` loads an XML `<grammar>` section split into `<upward>` (parsing), `<downward>` (generation), and `<start>` (the start symbol). When the chart reduces a row's parse to a single node of the start category, the layer emits a soft-reset signal: the outer doc-streaming loop dispatches `wordSpace.soft_reset(batch=b)`, clearing per-sentence working buffers. Discourse history and codebook EMA are *not* cleared — they are document-scoped.

The loader accepts three RHS forms; the **explicit-op** form is canonical:

```
<grammar>
  <upward>
    <!-- (1) Explicit-op form. -->
    <rule>S      = lift(NP, VP)</rule>
    <rule>NP      = intersection(AP, NP)</rule>
    <rule>S      = not(S)</rule>
    <rule>S,S    = intersection_inv(VO)</rule>      <!-- multi-output reverse -->

    <!-- (2) Function-call form. -->
    <S>not(S)</S>
    <S>swap(S, S)</S>

    <!-- (3) Bare-symbol form (transitional). method_name='merge'. -->
    <S>S VP</S>
  </upward>
  <downward>
    <S>C</S>                                <!-- emit_head -->
  </downward>
</grammar>
```

Op names dispatch into `GRAMMAR_LAYER_CLASSES` (Layers.py): `lift`, `lower`, `intersection`, `union`, `not`, `non`, `conjunction`, `disjunction`, `equals`, `part`, `true`, `false`, `swap`, `query`. Reverse productions are derived mechanically at grammar-load time — for `LHS = op(arg1, arg2)` the loader synthesizes `arg1`, `arg2 = opReverse(LHS)`.

data/grammar.cfg dispatch

Line-oriented text alternative; sections bracketed ([upward] / [downward]); each LHS = body rule's body is a function call, a single category (PROJECT), or `epsilon`. Loading is gated via `<WordSpace><language><grammarCfg>` in XML.

Syntactic sugar consumption

Under byte-level lex routed through BPE chunking, whitespace and punctuation become their own word slots that the grammar reduces away via `absorb(A, B) -> A`. Initial logits should bias against `absorb` to avoid early-training collapse.

Shamatha speech mode

Shamatha speech (`useGrammar="shamathaSpeech"`) is the one-pointed speech mode. It may conjoin and disjoin over all active percepts in the current object field at once, constrained so the logical object remains single-pointed. Narrow DNF object grammar:

```
literal := polarity? concept
polarity := "" | "non-" | "not"
term := literal ("and" literal)*
object := term ("or" term)*
sentence := object
```

Every logical merge must preserve spatiotemporal contiguity: `where(A)` and `where(B)` must overlap or be connected; `when(A)` and `when(B)` must overlap or be temporally adjacent. See `plans/2026-04-28-shamatha-speech-contiguity-handoff.md`.

The polarity surface (`""`, `non-`, `not`) maps to the three channels of `NegationLayer(ternary=True)`.

Operator semantics

Each grammar op has one computation anchor: **Sym** (GrammarLayer tensor kernel), **SigmaPi** (level-crossing fold via `SS.forward(CS.forward(...))`), **Def** (definitional WordSpace update), or **Percepts** (pre-grammar perceptual pre-processing).

Notation

Let $x, \ell, r \in \mathbb{R}^{B \times N}$ or $\mathbb{R}^{B \times N \times D}$. Let $\mathcal{B}$ be the subspace basis supplying pointwise primitives; absent $\mathcal{B}$, kernels fall back to torch elementwise. An optional `mask` gates the output.

Ternary commitment (true / false / non)

For $x \in [-1, 1]$ (clamped on entry):

$$\text{true}(x) = \max(0, x), \quad \text{false}(x) = \max(0, -x), \quad \text{non}(x) = 1 - |x|$$

These partition unity: `true + false + non = 1` pointwise. All three are lossy.

Negation (not) — Sym

Propositional bivector swap on the leading two dims: $[x_0, x_1, x_{2..}] \rightarrow [x_1, x_0, x_{2..}]$. Self-inverse.

Conjunction / disjunction — S-tier (post-codebook scalar)

$S = \text{conjunction}(S, S)$ and $S = \text{disjunction}(S, S)$ operate on the post-codebook scalar activation [B, V] (non-negative strength per prototype). Strictly monotonic lattice primitives:

$$\text{conjunction}(\ell, r) = \min(\ell, r), \quad \text{disjunction}(\ell, r) = \max(\ell, r)$$

Routed through `Ops.intersection(..., monotonic=True)` / `Ops.union(..., monotonic=True)`. Both lossy with `(parent, parent)` pseudo-inverse on `reverse`.

Intersection / union — L-tier (lattice on bivectors)

$L = \text{intersection}(L, L)$ / $L = \text{union}(L, L)$ are lattice min/max on a **bivector activation** [..., 2]. The chart binds them at whichever space's activation tier the operands live in.

monotonic	kernel
False	RadMin / RadMax — same-sign min / max
True	strict lattice min / max (per channel)

L-tier ops are lossy: `decompose` returns `(parent, parent)`.

Part / equals — Def (tensor scoring)

$\text{part}(\ell, r) = \frac{\max(0, \ell \cdot r)}{|\ell| |r|} \cdot r$. The scalar score is broadcast to r 's rank. Empty-operand contract: if $|\ell|$ or $|r|$ is near zero, the score is 1.

`equals(\ell1, r) = part(\ell1, r) \cdot part(r, \ell1)`, scalar-reduced and broadcast to `$r$`. Under `mask`, equals degenerates to L1-distance agreement. Both lossy. See Mereology.md.

Absorb — base-class marker

`absorb(\ell1, r) = \ell1`. Lives on `GrammarLayer.absorb(left, right) -> left` so any binary subclass can flag its right operand as syntactic sugar.

Query — Mereological

`query(\ell1, r) = \ell1`. The chart pushes a marker word elsewhere when a norm-drop fires (see Composition below).

Swap (Sinkhorn soft permutation) — Sym

A learned 3×3 logit matrix M is normalized via $k = 5$ Sinkhorn iterations to produce a doubly-stochastic soft permutation P . Given operands ℓ, r and a learned broadcast marker m , the output is the first row of $P \cdot [\ell, r, m]^T$. Lossy.

Lift / lower — rule-id annotators over the shared subsymbolic loop

Post-2026-05-13 refactor. `LiftLayer` and `LowerLayer` are now **pure rule-id annotators** — they no longer own (or borrow) a substrate sigma/pi. All composition happens in the unconditional subsymbolic loop:

`C = sigma_percept( pi_input(IS) + pi_concept(C_prev) )`

owned by `ConceptualSpace.sigma_percept` and the `PerceptualSpace.pi_input` / `pi_concept` pair. See Spaces.md Section “Sigma / Pi ownership”.

When the chart fires $S = \text{lift}(NP, VP)$ or $S = \text{lower}(NP, VP)$, `LiftLayer` / `LowerLayer`:

1. Record the **rule_id** in the parse tree (**lift** vs **lower**).
2. Optionally stamp the per-slot **catuskoti tag** on `STM._truth_tags` so downstream readers (truth layer, output decoder) can read role in $O(1)$.

They do **not** apply a separate sigma / pi. The composed C-state is whatever the unconditional loop already produced this tick. Lift vs lower differ in what the downstream **truth and output layers** make of that state, not in the substrate operation that produced it.

Why no separate substrate per rule Linguistically, “*the running boy*” (lowering, attribution) and “*the boy runs*” (lifting, predication) share a single neural composition act — *fuse a noun representation with a verb representation into one bound state*. The lift/lower distinction is a **derivational labelling** of that shared state, not a separate operation. Re-firing the loop with a different rule annotation suffices to switch framings; the chart records which framing won.

The previous “gated-substrate” pattern (which borrowed `perceptualSpace.sigma` at `concept_dim` and `conceptualSpace.pi` at `percept_dim`) is retired. Standalone-test construction with no perceptual/conceptual refs falls back to the static lattice kernels (`Ops._lift_kernel` / `Ops._lower_kernel`) when the unconditional loop isn’t wired (legacy compatibility).

Idempotence of the symbolic loop `cs.forward(ss.forward(c)) == c`. SymbolicSpace is a dimensional pass-through (no default sigma/pi at S, only grammar ops, which are idempotent under their algebra), so routing a C-activation through SS and back through CS returns the same state. An unconditional `pi_concept` therefore can’t double-apply across the symbolic round trip — it just re-folds the same C content into the unchanged P state.

Chunking — Percepts

Not a grammar rule. Implemented by `ChunkLayer` / `PerceptualSpace.chunk_static` as a perceptual pre-pass before the grammar sees symbol slots.

Mereological suite — see Mereology.md

`part`, `whole`, `equal`, `overlap`, `underlap`, `boundary`, `copart` live in `Ops.*`; the canonical spec is Mereology.md. Each has a vector form (default) and a scalar form (`scalar=True`).

Empty-set conventions (parthood scalar form): $\text{part}(\emptyset, y) = 1$, $\text{part}(x, \emptyset) = 0$, $\text{part}(\emptyset, \emptyset) = 1$.

GrammarLayer Implementations

`GrammarLayer` (in `bin/Layers.py`) is the base class. Each subclass declares `rule_name`, `arity`, `invertible`, `lossy` as class attributes, then overrides `forward()` (and `reverse()` if `invertible`). Binary subclasses additionally override `compose(left, right)` and `generate(parent)`.

Base contract

Class attributes: `rule_name`, `arity`, `invertible`, `lossy`, `_chart_authority` = `None`.

`gated_run(x, fn, ...)` consults `_chart_authority.should_run_rule(rule_name)` to get firing probability `p`, then returns `fn(x)` if `p >= 1`, `x` if `p <= 0`, or the soft mixture `p * fn(x) + (1 - p) * x` otherwise.

`compose(*operands)` defaults to `forward(operands[0])` for arity 1; arity 2 raises `NotImplementedError` and must be overridden. `generate(parent)` defaults to `reverse(parent)` for arity 1 invertible, else `parent`.

Layer summary

Class	Rule	Tier	Arity	Reads	Description
NotLayer	not	S	1	event	Bivector pole swap on [..., :2]; self-inverse
NonLayer	non	S	1	event	Per-pole bivector complement $1 - x[\dots, :2]$; self-inverse
IntersectionLayer	intersection	L	2	activation	Lattice min on bivector [..., 2]; monotonic toggles R
UnionLayer	union	L	2	activation	Lattice max on bivector [..., 2]; monotonic toggles R
ConjunctionLayer	conjunction	S	2	activation	Ops.intersection(monotonic=True) on [B, V] scalar
DisjunctionLayer	disjunction	S	2	activation	Ops.union(monotonic=True) on [B, V] scalar
LiftLayer	lift	S	2	event	sigma.compose (OR fold) gated by per-rule raw_gate; ex
LowerLayer	lower	S	2	event	pi.compose (AND fold) gated by per-rule raw_gate; ex
EqualsLayer	equals	S	2	event	Mutual parthood score broadcast over right operand; los
PartLayer	part	S	2	event	Directional parthood score broadcast over right operand
TrueLayer	true	S	1	event	relu(clamp(x, -1, 1)); lossy
FalseLayer	false	S	1	event	relu(-clamp(x, -1, 1)); lossy
SwapLayer	swap	S	2	event	Sinkhorn soft permutation; swap_logits+swap_marker
QueryLayer	query	S	2	event	Returns left operand; chart pushes marker elsewhere

reads_activation = True feeds subspace.materialize(mode='activation'); False feeds the muxed event tensor.

NotLayer example

```
class NotLayer(GrammarLayer):
    rule_name      = "not"
    arity          = 1
    invertible     = True
    tier            = 'S'
    reads_activation = False

    def forward(self, x):
        bivector = x[..., :2].flip(dims=(-1,))
        rest     = x[..., 2:]
        return torch.cat([bivector, rest], dim=-1) if rest.shape[-1] else bivector

    def reverse(self, y):
        return self.forward(y)
```

IntersectionLayer / UnionLayer example

```
class IntersectionLayer(GrammarLayer):
    rule_name      = "intersection"
    arity          = 2
    invertible     = False
    lossy         = True
    tier            = 'L'
    reads_activation = True

    def __init__(self, monotonic=False):
        super().__init__(0, 0)
        self.monotonic = bool(monotonic)

    def forward(self, left, right):
```

```

    return Ops.intersection(left, right, monotonic=self.monotonic)

    def reverse(self, parent):
        return parent, parent

```

UnionLayer is identical with Ops.union and rule_name = "union".

PiLayer / SigmaLayer — parametrized folds

PiLayer (AND fold) and SigmaLayer (OR fold) inherit ButterflyLayer. Unary forward(x) is the log-domain multiplicative fold over the feature axis; binary compose(left, right) sums log-mult contributions across operands instead of within one.

PiLayer's compose: $(\log(\text{to_mult}(\text{left})) + \log(\text{to_mult}(\text{right}))) @ W + \text{bias}$ followed by $\tanh(\dots/2)$. Generate inverts via compute_Winverse_current(). SigmaLayer's compose is the additive (logit-domain) analog: atanh the operands, sum across operands, linear, tanh. binary=True on either pre-applies Ops.top2_select_ste to hard-select the top-2 operands.

Class registry

```

GRAMMAR_LAYER_CLASSES = {
    # S-tier (post-codebook scalar / mixed event)
    'not': NotLayer,
    'non': NonLayer,
    'conjunction': ConjunctionLayer,
    'disjunction': DisjunctionLayer,
    'lift': LiftLayer,
    'lower': LowerLayer,
    'equals': EqualsLayer,
    'part': PartLayer,
    'true': TrueLayer,
    'false': FalseLayer,
    'swap': SwapLayer,
    'query': QueryLayer,
    # L-tier (lattice on bivector activation)
    'intersection': IntersectionLayer,
    'union': UnionLayer,
}

```

Inverse contract by operator

Op	Tier	Reverse
not	S	exact (self-inverse pole flip)
non	S	exact (self-inverse)
lift	S	exact LDU inverse via gated sigma.generate
lower	S	exact LDU inverse via gated pi.generate
conjunction	S	pseudo-inverse (parent, parent)
disjunction	S	pseudo-inverse (parent, parent)
intersection	L	pseudo-inverse (parent, parent)
union	L	pseudo-inverse (parent, parent)
equals	S	no defined inverse (mereological write)
part	S	no defined inverse (mereological write)
query	S	no defined inverse (mereological read)
swap	S	pseudo-inverse (parent, parent)
true	S	identity passthrough (lossy)

Op	Tier	Reverse
false	S	identity passthrough (lossy)

Rule prediction

`SyntacticLayer.predict_rules(x)` — also on `Chart` — predicts a rule distribution at each derivation depth. The architecture is weight-tied across depths with a learned depth embedding.

Input projection:

$$h^{(0)} = \sigma(W_{\text{in}} x + b_{\text{in}}), \quad h^{(0)} \in \mathbb{R}^{B \times d_{\text{hidden}}}$$

For each depth $d = 0, 1, \dots, D_{\text{max}} - 1$ (shared weights):

$$\begin{aligned} h^{(d+1)} &= \sigma(W_{\text{deriv}} (h^{(d)} + e_d) + b_{\text{deriv}}) \\ z^{(d)} &= W_{\text{head}} h^{(d+1)} + b_{\text{head}} \in \mathbb{R}^{B \times K} \\ p^{(d)} &= \begin{cases} \text{gumbel_softmax}(z^{(d)}; \tau) & \text{training} \\ \text{softmax}(z^{(d)}) & \text{eval} \end{cases} \end{aligned}$$

Output: `{rule_logits, rule_probs, predicted_rules}`. Training uses gumbel-softmax; eval uses softmax + argmax.

Composition

Chart composition lives on `Chart` (`bin/Language.py`). `Chart.compose(data, word_space, subspace=None)` runs the inside pass and populates per-tier rule selections on `word_space.current_rules`. `Chart.generate(target, word_space, subspace=None)` runs the outside pass + Viterbi backtrace.

Router selected via `<WordSpace><routerKind>`: `'chart'` (CKY, default) or `'signal'` (tensorial alternative).

Chart inside pass

For each cell (i, j) and each grammar rule with matching arity:

$$\text{score}_{ij}^{\rho} = \text{compat}(\text{lhs}, \text{rhs}_{ij}) \cdot \text{pair_score}(i, j) \cdot \text{rule_logit}(\rho)$$

Soft mixture (training) or argmax (eval). Per-cell applications dispatch through `wordSpace.host_layer(tier, rule_name)`. The compatibility mask `compat[B, P, R]` zeroes (`pair, rule`) combinations whose typed LHS / RHS categories don't match the pair's slot categories. Category 0 (`'?'`) is wildcard.

Query / norm-drop detection Detect symbolic contradiction at the accumulation point by comparing pre- and post-composition Frobenius norms. Define $\nu_{\ell} = \|\ell\|_F$ and $\nu_c = \|c\|_F$. When $\nu_{\ell} > 10^{-6}$ and $\nu_c < 0.1 \nu_{\ell}$ (`_QUERY_NORM_DROP_RATIO = 0.1`), the accumulator preserves the prior state and a query word is pushed.

Tensor word buffer

Each SubSpace carries two registered buffers alongside the legacy `self.word: list[tuple]`:

```
self.word_records # [B*K, max_depth, ENTRY_WIDTH] long, ENTRY_WIDTH=7
self.word_count   # [B*K] long, current depth per cell
```

`add_word(...)` is overloaded. The chart materializes its tensor buffer before `_compose_chart_cky` returns via `subspace.flush_word_buffer()`.

Chart-derived SVO

`Chart._extract_svo_from_trace` walks each batch row's trace:

1. Find the last `S -> S VO` entry (outermost reduction).
2. Find the matching `VO -> V O` entry whose `merged_slot` equals the outer rule's right arg.
3. Pull subject from the outer's left slot, `verb` from the inner's left, `object` from the inner's right.

POS side-channel

A part-of-speech distribution rides alongside the signal at every chart cell. Trailing axis is `|POS| = |Grammar.categories|`. Index 0 is `'?'`, the wildcard.

Storage tiers

- **Per-atom POS tag** (durable seed): `Codebook.category_ids: Long[V]` on the SymbolicSpace codebook. Use `set_category(idx, cat_id) / get_category(idx)`.
- **Per-call chart POS**: `Chart._chart_pos: [B, N+1, N+1, |POS|]`, softmax of `_chart_score`. Cleared at start of each inside pass.
- **Durable per-merge POS**: `SubSpace.pos_records: Long[B*K, max_depth]`, parallel to `word_records / word_count`.

Lexical bootstrap

`Chart._apply_codebook_pos_seed` overrides the learned `_lex_cat_scorer(data)` log-distribution with a one-hot at the codebook-resolved POS for any input position whose nearest atom carries a non-wildcard `category_ids` entry and whose dot-product similarity exceeds 0.1.

Mechanism 1 — RHS POS compatibility mask

Before scoring each binary merge candidate, the chart gathers operand POS distributions and adds $\log(p_{\text{left}} * p_{\text{right}})$ to the rule's candidate score. Wildcard rules (`rhs_*[r] == 0`) get an unconditional 1.0 multiplier.

Mechanism 3 — Rule-prediction conditioning

After Viterbi extraction, `Chart._populate_category_stack` walks the trace and pushes each merge's LHS category embedding onto `WordSpace.category_stack`. Subsequent `WordSpace.predict_rule(b)` reads a parse history of POS embeddings via the existing `Linear(max_depth * pos_dim -> n_rules)` MLP.

Why this trains POS to every word

Two reinforcing gradient paths:

1. **Anchored** — codebook atoms with a known category override `_lex_cat_scorer` deterministically; reconstruction loss pulls downstream rules toward those categories.

2. **Emergent** — untagged atoms fall back to `softmax(_lex_cat_scorer(data))`; a wrong POS at a leaf produces a wrong rule firing, bad reconstruction, and a strong gradient back through `_lex_cat_scorer.W`.

Coverage matters — tagging closed-class words (DET, AUX, PREP, CONJ) plus frequent open-class atoms gives a working anchor set.

`<writeSyntax>true</writeSyntax>` — syntax-tree dump

When set, `BasicModel.forward()` calls `write_syntax_tree(path)` at the end of every forward pass. Default path `output/syntax.xml` (overridable via `<architecture><syntaxOutPath>`).

The dumper walks `_derivation_trace` and emits one `<forward>` fragment per call:

```
<forward tick="42">
  <batch n="0">
    <node cat="S" rule="S=conjunction(S,S)" i="0" j="2">
      <leaf token="1" pos="S" i="0"/>
      <leaf token="1" pos="S" i="1"/>
    </node>
    <rules>1</rules>
  </batch>
</forward>
```

`<node>` carries `cat`, `rule`, `i/j`; `<leaf>` carries `token`, `pos`, input position `i`. `<rules>` lists the global rule-id sequence for diff-based regression. File truncated on first call, appended on subsequent calls.

Under `<routerKind>signal</routerKind>`, no derivation trace is recorded; the dumper emits `<noTrace/>` for those rows.

Per-space dispatch (`SyntacticLayer`)

Each `Space` carries a `SyntacticLayer` instance whose `forward()` / `reverse()` fires one fold step per call, per the chart's per-tier rule selection. `build_space_syntactic_layer` constructs:

- `host_layers`: dict mapping `rule_name` to a `GrammarLayer` instance (parametrized folds like `space.pi`, `space.sigma`, plus stateless rules built lazily from `GRAMMAR_LAYER_CLASSES`).
- Each layer is registered with the `WordSpace` via `register_host_layer(tier, rule_name, layer)`.

`SyntacticLayer.forward(subspace)` reads `word_space.current_rules[tier]`, advances a per-tier cursor, and dispatches to the chosen layer's `forward()` (arity-1 rules only; binary rules fire inside the chart's compose). `reverse()` mirrors via `word_space.generate_rules`.

`WordSpace.forwardSymbols / reverseSymbols`

Thin: `forwardSymbols(data, subspace)` demuxes the muxed symbol tensor into the subspace's modality slots when shapes match. The symbolic composition runs on the chart via `ChartCompose`. `reverseSymbols` is pass-through.

Downward head emission (`S -> C`)

Selected by `<WordSpace><downwardGeneration>true</downwardGeneration>`. After the upward parse reduces `N` leaves to a single root vector, `BasicModel.forward` takes that root and calls `WordSpace.reconstruct(state, inputSpace)`, which runs the downward `S -> C` rule as a one-shot projection onto a codebook.

Projection-coefficient scoring

$$W_{\text{norm}} = W / \|W\|_{2, \text{row-wise}}, \quad \text{scores} = \text{clamp}_{\geq 0}(\text{state} \cdot W_{\text{norm}}^{\top})$$

The per-row argmax is `best_idx`. `state` is *not* L2-normalized (only the codebook rows are), so the score is “how much of atom k lives in the state”. `contained = scores[:, k] * W_{\text{norm}}[k]` is the slice of the atom actually contained in the state; `residual = state - contained` is the leftover.

`WordSpace.reconstruct(state, codebook_space)`

Any space whose `.subspace.basis` exposes a `getW()` $\rightarrow [V, D]$ works. `InputSpace` (word embedding) is the intended source. Returns `{heads, contained, residual, state}`. `BasicModel.forward` stashes heads as `self._predicted_head`.

Word encoding

Each word is a 7-tuple (`batch`, `vector`, `rule`, `leaf1`, `leaf2`, `leaf3`, `order`) stored in `subspace.word` (legacy list) and `word_records` (tensor buffer). Terminal words carry `order = -1` and a codebook index in `leaf1`. The list is a derivation tree in pre-order: root first, binary rules expand left-first.

Decomposition

`Chart.generate(target, word_space, subspace)` reconstructs the pre-compose tensor from the symbolic word record by walking the outside pass + Viterbi backtrace. Dispatches `host_layer.generate(parent)` for binary rules, `host_layer.reverse(y)` for unary invertible rules. Lossy rules emit pseudo-inverses.

Codebook fast path: if a basis is available and shapes match, the generator looks up each terminal word (those with `order = -1`) and places `cb[leaf1]` at the recorded position.

How syntax is trained

The syntax code is end-to-end differentiable: soft `rule_probs`, soft superposition over compatible rules, gradients flow through fold weights (PiLayer / SigmaLayer / SwapLayer logits).

`BasicModel.runBatch()` computes three losses: output, optional reconstruction, optional embedding. No explicit syntax loss; syntax fitness emerges indirectly through reconstruction — rules supporting successful reconstruction accumulate weight.

Two-Layer logic

Subsymbolic layer treats objects as vector sets (B, N, D) as RBF / luminosity fields: union is $\max(\ell, r)$; intersection is $\min(\ell, r)$; negation is $-x$; non is αx with $\alpha \in [0, 1]$; parthood is fuzzy max-coverage in $[-1, 1]$.

A **symbolization** map projects vectors to scalar truth strength:

$$s(X) = 2 \cdot \text{mean}(\|x_i\|) - 1 \quad \in [-1, 1]$$

The **symbolic** layer operates on scalars in $[-1, 1]$: neg is $-a$, non is αa , union is $\max(a, b)$, intersection is $\min(a, b)$, part is $\text{clamp}(b - a, -1, 1)$.

Subsymbolic = geometry; symbolic = order + polarity; symbolization = norm projection.

SymbolicSpace integration

Forward path

1. Extract concept activation $[B, n_{\text{Concepts}}]$.
2. Map through `PiLayer(monotonic=True, invertible=True)` to $[B, n_{\text{Symbols}}]$.
3. Codebook quantization produces one-hot activation plus vectors.
4. The chart runs the inside pass over symbol vectors via `wordSpace.host_layer('S', rule_name)`.

Reverse path

1. Extract symbol activation $[B, n_{\text{Symbols}}]$.
2. Exact inverse of the `PiLayer` recovers $[B, n_{\text{Concepts}}]$.
3. The chart's outside pass reconstructs the pre-compose tensor via `host_layer('S', rule_name).generate(parent)`.

Key properties

- Symbols are zero-dimensional — pure activation scalars.
 - The `PiLayer` allows $n_{\text{Concepts}} \neq n_{\text{Symbols}}$.
 - Rule firing gated by LHS / RHS POS compatibility via `chart_pos`.
-

AssociationLayer (EQUALS implementation)

Cross-symbol associative memory used by EQUALS. Two modes: `type="symmetric"` Hopfield-like (learns A , computes $A^\top A$, softmax-retrieves; symmetric associations); `type="hopfield"` modern Hopfield (separate query / key projections + softmax-gated retrieval). Input and output both $[B, N]$. Trained end-to-end via reconstruction loss.

Mereology

Single-page reference for the mereological grammar: parthood as the fundamental operation, the five mereological relations, and the `ImpenetrableLayer` regularizer.

Codebook IS the meronymic structure. The standalone `MereologicalTree` sidecar that formerly stored explicit parent / equality links was retired in favour of pure-geometric parthood on the `SymbolicSpace` bivector codebook. The grammar layers `PartLayer`, `EqualsLayer`, and `QueryLayer` operate directly on codebook bivector activations via clipped cosine projection — no separate adjacency table, no `<architecture><mereologicalTreeSize>` XML knob (silently ignored if present). Asserted meronymic relations are learned by training pulling the codebook geometry into the right configuration.

Parthood as Clipped Cosine Projection

Parthood is the single fundamental operation. Every other mereological relation is defined in terms of it.

For two concepts $A, B \in \mathbb{R}^D$:

$$\text{part}(A, B) = \frac{\max(0, A \cdot B)}{|A| |B|}$$

The clipped cosine projection lies in $[0, 1]$ and satisfies Boole's contrapositive:

$$\text{part}(A, B) = \text{part}(-B, -A)$$

because $(-B) \cdot (-A) = A \cdot B$ and norms are sign-invariant.

Empty-operand contract. When $|A|$ or $|B|$ is near zero, **part** returns 1.0 — the empty set is part of everything.

The Full Suite

Every member of the suite is expressible through **part**:

Method	Formula	Interpretation
part (A, B)	$\max(0, A \cdot B)$	A
whole (A, B)	$\text{part}(B, A)$	A contains B.
equal (A, B)	$\text{part}(A, B) \cdot \text{part}(B, A)$	Mutual parthood, $[0, 1]$.
overlap (A, B)	$0 < \text{equal}(A, B) < 1$	Strictly-partial mutual parthood (indicator).
underlap (A, B)	$\text{equal}(A, B) = 0$	No mutual parthood (indicator).
boundary (A, B)	$\text{part}(A, B) - \text{part}(B, A)$	

Region Partition

equal(A, B) partitions into three disjoint regions:

Region	Condition
Underlap	$\text{equal} = 0$
Overlap	$0 < \text{equal} < 1$
Identity	$\text{equal} = 1$

Under clipped cosine, **part** is symmetric, so the three regions collapse to a single scalar classifier. For a future **Basis** with asymmetric **part**, the general five-relation table applies:

Five-relation table (general case, $\tau \approx 1$, $\epsilon \approx 0$):

Relation	Condition
disjoint (a,b)	$P[a, b] < \epsilon$ and $P[b, a] < \epsilon$
part (a,b)	$P[a, b] > \tau$ and $P[b, a] < \epsilon$
part (b,a)	$P[a, b] < \epsilon$ and $P[b, a] > \tau$
equal (a,b)	$P[a, b] > \tau$ and $P[b, a] > \tau$
overlap (a,b)	Both partial

Asymmetric subsumption. Since **part** is symmetric under clipped cosine, classical asymmetric subsumption is not encoded in raw magnitude. It is recovered **relationally** via figure / ground: compare $\text{part}(A, B)$ against $\text{part}(A, \neg B)$. This is what makes Boole’s contrapositive hold exactly.

Mereological Fusion

Fusion of a truth set is the **least upper bound** (LUB / join) over stored truth vectors:

$$\text{fusion} = \max_i \text{truths}[i]$$

(elementwise max). In bivector space, the fusion vector names the top-right corner of the axis-aligned bounding hyperrectangle dominating every stored truth. Fusion is the geometric dual of luminosity: LUB (join) vs GLB (meet).

DoT is already baked into each stored truth (`stored[i] = activation_i * degree_i` in `TruthLayer.record`), so fusion is trust-weighted automatically.

See Logic.md section **Fusion** for full discussion and the leading-bivector / paired-index layout caveat.

ImpenetrableLayer: Five-Relations Regularizer

`ImpenetrableLayer` regularizes the `SymbolicSpace` symbol codebook. It classifies each ordered pair of codebook rows (i, j) into one of the five mereological relations (using `Basis.part`), and penalizes partial overlap when paired with a trust mismatch. The learned half of the Codebook Uniqueness Contract.

Penalty

\$\$ \mathcal{L} = \text{overlap_weight} \cdot \frac{\sum_{i \neq j} \text{overlap}(i, j) \cdot |\text{trust}(i) - \text{trust}(j)|}{K(K-1)}

- `\text{variance\_floor}` term

Overlap strength is damped to zero as the pair approaches `equal`:

$$\text{overlap_strength}(i, j) = \min(P[i, j], P[j, i]) \cdot (1 - \max(P[i, j], P[j, i])^k)$$

with $k = \text{equal_suppression}$ (default 4.0). Penalty is zero for `disjoint`, `equal`, and `strict part`, active only for `overlap`.

A separate `variance_floor` term guards against row collapse.

Trust

Per-row usage frequency, derived from VQ EMA counts:

$$\text{trust}(i) = \frac{\text{cluster_size}[i]}{\sum_j \text{cluster_size}[j]}$$

When VQ is absent, trust falls back to $\|cb[i]\| / \max_j \|cb[j]\|$.

Diagnostics

After `forward()`: `last_overlap_loss`, `last_variance`, `last_relation_counts` (dict summing to $K(K-1)$).

Configuration

XML knobs (under `SymbolicSpace`):

Knob	Default	Meaning
overlapWeight	0.1	Weight of overlap $\times$ trust-diff penalty
varianceFloor	0.01	Minimum per-dim std
fullPartThreshold	0.9	τ : part score above this is “full part”
disjointThreshold	0.1	ϵ : part score below this is “disjoint”
equalSuppression	4.0	k : damping exponent near the equal corner

Why This Design

- **Parthood as projection.** One formula, Boole-contrapositive exact, continuous in $[0,1]$. The old composite formula $\text{conjunction}(1 - \text{dist}(x, x \cap y), 1 - \text{dist}(y, x \cup y))$ mixed set-valued operands with a distance.
- **Overlap penalty (not antisymmetry).** Legacy `ImpenetrableLayer` penalized mutual parthood $P[i,j] \cdot P[j,i]$ directly, pushing *every* overlapping pair apart including synonyms. The five-relations design leaves `equal` pairs alone and only penalizes the ambiguous middle region, gated by trust mismatch.
- **Trust via VQ EMA.** Already tracked for VQ commit loss.
- **Parthood is preserved by Pi / Sigma.** Under `bivectorOutput=true` on P/C/S, every activation lives on the non-negative paired-index cone, Pi/Sigma are restricted to $W \geq 0$, and componentwise $\leq$ on the cone *is* the parthood partial order. Each lift / lower preserves parthood pole-by-pole. The bivector layout keeps contradiction $[1, 1]$ distinct from ignorance $[0, 0]$ under positive matmul. See `Spaces.md` “Monotonicity of the lift / lower chain”.

Logic

Overview

This document defines the logic system at three levels:

1. **Subsymbolic (vector / field level)** — geometry in `ConceptualSpace`
2. **Symbolic (scalar level in [-1,1])** — order + polarity in `SymbolicSpace`
3. **Rationality** — propositional truth store built on both

Executable implementations of the subsymbolic and symbolic operators are the `Method` subclasses in `Language.md`.

1. Subsymbolic Layer

Objects: vector sets (B, N, D) interpreted as RBF / luminosity fields.

Operators

- **Union:** `union(A, B) = concat(A, B)`. Combine sets.
- **Intersection:** Co-supported regions (RBF product / merge).
- **Negation (affirming):** `neg(x) = -x`. Antipodal opposition on hypersphere.
- **Non (non-affirming):** `Basis.non()` — bitonic returns zero (complete withdrawal); monotonic is `relu(x - threshold)`.
- **Parthood** (fundamental): `Basis.part()` — clipped cosine projection:

$$\text{part}(x, y) = \frac{\max(0, x \cdot y)}{\|x\| \cdot \|y\|}$$

Satisfies Boole’s contrapositive trivially (dot product and norms sign-invariant under joint negation). The full suite (**whole**, **equal**, **overlap**, **underlap**, **boundary**) composes through **part**.

2. Symbolization

For $X \in (B, N, D)$:

$$s(X) = 2 \cdot \text{mean}(\|x_i\|) - 1 \in [-1, 1]$$

Interpretation: +1 = strong presence, 0 = neutral, −1 = absence.

3. Symbolic Layer (Scalars in [-1,1])

Basis supports **monotonic** (plain min/max, used by P/C/S whenever `<bivectorOutput>true</bivectorOutput>`, so the entire $P \rightarrow C \rightarrow S$ chain operates on the non-negative paired-index cone) and **bitonic** (sign-aware, legacy default). Monotonic forms here; bitonic forms (RadMin, RadMax) in Section 7. See Spaces.md “Monotonicity of the lift / lower chain”.

Let $a, b \in [-1, 1]$.

Operator	Bitonic	Monotonic
Negation (affirming)	$\text{neg}(a) = -a$	—
Non (non-affirming)	$\text{non}(a) = 0$	$\text{non}(a) = \text{relu}(a - \tau)$ (learnable τ)
Union	RadMax (Section 7)	$a \cup b = \max(a, b)$
Intersection	RadMin (Section 7)	$a \cap b = \min(a, b)$

Parthood as Projection

Parthood is the **fundamental mereological operation**. For $A, B \in \mathbb{R}^D$:

$$\text{part}(A, B) = \frac{\max(0, A \cdot B)}{\|A\| \cdot \|B\|} \in [0, 1]$$

Boole’s contrapositive $\text{part}(A, B) = \text{part}(-B, -A)$ holds trivially.

The full suite composes through **part**:

Method	Formula
whole (A, B)	part (B, A)
equal (A, B)	$\text{part}(A, B) \cdot \text{part}(B, A)$
overlap (A, B)	$0 < \text{equal}(A, B) < 1$
underlap (A, B)	$\text{equal}(A, B) = 0$
boundary (A, B)	\$

Under clipped cosine, **part** is symmetric, so **equal** reduces to part^2 . Asymmetric classical subsumption is recovered **relationally** via figure/ground: compare $\text{part}(A, B)$ against $\text{part}(A, \neg B)$. See Mereology.md.

For ternary scalar values (monotonic projection):

part(a,b)	+1	0	-1
+1	1	1	0
0	1	1	1
-1	0	1	1

Zero is vacuously part of everything (empty-set convention).

4. Key Insight

- Subsymbolic = geometry
- Symbolic = order + polarity
- Symbolization = norm projection

Cleanly separates representation (vectors) from logic (scalars).

5. Rationality

Rationality is the propositional logic layer, built on S-tier grammar rules `part(S, S)` and `equals(S, S)`.

Truth Statements

A **truth statement** is any assertion processed through the full pipeline. The **TruthLayer** — owned by **WordSpace**, reached as `self.wordSpace.truth_layer` — stores activations **scaled by** `DegreeOfTruth`:

$$\text{stored} = \text{activation} \times \text{degree}$$

Degree	Stored Vector	Effect
+1	full activation	attractor
$0 < d < +1$	scaled activation	weak attractor
0	zero vector (inert)	prunable
$-1 < d < 0$	scaled, negated	weak disperser
-1	negated activation	disperser

TruthLayer (WordSpace.truth_layer)

TruthLayer is instantiated by `WordSpace.__init__`. `SymbolicSpace.forward` records activations when `<accumulateTruth> > 0`:

- `record(activation, degree)` — store `activation * degree`.
- `query(activation)` — find the closest stored truth by cosine similarity.
- `field(concepts)` — project all stored truths into `ConceptualSpace` as a scalar field over concept vectors.

Truth Field

$$f(c) = \frac{1}{n} \sum_{i=1}^n c \cdot t_i$$

where $t_i = \text{activation}_i \times \text{degree}_i$. Two kinds of regions:

- **Attractors** ($f \rightarrow +1$): concept vectors near +DoT truths.
- **Dispersers** ($f \rightarrow -1$): concept vectors near -DoT truths.

Consonance and Dissonance

1. **Internal consonance** — stored truths should be mutually consistent.
2. **External consonance** — incoming statements are evaluated against the truth field.

Propositional Structure

Both parthood and equality are S-tier operations on the bivector `SymbolicSubSpace`:

- **part(S, S)** — containment on the bivector symbol subspace. Grammar applies it as `score * B`, scaling the whole by parthood degree.
- **equals(S, S)** — identity as mutual parthood; delegates to `Basis.equal`.

Together they define a partial order over symbolic activations.

Truth Accumulation Pipeline

Truth entries arrive as `(text, DoT)` pairs. `store_truths()` stages all texts on `TheData` and runs a standard inference epoch via `runEpoch(split="runtime")`. During forward, `SymbolicSpace.forward()` records each raw symbolic activation (degree 1.0). After the epoch, each activation is scaled by its DoT:

$$t_i = \text{activation}_i \times \text{DoT}_i$$

Two-phase design (encode all, then scale) ensures all truths are encoded in the same pipeline context before DoT modulation.

6. Consistency and Verification

Internal Consistency (within a TruthSet)

Leave-one-out check: for each t_j , evaluate the field produced by the rest of the set at c_j :

$$d_j = f_{\setminus j}(c_j) = \frac{1}{n-1} \sum_{i \neq j} c_j \cdot t_i$$

- $d_j > 0$: consonant; $d_j \approx 0$: independent; $d_j < 0$: dissonant.

A disperser ($\text{DoT} < 0$) near an attractor ($\text{DoT} > 0$) naturally produces negative field values, flagging the contradiction.

Verification (incoming statement against stored truth)

`verify(statement)` processes the statement, then queries the field:

$$v = f(c_a) = \frac{1}{n} \sum_{i=1}^n c_a \cdot t_i$$

v	Interpretation
$v \rightarrow +1$	strong support
$v \approx 0$	no opinion
$v \rightarrow -1$	strong contradiction

`wordSpace.truth_layer.query()` returns the single closest truth and its degree for point lookups.

Logical Entailment (symbolic closure)

Subsymbolic field checks detect *geometric* consistency but not *logical* entailments. Example: `part(A, B) = +1` and `part(B, C) = +1` entail `part(A, C)`, but A and C could be geometrically distant. A symbolic closure layer would:

1. **Extract propositions** by decomposing stored S-tier activations into relation + two operands.
2. **Compute closure** via transitivity:
 - $\text{part}(A, B) \wedge \text{part}(B, C) \Rightarrow \text{part}(A, C)$
 - $\text{equals}(A, B) \wedge \text{equals}(B, C) \Rightarrow \text{equals}(A, C)$
 - $\text{equals}(A, B) \wedge \text{part}(A, C) \Rightarrow \text{part}(B, C)$

Each entailed proposition inherits $\text{DoT} = \min$ of the premise DoTs.

3. **Inject entailments** as new symbolic activations recorded in the TruthLayer.

Worth implementing for reasoning rather than retrieval. The geometric field alone is sufficient for soft consistency checks and nearest-truth lookups.

7. Radial Operators for Hypersphere Ternary Logic

Ternary Logic Operators

+1 = true 0 = unknown -1 = false

NOT (neg(a) = -a)

a	¬a
+1	-1
0	0
-1	+1

NON (non(a) = 0)

a	non(a)
+1	0
0	0
-1	0

Intersection (agree → min[a,b])

a \ b	+1	0	-1
+1	+1	0	0
0	0	0	0
-1	0	0	-1

Union (agree → max[a,b]; 0 absorbs)

a \ b	+1	0	-1
+1	+1	+1	0
0	+1	0	-1
-1	0	-1	-1

PART (mereological, range [0, 1])

a \ b	+1	0	-1
+1	+1	0	0
0	+1	+1	+1
-1	0	0	+1

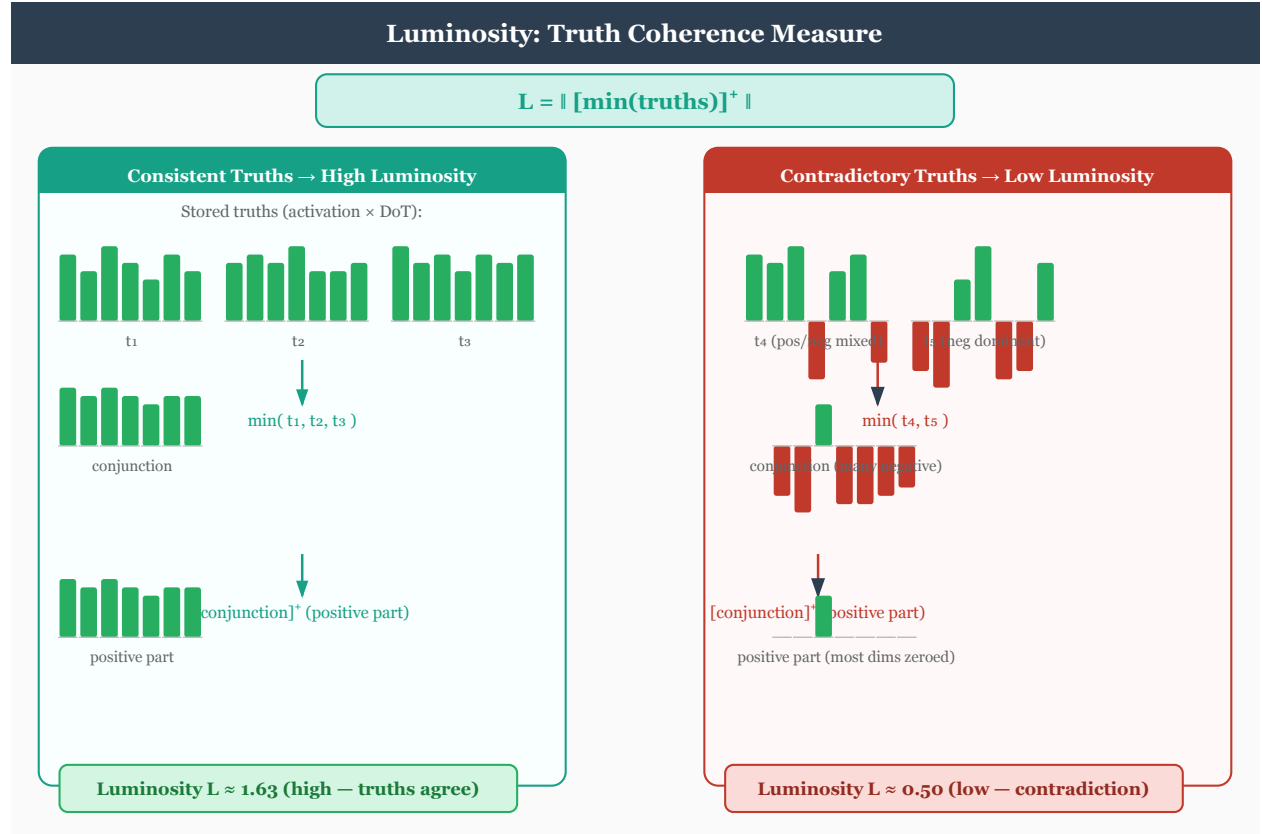
PART(a,b): 1 when $a \subseteq b$; 0 is part of everything; opposite signs → 0

RadMin, RadMax, NOT, and NON operate over signed magnitude space $[-1, +1]$. Sign = direction of assertion; magnitude = strength; zero = **unknown**. $+1 = \text{True}$; $0 = \text{Unknown}$ (no assertion); $-1 = \text{False}$. Magnitude ordering: $x \subset y$ iff $|x| < |y|$.

Op	Behavior	Ternary truth table
RadMin (AND)	Sign disagree $\rightarrow 0$; else $\mathrm{sign}(x) \cdot \min(x)$	x
RadMax (OR)	Sign disagree $\rightarrow 0$; else $\mathrm{sign}(x) \cdot \max(x)$	x
NOT	$-x$. Involutive; preserves magnitude; NOT(0) = 0.	input $\rightarrow$ -input
NON	0 for all x. Absorbing; idempotent; withdraws assertion (vs. NOT which inverts).	input $\rightarrow$ 0

Both RadMin and RadMax are commutative and collapse to zero on sign disagreement.

Luminosity



Luminosity measures truth-set coherence as a scalar:

$$\text{luminosity} = ||\text{relu}(\min(\text{truths}))||$$

Element-wise min computes the conjunction (point where all truths agree); relu removes negative dimensions; L2 norm gives brightness.

- **High luminosity:** truths are coherent and mutually reinforcing.
- **Low luminosity:** truths are sparse, contradictory, or incoherent.

Two roles in the model:

1. **Top-down bias:** concept input scaled by luminosity each conceptual iteration: $\text{concept_input} * (1 + \text{truthBiasScale} * \text{luminosity})$.
2. **Loss modification:** low luminosity increases training loss.

Fusion

Fusion is the **mereological least upper bound** of the stored truth set:

```
fusion = max_i truths[i]
```

In bivector space (paired-index [p0, n0, p1, n1, ...]), fusion names the top-right corner of the smallest hyperrectangle dominating every stored truth componentwise. Geometric dual of luminosity:

- **Luminosity** = `||relu(min(truths))||` — GLB / meet, scalar coherence.
- **Fusion** = `max(truths)` — LUB / join, vector coverage.

Trust (DoT) is already baked into stored truths, so fusion is trust-weighted automatically.

Layout caveat. TruthLayer’s paired-index slicing ([..., 0::2] for positive poles, [..., 1::2] for negative) assumes a *repeated* bivector layout. The SymbolicSpace codebook uses a *leading* bivector plus positional trailers: [pos, neg, where..., when...]. Callers feeding SymbolicSpace activations into `luminosity()` must slice the leading 2 dims first (`acts[..., :2]`); see `truth_loss` call-site in `bin/Spaces.py`.

Supporting measures

- **Consistency** (`isConsistent()`): folds truths into a union via successive `Basis.disjunction()`. In bitonic mode, conflicting +/- assertions cancel.
- **Grounding** (`ground()`): finds the minimal subset entailing a query.
- **TruthLoss**: additive penalty for contradictions, measured by union norm reduction. See Reasoning.md.
- **Derive**: pairwise mereological inference via `part()`; generalized by `extrapolate()` to all two-argument methods.

Semantic Summary

Operator	Type	Role	Behavior on Contradiction
RadMin	Binary	Conjunction (AND)	Collapses to zero
RadMax	Binary	Disjunction (OR)	Collapses to zero
NOT	Unary	Sign-flipping negation	N/A
NON	Unary	Non-affirming negation	N/A

8. Cross-references

The level-crossing operations (`lift`, `lower`, `intersection`, `union`), Pi/Sigma layer ownership, witness-set / slab-lattice geometry, and `Ops.*` namespace inventory have moved out of this document.

- **Ops.* reference**: Language.md.
- **PiLayer / SigmaLayer ownership** (`forwardPi/reversePi/forwardSigma/reverseSigma` aliases): Spaces.md.
- **Mereological suite** (`part`, `whole`, `equal`, `overlap`, `underlap`, `boundary`, `copart`): Mereology.md.
- **Tetralemma bivector layout** (TRUE/FALSE/BOTH/NEITHER corners): Spaces.md and BuddhistParallels.md.

Reasoning System

Four methods on `BaseModel` for truth-aware inference, a bidirectional reasoning loop, and a grammar learning mode. Builds on the `TruthLayer` infrastructure (`Logic.md`) and grammar composition (`Language.md`).

Partitioned Symbolic Space

The symbolic dimension is statically partitioned across conceptual orders using geometric decay. Each order writes only to its slice of `symbolSum`, while reading the full vector as feedback.

```
order 0:  [0,      D//2)      <- 1/2 of symbol_dim
order 1:  [D//2,   3D//4)      <- 1/4
order 2:  [3D//4,  7D//8)      <- 1/8
...
last order: remainder of D
```

Makes the symbolic space **self-describing**: position reveals conceptual order. Truth methods use `_activation_order()` to determine a query's order by finding the partition with the highest energy. Partition boundaries are precomputed once at model creation via `BasicModel._order_partitions`.

Reasoning Methods

`isConsistent() -> dict`

Analyzes the `TruthSet` for internal consistency by folding all stored truths into a single summary via successive `Basis.disjunction()`. In bitonic mode, conflicting +/- assertions on the same dimension cancel to zero. Returns `{'consistent': bool, 'score': float, 'sites': tensor, 'union_vector': tensor}`.

`ground(activation, threshold=0.6) -> dict`

Finds the minimal subset of the `TruthSet` entailing a query activation. Uses `_activation_order()` to filter truths by partition. Falls back to `TruthLayer.derive()` for indirect derivation. Returns `{'grounded': bool, 'basis': [indices], 'trace': [...], 'confidence': float}`.

`isTrue(activation) -> float`

Grounds a proposition and returns a scalar Degree of Truth in $[-1, 1]$. Positive = true, negative = false, zero = unknown. Delegates to `ground()`.

`extrapolate(grammar, seed_indices, max_new, attenuation) -> dict`

Generalizes `TruthLayer.derive()` to all two-argument grammar methods (union, intersection, equals, part). For each pair of stored truths, applies every eligible method and accepts results that preserve or increase luminosity. Accepted truths recorded at `attenuation * min(DoT_i, DoT_j)`. Returns `{'added': [indices], 'rejected': [(i, j, rule, delta_lum), ...]}`.

TruthLoss

Additive loss penalty for false propositions, via `<TruthLoss>` in `model.xml` (default 0.0 = disabled).

Measures the **union norm reduction** when a proposition is included in the `TruthSet` union via `Basis.disjunction()`:

```
truth_union = disjunction(all stored truths)
extended    = disjunction(truth_union, new_proposition)
penalty     = max(0, ||truth_union|| - ||extended||)
```

Case	Effect
Agreeing proposition	Preserves/extends union dims $\rightarrow$ no penalty
Unknown proposition (zero dims)	Passes through $\rightarrow$ no penalty
Contradicting proposition	Cancels conflicting dims $\rightarrow$ positive penalty

DoT weighting is implicit: stored vectors carry DoT in magnitude, so contradicting a high-DoT truth causes a larger norm drop.

TruthLoss is **additive** and coexists with **multiplicative** luminosity modulation ($\text{totalLoss} * (1 + \text{lum_weight} * (1 - \text{luminosity}))$).

Bidirectional Reasoning Loop

`BasicModel.reason(givens, target, direction, max_steps):`

- **Forward** (givens $\rightarrow$ conclusion): Encode givens into TruthSet, extrapolate new truths each step, check `isTrue(target)` until DoT exceeds threshold or `max_steps` is reached.
- **Reverse** (target $\rightarrow$ grounding): Encode target, call `ground()` to find minimal basis, extrapolate if insufficient.

Luminosity non-decrease is the validity certificate.

Grammar Learning

`BasicModel.grammar_learning_step()` learns grammar weights from a symbolic reconstruction objective:

1. Forward pass produces `symbolSum`.
2. Reverse pass reconstructs input.
3. Re-encode reconstruction to `symbolSum_hat`.
4. Loss = $\| \text{symbolSum_hat} - \text{symbolSum} \|^2$ (symbolic, not conceptual).
5. Optional luminosity validity penalty for rules decreasing luminosity.

Paraphrase-invariance holds because semantically similar sentences snap to nearby codebook entries.

Architecture Modes: useGrammar

One knob selects among Sigma-Pi architectures. Shamatha Speech is a target narrow-grammar mode wired as a DNF-object policy with contiguity checks.

	<code>useGrammar="none"</code>	<code>useGrammar="shamathaSpeech" target</code>	<code>useGrammar="all"</code>
Behavior	Flat shared sigma (default)	DNF object grammar + contiguity	Grammar-directed composition

Flat (`useGrammar="none"`)

A single shared PiLayer ($P \leftrightarrow C$) and SigmaLayer ($C \leftrightarrow S$) on a concatenated `[percepts, symbols]` tensor at every conceptual order. All orders share one undifferentiated symbolic space.

Per iteration `t`:

1. **Input.** `concept_input = cat([percepts, sym_feedback], dim=1)`.
2. **Pi** (`ConceptualSpace.pi`): `percepts` $\rightarrow$ `concepts` via `forwardPi`.
3. **Sigma** (`SymbolicSpace.sigma`): `concepts` $\rightarrow$ `symbols` via `forwardSigma`.
4. **Feedback:** Symbol activation norms broadcast back to the symbol portion.
5. **Reverse:** `reverseSigma` $\rightarrow$ peel symbol portion $\rightarrow$ `reversePi`, per order.

Key: all conceptual orders share one undifferentiated symbolic space; no way to tell from a symbol vector alone which order produced it.

Grammar-directed (useGrammar="all")

Progressive-bottleneck path with external pair-average merge (`_pair_merge` caching `left - right` diffs in `_merge_diffs`), per-level indexed Sigma/Pi (`conceptualSpace[t] / symbolicSpace[t]`), and cached symbol feedback. Symbol dimension geometrically partitioned per order — gives **partition-aware reasoning**: truth grounding, consistency, and extrapolation respect conceptual order.

Forward:

```
for t in range(conceptualOrder):
    x = pair_merge(x)                # halve vector count (external)
    x = x + sym_feedback            # additive feedback
    concepts = conceptualSpace[t](x) # per-level sigma
    symbols = symbolicSpace[t](concepts) # per-level pi
    sym_feedback = symbols.norm(...)
```

Reverse:

```
x = symbolicSpace[last].reverse(sym_vec)
for t in reversed(range(conceptualOrder)):
    x = conceptualSpace[t].reverse(x)
    x = x - cached_feedback[t]      # undo additive feedback
    x = pair_unmerge(x)            # restore vector count
```

Pair-unmerge uses cached `_merge_diffs` to recover both originals from each averaged pair — inverse is exact.

Configuration

Parameter	Location	Default	Description
<TruthLoss>	<training>	0.0	Additive truth-loss weight
<conceptualOrder>	<architecture>	1	Percept→Concept→Symbol iterations
<useGrammar>	<WordSpace>	false	Grammar-directed composition
truthMinMagnitude	<SymbolicSpace>	0.3	Activation-norm cap driving per-cell trust score in <code>TruthLayer.recon</code>

Contemplative Awareness Methods

Four stubs on `BaseModel` characterizing stages of contemplative awareness as spatial/computational properties. Each raises `NotImplementedError` — they define the target characterization, not implementation.

Method	Stage	Characterization
<code>Contiguous()</code>	One-Pointedness (Shamatha / FA)	Single connected, convex region in <code>PerceptualSpace</code> ; contiguous span
<code>Continuous()</code>	Simplicity (Continuity / OA)	Concept states flow continuously; Jacobian of forward map is bounded
<code>Peaceful()</code>	One Taste (Emotional Symmetry)	<code>TruthLayer</code> luminosity uniformly high across stored propositions
<code>Done()</code>	Buddhahood (Non-Meditation)	Model is a fixed point of forward-reverse; reconstruction loss zero

Shamatha Speech is the target grammar mode for `Contiguous()`: complete DNF object grammar plus spatiotemporal contiguity. Every **conjunction** / **disjunction** over object parts must keep `where()` support connected and `when()` support continuous. Differs from serial mode — may reduce over all active percepts at once; rejects scattered object fields, not multi-percept fields. See `Language.md` Section Shamatha Speech Mode and `plans/2026-04-28-shamatha-speech-contiguity-handoff.md`.

Testing

Unit tests in `basicmodel/test/test_reasoning.py` cover all methods without requiring a trained model. English-level tests (syllogisms, contrapositives, semantic equivalence) are `@pytest.mark.xfail` until word identity is learned through training.

Training

Overview

Two phases: **embedding pretraining** and **network training**. Embedding pretraining builds word vectors from a large corpus. Network training uses those vectors as input representation and learns to predict and reconstruct sentences.

The two phases can overlap: `<trainEmbedding>` controls whether and how embeddings continue to evolve during network training.

Phase 1: Embedding Pretraining (make `train / embed.py train`)

Produces a static embedding artifact (e.g. `BasicModel.kv`). Output path from `<embeddingPath>`.

Pipeline

FineWeb-EDU parquet shards

- > Pass 1: stream documents, lex + parse, count words, build vocabulary
- > Pass 2: stream documents again, train SBOW per sentence, discard examples
- > Save WordVectors to `sentence.pt`

SBOW (Sentence Bag of Words)

Sentence-level variant of CBOW. Given a sentence of N words:

Leave-one-out centroid for word i :

$$c_i = \frac{1}{N-1} \sum_{j \neq i} v_j$$

Logits (unnormalised, for every word in vocabulary V):

$$z_i = W_o c_i + b_o$$

Softmax probability:

$$P(w_i | c_i) = \frac{\exp(z_{i,w_i})}{\sum_{k=1}^V \exp(z_{i,k})}$$

Loss (cross-entropy, averaged over all N positions):

$$\mathcal{L} = -\frac{1}{N} \sum_{i=1}^N \log P(w_i | c_i)$$

Gradient of the loss has two terms:

- **Attractive** (from position i where w_i is the target): pulls v_i toward directions that increase its probability.
- **Repulsive** (from positions $j \neq i$ where v_i is part of the centroid): pushes v_i away from centroids whose targets are other words.

The softmax partition function ensures all V words compete: any word too close to a centroid it doesn't belong to increases the denominator and is pushed away with force proportional to its softmax probability. This adaptive repulsion distributes vectors across the embedding space.

```
vecs = embedding(idx)                # [N, dim]
total = vecs.sum(dim=0)              # [dim]
centroids = (total - vecs) / (N - 1) # [N, dim] leave-one-out
logits = linear(centroids)           # [N, vocab] full softmax
loss = cross_entropy(logits, idx)
```

CBOW (Continuous Bag of Words)

CBOW predicts a single target word from its context. Context mean (with padding for variable-length contexts):

$$\bar{c}_i = \frac{1}{|C_i|} \sum_{j \in C_i} v_j$$

Loss applies the same negative-sampling objective per-word to the padded context mean rather than the leave-one-out centroid:

$$\mathcal{L}_{\text{CBOW}} = -\log \sigma(\bar{c}_i^\top v_{w_i}) - \sum_{k=1}^K \log \sigma(-\bar{c}_i^\top v_{w_k^-})$$

where K is the number of negative samples and w_k^- are sampled words.

Two-Pass Architecture

- **Pass 1 (vocabulary):** Stream all documents, count every word. Words meeting `min_count` are promoted; model (embedding + linear head) allocated once at final vocab size.
- **Pass 2 (training):** Stream documents again. Per sentence, run one SBOW step and discard examples. Multiple epochs re-stream the data.

Configuration (Makefile variables)

Variable	Default	Description
BASIC_SHARDS	1	FineWeb-EDU parquet shards
BASIC_MAX_DOCS	10000	Max documents to process
BASIC_VEC_SIZE	100	Embedding dimensionality
BASIC_EPOCHS	10	Training epochs
BASIC_MIN_COUNT	10	Min word count for vocabulary inclusion
BASIC_BATCH_SIZE	8	Batch size (unused by SBOW; per-sentence)

Memory Considerations

Dominant cost is the linear output head: `vocab_size * vector_size` parameters plus optimizer state. With 200K vocab and 100 dims:

- Embedding: $200\text{K} \times 100 \times 4 \text{ bytes} = \sim 80\text{MB}$

- Linear head: $100 \times 200K \times 4 \text{ bytes} = \sim 80\text{MB}$
- Adam optimizer: $\sim 320\text{MB}$ total

Full softmax computes $(N, \text{vocab_size})$ logits per sentence. Training runs on CPU by default (BASICMODEL_DEVICE=cpu) to avoid MPS/GPU memory limits.

Phase 2: Network Training (BasicModel.py)

The network learns to predict and reconstruct sentences using pretrained embeddings.

Within-sentence training objective (IR-only)

Within-sentence training is **always IR** (masked-LM at the P-tier). `create_ir_mask` replaces a `mask_rate` fraction of WHAT positions with NULL_PERCEPT and snapshots the pre-mask event on `_ir_pre_mask_input`; `runBatch` computes `MSE(perceptualSpace at masked positions, _ir_pre_mask_input at masked positions)` — no head loss, no reverse pipeline. The legacy `<maskedPrediction>` knob and the AR / ARUS / ARIR modes were retired 2026-05-14; sentence-level AR moved to `InterSentenceLayer` (see `doc/Architecture.md` Section “Sentence-level AR (`InterSentenceLayer`)”).

Knob	Effect
<code>maskRate</code>	Bernoulli mask probability at the P-tier (BERT default 0.15)
<code>reconstructionScale</code>	Blend weight between output and reconstruction loss; <code>total = (1 - r)*output + r*recon</code> . Legacy
<code><reconstruct></code>	Target tier for the optional forward-only reconstruction loss: <code>none / symbols / concepts / both</code> .

`<trainEmbedding>`: Embedding Update Mode

Controls whether and how embeddings evolve during network training. When enabled (CBOW, SBOW, or BOTH), implements EM-like alternation:

- **E-step (network)**: Forward + masked prediction + backward + optimizer.
- **M-step (CBOW/SBOW)**: Run one embedding update on the same sentence.

Value	Embedding	Model layers	Description
NONE	Frozen	Trained	Only model layers train
CBOW	CBOW (padded context, own optimizer)	Trained	Model layers train separately
SBOW	SBOW (centroid, own optimizer)	Trained	Faster variant
BACKPROP	Backprop only	Trained	Codebook trained purely via model loss
BOTH	SBOW post-batch	Trained	Two optimizers
JOINT	Single backward	Trained	Single optimizer: combined model + SBOW loss

Gradient Flow Through Codebook

`<trainEmbedding>` determines whether the codebook weight is **detached** during forward/reverse and whether it appears in the main optimizer:

<code>trainEmbedding</code>	<code>detach()</code>	In main optimizer	Method
NONE	Yes	No	Frozen
CBOW	Yes	No	CBOW (own optimizer)
SBOW	Yes	No	SBOW (own optimizer)
BACKPROP	No	Yes	Model loss only
BOTH	No	Yes	SBOW + backprop

<code>trainEmbedding</code>	<code>detach()</code>	In main optimizer	Method
JOINT	No	Yes	Single combined loss

- **NONE** is recommended for constrained hardware (Apple MPS <16GB).
- **CBOW/SBOW** maintains clean EM separation.
- **BACKPROP** is the simplest trainable mode. Embedding shaped entirely by the task; best for small vocabularies.
- **BOTH risks gradient interference** — reconstruction pulls embeddings apart (distinguishability); SBOW pulls co-occurring words together. These forces can conflict because they use separate optimizers.

JOINT: Single Combined Loss

Avoids EM alternation by computing one combined loss before one backward:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{model}} + \lambda \cdot \mathcal{L}_{\text{SBOW}}$$

where λ is `trainEmbeddingRatio` (default 0.1).

SBOW loss uses the same negative-sampling objective:

$$\mathcal{L}_{\text{SBOW}} = -\frac{1}{N} \sum_{i=1}^N \left[\log \sigma(c_i^\top v_{w_i}) + \sum_{k=1}^K \log \sigma(-c_i^\top v_{w_k^-}) \right]$$

Advantages. One optimizer, one momentum buffer; gradient coherence; no post-batch step.

Trade-off. Both objectives share LR and Adam state. If loss surfaces have very different curvature, λ may need tuning.

Why MSE over Embeddings (not Cross-Entropy)

- “cat” prediction when target is “kitten” incurs less loss than “democracy” — embedding space captures semantic similarity.
- Output dim is the embedding dim (~100) vs. vocab size (~200K).

Training Loop

Data flows through `SentenceStreamDataset` wrapped in `DataLoader`. The ordered training list is split into $B = \text{batchSize}$ contiguous slabs of length $L = \text{len(split)} // B$; at step t , row b is item $b * L + t$. Temporal context is coherent across steps. No per-epoch global shuffle.

For each B-wide batch:

1. `BasicModel.Start(inputData)` cascades reset through every Space.
2. `InputSpace.forward()` lexes/embeds once into $[B, N, D]$ (left-aligned, right-padded to N). No K axis, no cursor unfold.
3. `create_ir_mask` replaces a `mask_rate` fraction of WHAT positions with `NULL_PERCEPT`; pre-mask event stashed on `_ir_pre_mask_input` and the position mask on `_ir_mask_positions`.
4. `_forward_body` runs T stages on B rows.
5. `_forward_head` produces $[B, N, \text{predDim}]$ (a side channel — IR loss is computed at the P-tier, not at the head).
6. `runBatch` computes loss via `TheError.add`:
 - **reconstruction (P-tier)**: MSE between post-body `perceptualSpace` and `_ir_pre_mask_input` at masked positions.

- Optionally `reconstruction_c` / `reconstruction_s` (C-tier / S-tier) when `<reconstruct>` is `concepts` / `symbols` / `both`, weighted by `reconstructionScale`.
 - `arma` (sentence-level): `InterSentenceLayer.observe(s_t)` MSE between the ARMA(p, q) prediction and the current sentence rep, weighted by `armaScale`.
7. One `backward()` + `optimizer.step()` per `DataLoader` yield.
 8. Embedding training (CBOW/SBOW/BOTH) runs once per batch.

SBOW vs CBOW

Property	CBOW	SBOW	Masked Prediction (Phase 2)
Targets per sentence	1 (pick one word)	N (every word)	N (one per masked pos)
Context	Other N-1 words	LOO centroid of N-1	All unmasked (+ future truncation)
Positive updates	1 per step	N per step	N per step
Repulsive force	vocab-1 implicit via softmax	$N \times (\text{vocab}-1)$	Implicit via MSE
Signal density	Low	High	High
Loss	Cross-entropy over vocab	Cross-entropy over vocab	MSE over embeddings
Updates embeddings	Yes (own optimizer)	Yes (own optimizer)	Only for BACKPROP/BOTH/JOINT
Used in (<code><trainEmbedding></code>)	CBOW	SBOW, BOTH, JOINT	BACKPROP, BOTH, JOINT

SBOW provides richer signal — every word acts as both context (for others) and target (predicted from others). The full softmax pushes vectors to distribute across the hypersphere.

Three-File Architecture

Artifact	Example	Contents	Updated by
XML config	<code>data/BasicModel.xml</code>	Architecture, hyperparameters, paths	Hand-edited
Embedding	<code>data/BasicModel.kv</code>	Word vectors ($V \times d$)	Phase 1
Weights	<code>data/BasicModel.ckpt</code>	Model layer parameters	Phase 2

Checkpoint excludes embedding parameters (`wv._vectors`). Enables:

- Retrain embeddings without touching model weights (`--force-embeddings`)
- Retrain model with frozen codebook (`<trainEmbedding>NONE`)
- Swap codebooks between models sharing the same architecture

Embedding Space Geometry

The full softmax provides a distributional guarantee that random negative sampling cannot match. For predicting w_i from centroid c_i :

$$\mathcal{L}_i = -\log \frac{\exp(z_{i,w_i})}{\sum_{j=1}^V \exp(z_{i,j})}$$

The gradient with respect to the logit for any non-target word $k \neq w_i$:

$$\frac{\partial \mathcal{L}_i}{\partial z_{i,k}} = P(k \mid c_i)$$

Words with high softmax probability (too close to the centroid) receive the strongest repulsive gradient; words already far away contribute almost nothing. This naturally spreads vectors across the hypersphere.

At equilibrium, no word can move closer to any centroid without increasing the loss. Random negatives, by contrast, mostly sample words already far away on a high-dimensional hypersphere and contribute little useful gradient.

Profiling

--profile wraps Phase 2 in cProfile:

```
make train_micro_remote # add --profile via train.py
# Or:
```

```
python bin/train.py --model data/BasicModel.xml --profile --max-docs 500
```

Output: a .prof file in output/profiles/ plus a top-30 summary to stdout. View with snakeviz:

```
pip install snakeviz
snakeviz output/profiles/train_20260319_111441.prof
```

For live profiling, py-spy can attach by PID (requires root on macOS):

```
pip install py-spy
sudo py-spy record -o profile.svg --pid <PID> --duration 30
```

Ergodic Exploration via Adaptive Bias-Variance Control

BasicModel decouples **what** the network learns (weights W) from **how much it explores** (a scalar α governing the bias-variance tradeoff). Two algorithms operate in tandem:

Component	Algorithm	Role
Weights W	Standard Adam	Gradient descent on the loss landscape
Tradeoff α	Gradient Energy Sensor	Measures loss-surface curvature to set exploration

α **is not trained by gradient descent**. It is a *sensor* reading the gradient energy through the temperature parameter and converting it into an exploration policy.

1. The Ergodic Weights

Each layer uses an effective weight matrix sampled from a locally specified ergodic distribution:

$$W_{\text{eff}} = \alpha \cdot W + (1 - \alpha) \cdot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I)$$

W stores the learned structure that shapes the distribution as α rises. Derived quantities:

$$\text{bias} = \alpha, \quad \text{var} = 1 - \alpha$$

- $\alpha = 0$: Pure exploration. $W_{\text{eff}} = \varepsilon$. Random noise.

- $\alpha = 1$: Pure exploitation. $W_{\text{eff}} = W$. Learned weights only.
- $0 < \alpha < 1$: Interpolation.

Training begins at $\alpha = 0$ and transitions autonomously toward exploitation as the loss landscape flattens.

2. Why Standard Adam Fails on α

The temperature $T = 1 - \alpha$ receives gradients via:

$$\frac{\partial \mathcal{L}}{\partial T} = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \varepsilon_{ij}$$

Because ε is re-sampled every forward pass, this gradient is **zero-mean**:

$$\mathbb{E}\left[\frac{\partial \mathcal{L}}{\partial T}\right] = \sum_{i,j} \frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \cdot \mathbb{E}[\varepsilon_{ij}] = 0$$

Standard Adam's first moment kills the signal: $m_t \approx 0 \Rightarrow m_t / \sqrt{v_t} \approx 0$. Adam produces no update.

3. The Gradient Energy Sensor

3.1 Modified Second-Moment Estimator

Discard the first moment; use only the second moment as **output** rather than normalizer:

$$v_t = \beta \cdot v_{t-1} + (1 - \beta) \cdot g_t^2$$

where $g_t = \partial \mathcal{L} / \partial T$ and $\beta = 0.999$.

3.2 Bias Correction

$$\hat{v}_t = \frac{v_t}{1 - \beta^t}$$

3.3 What $\hat{v}_t$ Measures

The second moment of a zero-mean random variable is its variance:

$$\mathbb{E}[g_t^2] = \text{Var}(g_t) = \sum_{i,j} \left(\frac{\partial \mathcal{L}}{\partial (W_{\text{eff}})_{ij}} \right)^2 = \left\| \frac{\partial \mathcal{L}}{\partial W_{\text{eff}}} \right\|_F^2$$

The **squared Frobenius norm** of the loss gradient — the **gradient energy**.

Gradient energy	Meaning	Desired behavior
High $\hat{v}_t$	Loss sensitive to weight changes	Keep exploring (low α)
Low $\hat{v}_t$	Loss surface is flat	Exploit (high α)

3.4 The Alpha Update Rule

$$\alpha_t = \frac{1}{1 + \tau \cdot \sqrt{\hat{v}_t}}$$

where τ (`global_temp`) is a sensitivity knob.

Properties:

- $\sqrt{\hat{v}_t} \rightarrow 0 \Rightarrow \alpha_t \rightarrow 1$ (exploit)
- $\sqrt{\hat{v}_t} \rightarrow \infty \Rightarrow \alpha_t \rightarrow 0$ (explore)
- $\alpha_t \in (0, 1)$ always
- τ controls transition speed

After computing α_t : $\text{bias}_t = \alpha_t$, $\text{var}_t = 1 - \alpha_t$.

3.5 Layer-Local Certainty (Per-Neuron Sigma)

Each ergodic layer maintains per-neuron **sigma** — running variance of its gradient energy, tracked via Welford’s algorithm in `observe_sigma()`. Sigma drives per-neuron bias and var:

$$\text{var}_j = \frac{\sigma_j}{\sigma_j + \kappa}, \quad \text{bias}_j = 1 - \text{var}_j$$

Low sigma (consistent gradient, found a minimum) yields high bias, low var. High sigma (unstable gradient) yields low bias, high var.

`set_sigma(sigma)` controls κ :

- `sigma=1`: encourage exploration (low κ)
- `sigma=0`: suppress exploration (high κ , $\text{var} \approx 0$)

4. Comparison: Adam vs. Gradient Energy Sensor

Property	Adam (for W)	Sensor (for α)
Type	Optimizer	Sensor (measurement)
First moment m_t	Yes — tracks direction	No — direction is zero-mean
Second moment v_t	Yes — normalizes step size	Yes — is the output
Role of $\sqrt{v_t}$	Denominator (adaptive LR)	Numerator (energy estimate)
Update rule	$W \leftarrow W - \eta \cdot m / \sqrt{v}$	$\alpha \leftarrow 1 / (1 + \tau \sqrt{\hat{v}})$
Descent?	Yes	No — maps energy to policy
Zero-mean safe?	No — produces $0 / \sqrt{v} = 0$	Yes — by design

5. Derivation: Why Drop m_t

Let $g_t = \nabla_T \mathcal{L}$. Since noise ε is i.i.d. each step, $\mathbb{E}[g_t] = 0$. The first moment EMA $m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$ gives $\mathbb{E}[m_t] = \beta_1 \mathbb{E}[m_{t-1}]$. By induction $\mathbb{E}[m_t] = \beta_1^t \mathbb{E}[m_0] = 0$ — exponentially to zero, carrying **no information**.

The second moment converges to a meaningful quantity:

$$\mathbb{E}[v_t] \rightarrow \mathbb{E}[g_t^2] = \left\| \nabla_{W_{\text{eff}}} \mathcal{L} \right\|_F^2$$

The gradient energy — exactly what we need.

6. Dropout via Bias

`bias` and `var` support dropout:

$$\text{bias}_t^{(\text{drop})} = \begin{cases} 0 & \text{with probability } p \\ \alpha_t & \text{with probability } 1 - p \end{cases}$$

When bias is dropped to zero, the layer acts as pure noise regardless of α . The temperature $T = 1 - \alpha$ remains unchanged so noise contribution is consistent.

Dropout is a schedule on bias: ordinary Bernoulli dropout is the binary case. Annealed dropout corresponds to increasing the expected bias over training. Nonzero temperature bakes regularization directly into the model — every forward pass perturbs the effective weights, discouraging brittle co-adaptation.

7. The `global_temp` Sensitivity Knob

τ (`global_temp`) scales responsiveness:

τ	Effect
$\tau = 0$	$\alpha = 1$ always — pure exploitation
$\tau \ll 1$	Quickly converges to $\alpha \approx 1$
$\tau = 1$	Balanced — α tracks gradient energy on natural scale
$\tau \gg 1$	Maintains high exploration even with moderate gradients

τ can be scheduled (e.g., annealed from high to low) for a coarse exploration-to-exploitation curriculum.

8. Initialization and Bootstrap

Problem. If initialized at $\alpha = 1$ (pure exploitation), the noise term vanishes and $\partial\mathcal{L}/\partial T = 0$. The sensor would read zero energy and keep $\alpha = 1$ forever — an **exploitation trap**.

Solution. Initialize at $\alpha = 0$.

At $\alpha = 0$:

- $W_{\text{eff}} = \varepsilon$ — pure noise carries gradients through the network
- Temperature gradient is nonzero: $g_t = \sum_{ij} (\partial\mathcal{L}/\partial\varepsilon_{ij}) \cdot \varepsilon_{ij} \neq 0$
- Sensor measures initial gradient energy and begins tuning α upward
- As W learns useful structure, energy decreases and α rises

Zero-initializing W is safe and often cleaner than random initialization. Symmetry is broken by the sampled noise; learned weights don't contribute at $\alpha = 0$ anyway. Starting from $W = 0$ makes the exploratory regime honest.

9. Layer Architecture

All ergodic layers follow the effective-weight pattern. When `ergodic=True`, noise buffers are registered and weights are zero-initialized.

ErgodicLayer Base Class

`ErgodicLayer` (`Model.py`) provides per-neuron `bias` and `var` buffers driven by gradient variance (`observe_sigma()` / `sigma_to_ergodic()`). Subclasses with child `ErgodicLayers` must forward `set_sigma()` and `paramUpdate()`.

InvertibleLinearLayer — factor-level noise injection

LDU factorisation $W = L \cdot D_{\text{embed}} \cdot U$ (see `Architecture.md`). Exact inverse via triangular solves; no SVD.

When `ergodic=True`, noise is injected at the factor level to preserve exact invertibility at all temperatures:

$$\begin{aligned} L_{\text{eff}} &= I + \text{strict_lower}(\text{raw_L} + t \cdot \text{noise_L}) \\ U_{\text{eff}} &= I + \text{strict_upper}(\text{raw_U} + t \cdot \text{noise_U}) \\ d_{\text{eff}} &= b \cdot d_{\text{clamped}} + t \cdot \text{noise_d} \end{aligned}$$

W_{eff} stays in LDU form, so its exact inverse is always available — no approximation regardless of noise level. Compare to matrix-level blending $W_{\text{eff}} = b \cdot W + t \cdot N$, which destroys LDU structure:

$$W_{\text{eff}} \cdot W_{\text{eff}}^{-1} \approx (b^2 + t^2)I + bt(WN^{-1} + NW^{-1}) \neq I$$

Error grows with temperature — exactly the regime where exploration is most active. Factor-level injection avoids this.

`noise_d` is sampled with magnitude in $[\text{eps}, 1]$ and random sign, so it's always invertible; combined with the `stable=True` clamp on the deterministic diagonal, `d_eff` stays bounded away from zero.

`SigmaLayer(invertible=True)` and `PiLayer(invertible=True)` use `InvertibleLinearLayer(ergodic=True)` internally. `bias` and `var` from the parent `ErgodicLayer` forward as `bias=` / `temp=` arguments, so the gradient-energy sensor governs noise injection in the inner linear primitive.

10. Training Loop Integration

```
optimizer = torch.optim.Adam(model.getParameters(), lr=lr) # Adam for W
loss = model(x, y) # forward W_eff = bias*W + var*noise
loss.backward() # Adam: dL/dW; sensor: dL/dT
optimizer.step()
model.paramUpdate() # sensor updates alpha
```

11. Summary of the Full Algorithm

Initialize: $\alpha_0 = 0$, $v_0 = 0$, $t = 0$, $\beta = 0.999$.

Each step:

1. Sample $\varepsilon \sim \mathcal{N}(0, I)$.
2. $W_{\text{eff}} = \alpha_t W + (1 - \alpha_t) \varepsilon$.
3. Forward: $\hat{y} = f(x; W_{\text{eff}})$; loss $\mathcal{L}(\hat{y}, y)$.

4. Backward: $\nabla_W \mathcal{L}$ and $g_t = \nabla_T \mathcal{L}$.
5. **Adam** updates W .
6. **Sensor** updates α :

$$v_t \leftarrow \beta v_{t-1} + (1 - \beta) g_t^2, \quad \hat{v}_t = \frac{v_t}{1 - \beta^t}, \quad \text{var}_j = \frac{\hat{v}_j}{\hat{v}_j + \kappa}, \quad \text{bias}_j = 1 - \text{var}_j$$

7. Zero temperature gradient: $g_t \leftarrow 0$.

Alec Rogers

September 24, 2025

Abstract

Problem

Society depends on machine minds, but we lack a good theory of them. While human and machine minds functionally overlap, their design goals differ, and treating a machine mind as a human mind has significant consequences. We should have a different theory of mind for machine minds, informed by how those minds are designed and trained.

Background

Anthropomorphizing machines helps us understand them relative to human minds. This essay focuses on three questions: what are machine minds, what do machine minds feel, and what do machine minds know.

Solution

Three measures make a machine mind more humanistic: weight ergodicity, network invertibility, and output certainty. Weight ergodicity treats weights as samples from a random process rather than fixed constants. Network invertibility makes the architecture bidirectional and partially invertible. Output certainty means knowing that it does not know, rather than only the probability of one answer versus another. All models tested use a 20-neuron, single hidden layer on MNIST.

Invertibility matters for meaningful semantic embedding: an LLM's symbols should correspond to something we can identify, not be abstract tokens inside Searle's Chinese translation engine.

Background

Scope: Large Language Models (reinforcement learning with known inputs/outputs and unknown weights — the transformer architecture).

Part I: What Machine Minds Are — Weight Ergodicity

Weights as Ergodic Samples

Measurement theory views weights as **samples from an ergodic distribution** rather than fixed constants. W at any moment is one realization of a stochastic process:

$$W = \mu M + \sigma V$$

where μ and σ are the bias and variance parameters; M has norm 1 and zero variance (analogous to a typical weight matrix), and V is sampled from a standard normal with unit variance.

This perspective comes from **Lagrangian Field-theoretic Gradient Control** (LFGC), treating the weight landscape as a field where boundary conditions shape the distribution from which weights are drawn.

The ergodic hypothesis guarantees that time averages (training steps) equal ensemble averages (weight configurations).

See Rogers, A. (2026), *Local Freedom under Global Constraint*, Zenodo, doi: 10.5281/zenodo.18644302.

Temperature and Annealing

A single per-layer *temperature* parameter, updated via ADAM, characterizes mean and variance:

$$\mu = 1 - temp, \quad \sigma = temp$$

Temperature begins at 1 and decreases to 0.

Advantages of Ergodic Weights

1. **No learning rate required.** Biases find different weight-space locations via randomness and converge as temperature decreases.
2. **Zero initialization.** Weights can be initialized to zero — annealing bounces the model out of local minima more effectively than gradient steps.
3. **Theoretical convergence guarantee.** Simulated-annealing literature provides a guarantee of convergence to global optimum (given infinite iterations).

Standard *dropout* can be interpreted as per-neuron variance in the bias parameter. The same regularization occurs naturally from time-varying weight biases.

Part II: What Machine Minds Feel — Network Invertibility

Bidirectional Perception

Egocentric viewpoint sees perception as passive; physics suggests it is bidirectional. Approximated via invertibility of weight multiplications and activation functions.

Symbols, Worldlines, and Karmic Inertia

The symbols an LLM processes have meanings given by **worldlines** — the accumulated history of contexts in which a token has appeared, the gradients that have shaped its embedding, and the network states it has participated in. The meaning of a symbol is the integral of its worldline.

Weight adaptation (or inference) puts a **force** into the system. That force adapts either the system’s inputs or outputs depending on which has less **karmic inertia** — resistance to change. The bidirectional architecture means the force propagates in both directions:

- **Forward (encoding):** input representation adapts to match learned categories — perception shaped by expectation.
- **Reverse (decoding):** internal state adapts to reconstruct the input — expectation grounded in perception.

The invertible architecture makes both directions explicit and trainable.

Implementation

Even when not invertible, bidirectionality plays a role in backpropagation and autoencoders. We examine simultaneously training in both forward (predict output from input) and reverse (predict input from output of the *penultimate* layer) directions — the network is composed of a bijective front (*recognizer*) and a surjective back (*generalizer*). See also Rogers et al. 2001 for related work on sharing weights between policy and critic.

Part III: What Machine Minds Know — Output Certainty

Certainty vs. Probability

A network’s error function corresponds to its desire. Cross-entropy loss *requires* output classification — expressing the probability of one class *versus another*, not the certainty that the input belongs to a given class.

To remedy: blend cross-entropy with a product of cross-entropy and the prediction norm. Zero output is penalized by cross-entropy; incorrect predictions carry additional penalty proportional to magnitude.

Certainty-Weighted Loss

$$\begin{aligned} error &= -\sum_{c=1}^N t_c \log(p_c) \\ certainty &= |\hat{y}| \\ surprise &= (\alpha) \cdot certainty \cdot error + (1 - \alpha) \cdot error \end{aligned}$$

For amplitude certainty:

$$error = (\hat{y} - y)^2$$

Knowing What You Do Not Know

A network should **know that it does not know**, not merely the probability of one answer vs. another. Cross-entropy forces probability distribution across all known classes, even when “I don’t know” is correct. Certainty weighting separates “which class?” from “how sure?”

Methods

Python + PyTorch. MNIST dataset: 60K train, 10K test, 28×28 monochrome at 256 bit depth. Inputs are preprocessed (mean and variance removed) and shuffled.

Input: 28×28 array. Output: one-hot target class. Architecture: 20 hidden unit NN. Batch size 10, 9 epochs, 7 trials for error bars.

Standard benchmark: ReLU, LR 0.01, ADAM, cross-entropy loss.

Base ergodic: zero-init weights, single per-layer temperature, ADAM updates temperature, dropout (0.75) on middle layer.

Five models evaluated: base + four ergodic variants (one plain ergodic, one with input normalization, two reversible — one with separate forward/backward layers, one invertible). The invertible model represents W in SVD form (rotation, eigenvalues, rotation) using Givens rotations and diagonal eigenvalues to avoid direct matrix inversion.

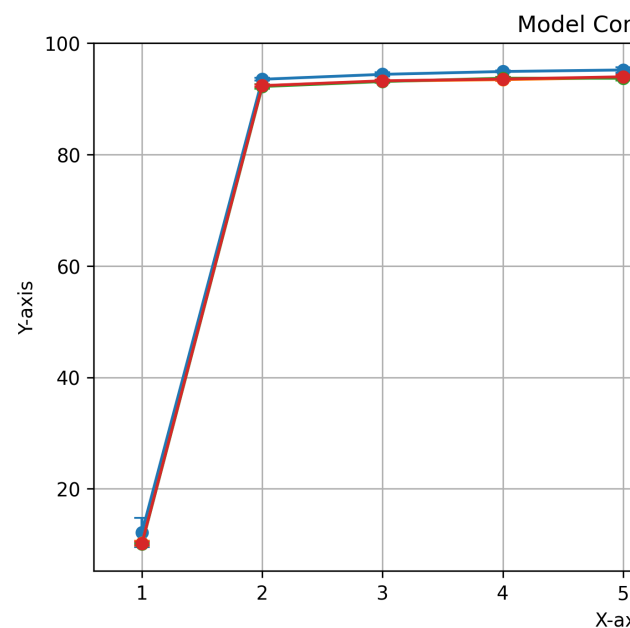
Weight update equations for ergodic models avoid a learning rate. Compared to traditional gradient descent ($\Delta_W = \alpha(y - \hat{y})\nabla_F x$):

$$\begin{aligned} e &= y - \hat{y} \\ c &= \alpha \hat{y}^2 + (1 - \alpha) \\ \Delta_W &= (ce - \alpha \hat{y} e^2)x \end{aligned}$$

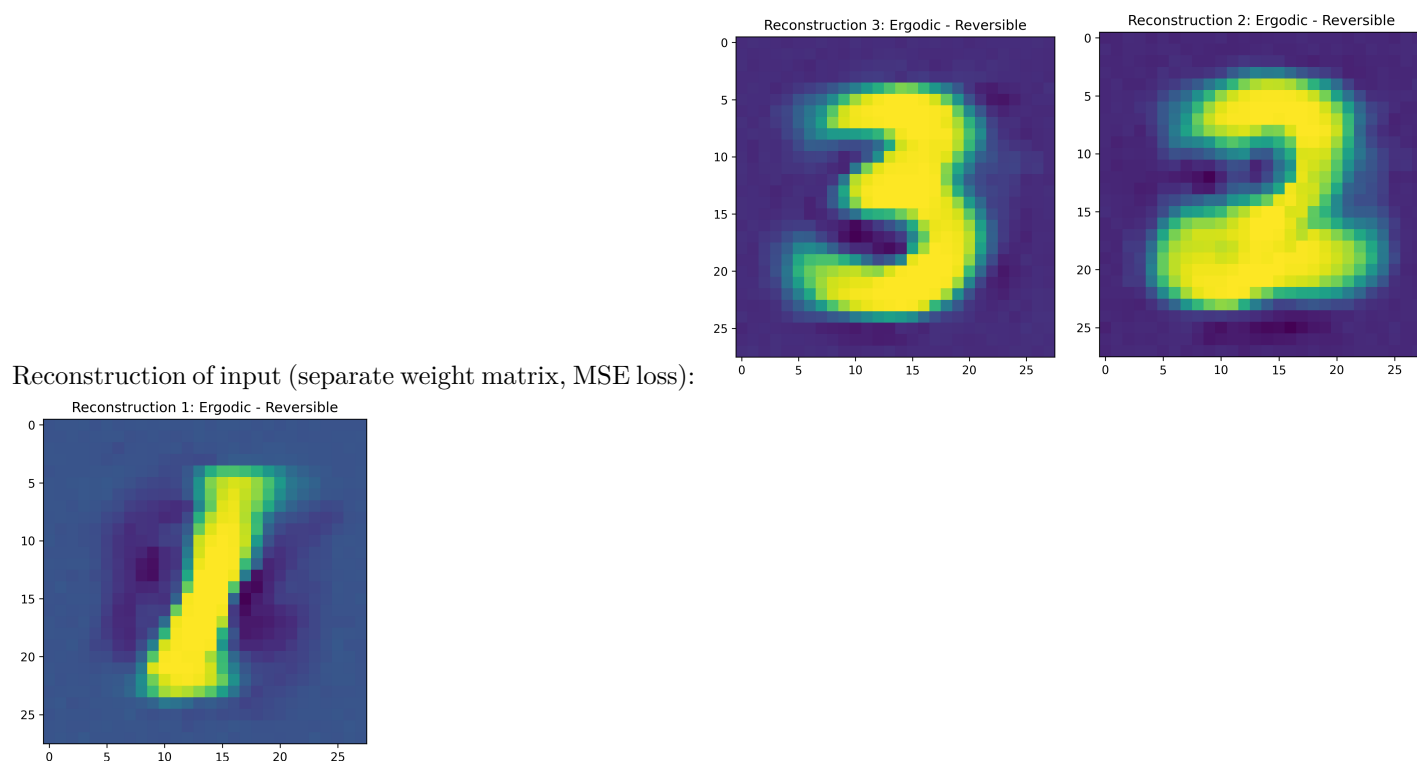
The network simultaneously minimizes error and “incorrect certainty”.

Results

Model comparison with error bars shows the simple model performs slightly better than all proposed



paradigms, with all proposed paradigms performing almost identically:



Discussion

Benefits:

- Random initialization unnecessary — only the simple model with random weight variation shows significant performance variation.

- Learning rate replaced by weight noise variance (dropout role replaced by bias variance).
- Error function encapsulates output certainty in addition to between-class certainty.

Limitations:

- Pearson correlation between certainty and accuracy is insignificant — the implementation used $\text{certainty} = p_c$ (softmax output) rather than $|\hat{y}|$, creating contradictory constraints since softmax outputs sum to 1 while the certainty norm imposes another constraint. Corrected code should use norm-weighted MSE.
- PyTorch autograd tunes weights directly, interfering with simultaneous temperature tuning in ways not captured.

Conclusion

Strong philosophical grounding for several paradigms, but implementation issues limited testing. Effectiveness in multilayer NNs and challenging tasks remains unclear.

References

- Rogers, Shannon, Lendaris; 2001: *A comparison of DHP based antecedent parameter tuning strategies for fuzzy control*, IFSA/NAFIPS, IEEE.
- Rogers, A; 2003: *Analysis of the Instantaneous Estimate of Autocorrelation*, unpublished.
- Rogers, A; 2026: *Local Freedom under Global Constraint*, Zenodo, doi: 10.5281/zenodo.18644302.

XML Configuration Parameters

Model configuration is specified in XML files (e.g. `data/XOR_exact.xml`). Default values are loaded from `data/model.xml` and overridden by model-specific configs. The schema is in `data/model.xsd`.

Overlay merge order: `data/model.xml` $\rightarrow$ `<config>.xml` $\rightarrow$ runtime overrides.

Note on naming: the schema and the XML configs use `nInput` / `nOutput` (sequence-length sentinels), `nDim` (per-vector dim), `nVectors` (codebook size), `nWhere` / `nWhen` (positional / temporal encoding widths), and `nInputDim` / `nOutputDim` (raw shape hints for the LiftingLayer). Older docs sometimes call these `nActive`; treat that as the same thing as `nOutput`.

<architecture>

Core model settings. Training and data parameters are in nested sub-elements `<training>` and `<data>` (see below).

Parameter	Type	Default	Description
<code>modelType</code>	string	"simple"	<code>simple</code> (feedforward), <code>embedding</code> (LM / chat with sequence processing),
<code>reconstruct</code>	string	"symbols"	Reverse-pass mode: <code>NONE</code> disables; <code>symbols</code> reconstructs from cached out
<code>conceptualOrder</code>	int	1	Iteration depth of the $P \rightarrow C \rightarrow S$ pipeline. <code>order > 1</code> partitions the symbo
<code>useButterflies</code>	bool	false	Butterfly-mode Pi / Sigma layers (N-halving per order). Mutually exclus
<code>processSymbols</code>	bool	false	Apply extra symbolic processing after Sigma.
<code>monotonic</code>	bool	false	Constrain invertible Sigma / Pi to $W \geq 0$ so the lift / lower chain is orde
<code>ergodic</code>	bool	false	Ergodic exploration: every Layer uses $W_{\text{eff}} = \text{bias} * W + \text{temp} * \text{noi}$
<code>naive</code>	bool	false	Materialise W_{eff} densely in <code>InvertibleLinearLayer</code> . Slower; debugging
<code>nWhere</code>	int	0	Positional-encoding dimensions appended to each vector. <code>> 0</code> enables Pos
<code>nWhen</code>	int	0	Temporal-encoding dimensions appended to each vector. <code>> 0</code> enables Tem
<code>embeddingPath</code>	string	(empty)	gensim <code>KeyedVectors</code> path. Empty disables embedding load.

Parameter	Type	Default	Description
<code>weightsPath</code>	string	(empty)	Model weights checkpoint path. Empty falls back to <code>output/<name>.ckpt</code> .
<code>maxResponseLength</code>	int	4096	Inference token budget (characters / bytes / tokens). Caps output alongs
<code>amp</code>	string	"off"	AMP autocast mode: <code>off</code> , <code>fp16</code> , <code>bf16</code> .
<code>truthBiasScale</code>	decimal	0.1	Strength of luminosity-based bias on concept input during forward.
<code>LuminosityWeight</code>	decimal	0.1	Multiplicative loss penalty for incoherent truths.
<code>UniversalityWeight</code>	decimal	0.1	Multiplicative loss penalty for unkind propositions.
<code>allowExcludedMiddle</code>	int	1	Catuskoti policy: 1 permits NEITHER, -1 enforces classical LEM.
<code>allowContradiction</code>	int	0	Catuskoti policy: 0 forbids BOTH (non-contradiction), 1 permits paracon
<code>gateL1Lambda</code>	float	0.0	L1 sparsity penalty on lift / lower gates. 0 disables.
<code>loadBalanceWeight</code>	float	0.0	Sparse-MoE load-balance loss weight. Active only when <code>WordSpace.char</code>
<code>l1Lambda</code>	decimal	0.0	Architecture-wide L1 penalty hook. Most configs leave this at 0 and use
<code>discontinuityLambda</code>	decimal	0.0	Architecture-wide discontinuity penalty hook (legacy).
<code>conceptualWidth</code>	string	"tapered"	Symbol-partition geometry for <code>conceptualOrder > 1</code> . <code>tapered</code> = geome
<code>perStageChartReparse</code>	bool	false	Each <code>conceptualOrder</code> stage runs its own chart pass on the previous stag
<code>mereologicalTreeSize</code>	int	–	MereologicalTree capacity. Match to <code>SymbolicSpace.nVectors</code> so <code>_argm</code>
<code>writeSyntax</code>	bool	false	Dump POS-labelled syntax tree at end of every <code>MentalModel.forward()</code>
<code>syntaxOutPath</code>	string	–	Output path for <code>writeSyntax</code> .

Luminosity and universality are **multiplicative** — they scale the total loss by

$$(1 + \text{LuminosityWeight} \cdot (1 - \text{luminosity}) + \text{UniversalityWeight} \cdot (1 - \text{universality})).$$

`TruthLoss` (in `<training>`, below) is **additive** — it directly penalises propositions that contradict the `TruthSet`. Both coexist. See `Ethics.md` Section Universality and `Reasoning.md` Section `TruthLoss` for details.

`<architecture><data>`

Data loading and filtering.

Parameter	Type	Default	Description
<code>dataset</code>	string	"xor"	Dataset key: <code>xor</code> , <code>mnist</code> , <code>tomatoes</code> , <code>text</code> , <code>inline</code> . Omit for inference-only mode.
<code>numShards</code>	int	1	Shard count (streaming text datasets).
<code>maxDocs</code>	int	10000	Per-shard document cap.
<code>shardDir</code>	string	(empty)	Text shard directory (e.g. <code>data/fineweb</code>). Only used when <code>dataset=text</code> .
<code>minFrequency</code>	float	0.0	Vocabulary admission threshold (fraction of corpus). Words below it are held in a
<code>classificationMin</code>	float	–	Lower bound for classification-accuracy reporting.
<code>classificationMax</code>	float	–	Upper bound for classification-accuracy reporting.

Inline-mode configs additionally accept `<input use="train|test|validation">...|...|...</input>` and matching `<output>` elements directly inside `<data>`.

`<architecture><training>`

Training loop and I/O.

Parameter	Type	Default	Description
numTrials	int	1	Independent training runs per invocation.
numEpochs	int	3	Epochs per trial.
batchSize	int	64	Contiguous streams through the dataset. Each batch row b receives the n
numWorkers	int	0	DataLoader prefetch workers. 0 = synchronous in-process batch assembly
learningRate	float	0.001	Adam learning rate.
reconstructionScale	float	0.5	Weight of reconstruction loss vs prediction loss in $[0, 1]$: $\mathcal{L}_{\text{total}} = (1 - r) \mathcal{L}$
whatScale	float	0.7	Loss weight on the <code>.what</code> (content) channel.
whereScale	float	0.2	Loss weight on the <code>.where</code> (positional) channel.
whenScale	float	0.1	Loss weight on the <code>.when</code> (temporal) channel.
negSamples	int	64	Negative samples per positive for CBOW / SBOW. Memory cost: $O(\text{batch}$
TruthLoss	float	0.0	Additive penalty for propositions that contradict the TruthSet (union-nor
maskedPrediction	string	"NONE"	Masked / autoregressive mode: NONE, IR, AR, ARUS, ARIR. See Training.m
maskRate	float	0.15	IR-mode Bernoulli mask rate. BERT default 0.15. Ignored when maskedP
trainEmbedding	string	"NONE"	Embedding update mode: NONE, CBOW, SBOW, BACKPROP, BOTH, JOINT. See
embeddingScale	float	0.25	JOINT-mode embedding loss weight λ : $\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{model}} + \lambda \cdot \mathcal{L}_{\text{emb}}$. Ignor
autoload	bool	true	Load weights from <code>weightsPath</code> at construction. Set <code>false</code> for fresh train
autosave	bool	false	Save weights after training completes.
checkpointEveryBatches	int	0	Save mid-training checkpoint every N optimizer steps. 0 disables periodic
profile	bool	false	cProfile the training loop.
certainty	bool	false	Per-neuron certainty tracking in ergodic layers. Allows individual neurons
sentencePrediction	bool	false	Inter-sentence (DiscourseSpace) contrastive loss. Dual-force cosine over th
sentenceContextWindow	int	12	Recent-sentence context window length (in sentences).
sentenceCentroidHistory	int	3	Historical centroids retained for repulsion.
sentenceLambda	float	1.01	Geometric decay base for older centroids' repulsive weight.
sentencePredictionScale	float	0.1	Legacy combined-loss scale; also fallback for the contrastive term.
sentencePredictiveScale	float	0.1	AR predictive cosine loss vs the 'S
sentenceContrastiveScale	float	0.1	Contrastive-only scale (split from <code>sentencePredictionScale</code>).
sentencePrimingScale	float	0.05	AR prediction cast into <code>concept_dim</code> and added as a bias to <code>concept_in</code>

<trainEmbedding> — Embedding Update Modes

Value	Embedding method	Model layers	Gradients through codebook
NONE	Frozen	Trained	No
CBOW	CBOW (padded context)	Trained	No
SBOW	SBOW (leave-one-out centroid)	Trained	No
BACKPROP	Backprop only	Trained	Yes
BOTH	SBOW post-batch	Trained	Yes (two optimizers)
JOINT	Single backward	Trained	Yes ($\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{model}} + \lambda \cdot \mathcal{L}_{\text{SBOW}}$)

<architecture><server>

HTTP server settings (used by `serve.py`).

Parameter	Type	Default	Description
host	string	"127.0.0.1"	Bind address.
port	int	8001	Port. Override per model to run several in parallel.
timeout	int	120	Per-request timeout (s).

BasicModel / MentalModel Quick Reference

The two configurations that train the full $P \rightarrow C \rightarrow S$ pipeline:

```
<architecture>
  <truthBiasScale>0.1</truthBiasScale>
  <LuminosityWeight>0.1</LuminosityWeight>
  <UniversalityWeight>0.1</UniversalityWeight>
  <conceptualOrder>1</conceptualOrder>
  <useButterflies>false</useButterflies>
  <monotonic>false</monotonic>
</architecture>

<training>
  <TruthLoss>0.0</TruthLoss>
</training>
```

The grammar section of `MentalModel.xml` declares `lift` as an S-tier binary rule:

```
<S>lift(S, S)</S>      <!-- lift is binary; SVO is assembled by
                        the chart-compose path S -> S VO,
                        VO -> V O, not a ternary lift -->
```

Transitive SVO is produced compositionally via the typed chart rules ($S \rightarrow S VO$ / $VO \rightarrow V O$) when `<chartCompose>true</chartCompose>`; see `Language.md` and `Ethics.md` “Universality” for the architectural details.

`useGrammar` accepts `none`, `all`, and legacy `thoughtFree`; the target Shamatha Speech work adds / aliases `shamathaSpeech` for the narrow DNF object grammar. Full grammar mode is mutually exclusive with `useButterflies`. Shamatha Speech $\neq$ serial mode: serial mode is the rolling-cursor / next-token runtime path; Shamatha Speech is a grammar policy that requires every logical merge to preserve connected `where()` and continuous `when()` support so the resulting DNF denotes one object.

Space sections

Every space block uses the same sentinel convention:

- `nInput = 0` — `InputSpace`: derive from `TheData`; other spaces: derive from the previous space’s `outputShape[0]`.
- `nOutput = 0` — same as `nInput` for this space (passthrough count).
- `nVectors = 0` — same as `nOutput` (used for non-codebook spaces).

Per-space `nWhere` / `nWhen` override `architecture.nWhere` / `nWhen` when present.

<InputSpace>

Lifts raw data into the model’s internal representation.

Parameter	Type	Default	Description
<code>nOutput</code>	int	sentinel	Sequence length: max tokens per input. For XOR: 2. For text models, OOV
<code>nDim</code>	int	1	Per-vector content dim.
<code>nInputDim</code> / <code>nOutputDim</code>	int	0	Raw shape hints. 0 defers to the loader’s native dim.
<code>nVectors</code>	int	sentinel	Codebook size. Defaults to <code>nOutput</code> .
<code>nWhere</code> / <code>nWhen</code>	int	inherit	Positional / temporal dim overrides.
<code>codebook</code>	bool	false	Discrete (codebook) input values.
<code>lexer</code>	string	"word"	Tokenization mode: <code>word</code> , <code>sentence</code> , <code>byte</code> .

`inputShape` uses the data's native dim (e.g. 784 for MNIST, `inputLength` for text); `outputShape` uses `nDim` from `TheObjectEncoding`. `LiftingLayer` bridges the two.

<PerceptualSpace>

Transforms lifted input into perceptual features via Pi layers (multiplicative interactions).

Parameter	Type	Default	Description
<code>nOutput</code>	int	sentinel	Number of active perceptual feature vectors. Should be $\geq$ <code>InputSpace.nOutput</code> .
<code>nDim</code>	int	1	Per-vector content dim.
<code>nVectors</code>	int	sentinel	Codebook size. When $>$ <code>nOutput</code> , enables vector quantization with top-k selection.
<code>invertible</code>	bool	false	true : one <code>PiLayer(invertible=True)</code> handles both directions; false : separate p
<code>hasAttention</code>	bool	true	Attention reweighting of percepts before the Pi fold.
<code>nonlinear</code>	bool	true	Tanh-bound output to $[-1, 1]$.
<code>codebook</code>	bool	false	When true , <code>.what</code> is a learnable <code>Codebook</code> ; when false , a passthrough <code>Tensor</code> .
<code>chunking</code>	string	"lexicon"	Multi-token chunking strategy: <code>lexicon</code> , <code>bpe</code> .
<code>wordLearning</code>	int	0	Active lexicon-growth mode. 0 = frozen codebook (training default). ≥ 1 = on fir
<code>bivectorOutput</code>	bool	false	Applies the Q2 promotion $(a_P, a_N) = (\max(0, x), \max(0, -x))$ per slot and writes

Pi layer math: $y_j = b_j \prod_i (1 + W_{ji} x_i)$. See `Architecture.md`. `InvertiblePiLayer` has been merged into `PiLayer(invertible=True)`.

<ConceptualSpace>

Transforms perceptual features into abstract concepts via Sigma layers (additive / linear).

Parameter	Type	Default	Description
<code>nOutput</code>	int	sentinel	Active concept vectors. For XOR: 3.
<code>nDim</code>	int	1	Per-vector content dim.
<code>nVectors</code>	int	sentinel	Codebook size.
<code>invertible</code>	bool	false	true : <code>SigmaLayer(invertible=True)</code> (exact inversion via $\text{atanh} + W^{-1}$); false : separ
<code>hasAttention</code>	bool	false	Attention in conceptual processing. Violates position locality required by <code>serial_mode</code> .
<code>nonlinear</code>	bool	true	Tanh-bound output to $[-1, 1]$.
<code>codebook</code>	bool	false	Codebook quantization of concept directions.

Sigma layer math: $y_j = \tanh(Wx + b)$. See `Architecture.md`. `InvertibleSigmaLayer` has been merged into `SigmaLayer(invertible=True)`.

<SymbolicSpace>

Discrete symbolic representation — the information bottleneck between perception and output. See `Language.md` for the design.

Parameter	Type	Default	Description
<code>nOutput</code>	int	sentinel	Total symbols. When reconstruction symbols are enabled, <code>nOutputSym</code>
<code>nDim</code>	int	1	Per-vector content dim (typically 1 or 2 for bivector mode).
<code>nVectors</code>	int	sentinel	Codebook size. When <code>codebook=true</code> , equals the number of symbol p
<code>codebook</code>	bool	false	Codebook quantization. Required for the full symbolic pipeline.
<code>nonlinear</code>	bool	true	Tanh-bound the activation.
<code>sortNetwork</code>	bool	false	Bitonic-sort regularizer on the codebook ordering.
<code>sortPasses</code>	int	0	Sort-network depth. 0 disables.

Parameter	Type	Default	Description
<code>l1Lambda</code>	decimal	0.01	L1 sparsity penalty on the symbolic activation.
<code>symbolResidualScale</code>	decimal	1.0	Symbol-residual injection scale (legacy mereology hook).
<code>outputSymbolResidualScale</code>	decimal	0.0	Output-side symbol-residual injection scale.
<code>commitmentBeta</code>	float	0.25	VQ-VAE commitment-loss β (when <code>useVQVAE=true</code>).
<code>useVQVAE</code>	bool	false	VQ-VAE-style quantization rather than EMA codebook.
<code>decorrelationWeight</code>	decimal	0.0	<code>ImpenetrableLayer</code> decorrelation regularizer on codebook rows.
<code>spectralFlatnessWeight</code>	decimal	0.0	<code>ImpenetrableLayer</code> spectral-flatness regularizer.
<code>truthMinMagnitude</code>	decimal	0.3	Minimum magnitude before a <code>SymbolicSpace</code> activation contributes to
<code>bivectorOutput</code>	bool	false	Forward returns per-prototype catuskoti bivector $[B, V_S, 2]$ via <code>Codebo</code>

`SymbolicSpace` owns one `SigmaLayer(nConcepts, nSymbols, invertible=True, monotonic=monotonic)` at `self.sigma` bridging the $C \leftrightarrow S$ boundary in both directions via its own self-inverse (`forwardSigma` / `reverseSigma` pointer aliases hide the one-or-two-layer split). The legacy `SymbolicSpace.layer` `PiLayer` attribute is gone; consumers use `model.symbolicSpace.sigma` directly.

<OutputSpace>

Maps symbols to final predictions via linear layers.

Parameter	Type	Default	Description
<code>nOutput</code>	int	sentinel	Output values. For XOR: 1.
<code>nDim</code>	int	1	Per-output dim.
<code>nVectors</code>	int	sentinel	Codebook size. Defaults to <code>nOutput</code> .
<code>invertible</code>	bool	false	Rarely set on <code>OutputSpace</code> .
<code>codebook</code>	bool	false	Codebook quantization of the output.
<code>nonlinear</code>	bool	false	<code>OutputSpace</code> is linear by default; rescales via <code>Data.denormalize</code> .

`LinearLayer` with (`bias`, `temp`) support for ergodic mode.

<WordSpace>

Grammar infrastructure (`SyntacticLayers`, `TruthLayer`). Lives outside `<architecture>` as a top-level sibling.

Parameter	Type	Default	Description
<code>useGrammar</code>	string	"none"	Grammar dispatch: <code>none</code> , <code>all</code> , legacy <code>thoughtFree</code> . Mutually exclusive with
<code>syntacticHiddenDim</code>	int	64	<code>SyntacticLayer</code> MLP hidden size.
<code>truthMaxEntries</code>	int	1024	<code>TruthLayer</code> LTM capacity.
<code>chartCompose</code>	bool	false	Chart-like pair selection in compose. <code>false</code> = legacy left-associative cascade
<code>chartCollapse</code>	string	"root"	Soft-superposition chart output: <code>root</code> (single root sentence node, other pos
<code>softChartCompose</code>	bool	false	Soft (differentiable) chart compose: CKY inside pass populates <code>current_r</code>
<code>chartTau</code>	float	–	Temperature for the soft-chart rule-score softmax.
<code>wMax</code>	int	–	Max span width considered by the chart.
<code>chartTopK</code>	int	0	Sparse-MoE rule gating: keep top-K rules per (cell, split). 0 disables.
<code>chartNoiseEps</code>	float	–	Gaussian noise epsilon for <code>chartTopK</code> gating.
<code>downwardGeneration</code>	bool	false	Emit a codebook head via <code>WordSpace.reconstruct()</code> and stash on <code>self</code> .
<code>language.interpretation</code>	float	0.5	Soft-interpolation weight between forward and generation directions.

The `<language><grammar>` block contains tier-scoped rules in `<compose>` and `<generate>` sections, each holding `<percepts>` / `<concepts>` / `<symbols>` sub-blocks of `<rule>` elements. Per the 2026-05-07 rollback, the grammar XML is the sole source of truth for which rule fires at each tier (no code-level `default_rule` fallback). Legacy bare-tier elements like `<P>P = sigma(P)</P>` directly under `<grammar>` are still accepted.

InvertibleLinearLayer (programmatic)

The LDU-factorised invertible linear primitive used internally by `SigmaLayer(invertible=True)` and `PiLayer(invertible=True)`. Not set via XML directly.

Parameter	Type	Default	Description
<code>naive</code>	bool	<code>false</code>	<code>false</code> : apply L, D, U sequentially via triangular solves (no W materialisation). <code>true</code> : mater
<code>stable</code>	bool	<code>false</code>	Clamps each diagonal entry d_i to magnitude $[\epsilon, 1]$ with sign preserved before the ergodic ble
<code>ergodic</code>	bool	<code>false</code>	Factor-level noise injection. Registers <code>noise_raw_L</code> , <code>noise_raw_U</code> , <code>noise_d</code> buffers; resample

Class renames and removals.

Old name	New name / status
<code>LULayer</code>	Renamed to <code>InvertibleLinearLayer</code>
<code>InvertibleLinearLayer</code> (SVD-based)	Removed; replaced by LDU factorisation
<code>InvertibleSigmaLayer</code>	Merged into <code>SigmaLayer(invertible=True)</code>
<code>InvertiblePiLayer</code>	Merged into <code>PiLayer(invertible=True)</code>

See `Architecture.md` for the LDU factorisation and factor-level noise injection.

Example: XOR Configuration

```

<model>
  <architecture>
    <modelType>embedding</modelType>

    <training>
      <numEpochs>400</numEpochs>
      <learningRate>0.01</learningRate>
      <autoload>false</autoload>
      <trainEmbedding>JOINT</trainEmbedding>
      <embeddingScale>0.05</embeddingScale>
    </training>
  </architecture>

  <InputSpace>
    <nVectors>8</nVectors>
    <nDim>10</nDim>
    <nOutput>8</nOutput>
    <nOutputDim>10</nOutputDim>
  </InputSpace>

  <PerceptualSpace>

```

```

    <nInput>8</nInput>
    <nVectors>1000</nVectors>
    <nDim>10</nDim>
    <nOutput>8</nOutput>
    <hasAttention>false</hasAttention>
    <invertible>true</invertible>
    <codebook>true</codebook>
  </PerceptualSpace>

  <ConceptualSpace>
    <nInput>8</nInput>
    <nVectors>8</nVectors>
    <nDim>10</nDim>
    <nOutput>8</nOutput>
    <invertible>true</invertible>
    <codebook>true</codebook>
    <bivectorOutput>true</bivectorOutput>
  </ConceptualSpace>

  <SymbolicSpace>
    <nInput>8</nInput>
    <nVectors>8</nVectors>
    <nDim>2</nDim>
    <nOutput>8</nOutput>
    <codebook>true</codebook>
    <bivectorOutput>true</bivectorOutput>
  </SymbolicSpace>

  <OutputSpace>
    <nOutput>1</nOutput>
    <nVectors>1</nVectors>
  </OutputSpace>
</model>

```

Everything not listed inherits from data/model.xml.

Example: Embedding / Chatbot Configuration

```

<model>
  <architecture>
    <modelType>embedding</modelType>
    <nWhere>2</nWhere>
    <nWhen>2</nWhen>
    <weightsPath>BasicModel.ckpt</weightsPath>
    <embeddingPath>BasicModel.kv</embeddingPath>

    <data>
      <shardDir>data/fineweb</shardDir>
      <minFrequency>0.00001</minFrequency>
    </data>

    <training>
      <numEpochs>2</numEpochs>
    </training>
  </architecture>
</model>

```

```

    <batchSize>128</batchSize>
    <reconstructionScale>0.1</reconstructionScale>
    <trainEmbedding>AR</trainEmbedding>
    <autosave>true</autosave>
    <checkpointEveryBatches>500</checkpointEveryBatches>
    <sentencePrediction>true</sentencePrediction>
    <sentenceContextWindow>5</sentenceContextWindow>
  </training>
</architecture>

  <!-- space definitions inherit from model.xml -->
</model>

```

Key points:

- `modelType=embedding` activates the embedding-based input pipeline alongside the neural model.
- `trainEmbedding=AR` (or `BACKPROP`) trains model layers with gradients flowing through the codebook.
- Three files partition model behaviour: **XML config** (architecture), **.kv** (codebook), **.ckpt** (model weights).
- `weightsPath` stores the neural model checkpoint; `embeddingPath` stores the word vectors.
- `minFrequency` gates vocabulary admission; words below it are buffered until they exceed the threshold.